

MASTER DEGREE IN ARTIFICIAL INTELLIGENCE -
UNIBO

Notes from the course of Machine Learning

A. A. 2022-2023

Contents

1	Classification	3
1.1	Decision Trees	5
1.1.1	Overfitting	11
1.1.2	Impurity functions	12
1.1.3	Building DT	13
1.2	Model selection: evaluation of a classifier	14
1.2.1	Error estimation	15
1.2.2	Statistical pruning of DT with error estimation	17
1.2.3	Other pruning techniques	18
1.2.4	Testing a classifier	18
1.2.5	Performance measures of a classifier	20
1.2.6	κ statistic	22
1.2.7	The cost of errors	23
1.3	Evaluation of a probabilistic classifier	24
1.3.1	Lift Chart	24
1.3.2	ROC curve	25
1.4	Statistical modeling - Naive Bayes Classifier	28
1.4.1	The Naive Bayes method	29
1.5	The Linear Perceptron	30
1.6	Support Vector Machines (SVM)	32
1.6.1	Maximum Margin Hyperplane	33
1.6.2	Non-linear class boundaries	35
1.6.3	SVM Complexity	36
1.6.4	Final remarks on SVM	36
1.7	Neural Networks	36
1.7.1	Multi-layer perceptron	37
1.7.2	Training the Neural Network	39
1.7.3	Computing the weight corrections	39
1.7.4	Training algorithm	40
1.7.5	Examples	42
1.8	K Nearest Neighbours Classifier	43
1.9	Multi-class classification	43
1.9.1	One-vs-one strategy	43
1.9.2	One-vs-rest strategy	44
1.10	Ensemble methods	44
1.11	Forest of randomized trees	46

1.12	Boosting	47
2	Regression - Forecasting continuous values	48
2.1	Linear Regression	48
3	The Data - Pre-processing and dissimilarities	52
3.1	Data pre-processing	52
3.1.1	Sampling	53
3.1.2	Principal Component Analysis (PCA)	55
3.2	Data type conversion	56
3.3	Similarity and dissimilarity	58
3.3.1	Similarity between binary vectors	61
3.3.2	Correlation	61
3.4	Data transformations	62
3.5	Imbalanced data in classification	64
4	Clustering	66
4.1	K-means	67
4.1.1	Distortion	70
4.1.2	Pros and cons of K-means	72
4.2	Evaluation of a clustering scheme	73
4.2.1	Cohesion and separation - prototype based	73
4.2.2	Silhouette	74
4.2.3	Choice of K	76
4.2.4	Supervised measures	76
4.3	Hierarchical clustering	78
4.3.1	Separation between clusters	79
4.3.2	Single linkage hierarchical clustering	79
4.3.3	Examples	80
4.4	Density based clustering	83
4.4.1	DBSCAN	83
4.4.2	Kernel Density Estimation	87
4.4.3	DENCLUE	87
4.5	Model based clustering	88
4.5.1	Gaussian Mixture	88
4.6	Final remarks	90
5	Association Rules	92
5.1	Introduction to Market basket Analysis	92
5.2	Frequent Itemset Generation	94
5.2.1	The Apriori principle	96
5.2.2	The Apriori algorithm	96
5.2.3	Factors Affecting Complexity	98
5.3	Rule Generation	98
5.3.1	Pattern evaluation	99
5.3.2	Statistical-based Measures	100
5.4	Multidimensional association rules	102

Chapter 1

Classification

The unsupervised mining techniques which can be in some way related to classification are usually known in literature with names different from classification. For this reason in this course with the term classification we will always mean **supervised classification**.

Consider the "soybean" example shown in the introduction: The data set X contains N individuals described by D attribute values each, we have also a Y vector which, for each individual x contains the class value $y(x)$. The class allows a finite set of different values (e.g. the diseases), say C , which are provided by experts (the supervisors). We want to learn how to guess the value of the $y(x)$ for individuals which have not been examined by the experts, so *we want to learn a classification model*.

What is a classification model?

It is an algorithm which, given an individual for which the class is not known, computes the class (classify new unknown data). The algorithm is parametrized in order to optimize the results, and according to the parameters, output for different input changes. Parameters are learned from the supervised dataset. A model/classifier is a decision function which, given a data element x whose class label $y(x)$ is unknown, makes a prediction as $M(x, \theta) = y(x)_{predicted}$, where θ is a set of values of the parameters of the decision function. The **learning process** for a given classifier M , given the dataset X and the set of supervised class labels Y , determines θ in order to reduce the prediction error as much as possible. For example see Figure 1.1.

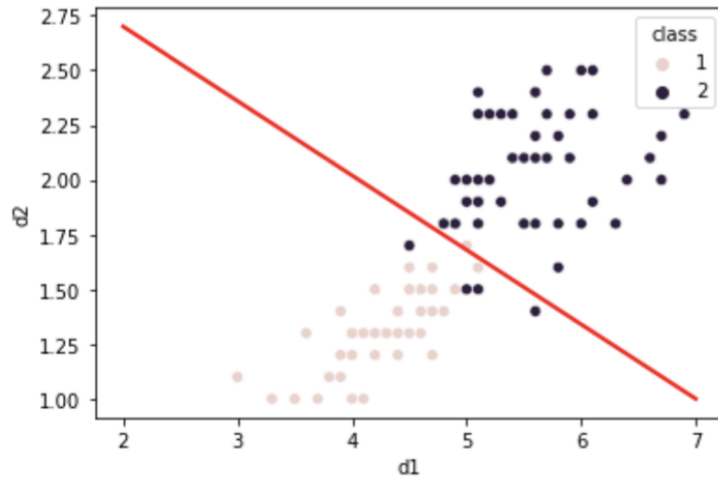


Figure 1.1: Example of supervised dataset with two dimensions, two classes. Use as decision function a straight line.

The model (decision function) of the Figure 1.1 makes some errors, even the best choice of parameters cannot avoid errors; this is what always happens. Different models can have different power to shatter the dataset into subsets with homogeneous classes. Given a dataset with N elements there are 2^N possible different learning problems; if a model M is able to shatter all the possible learning problems with N elements, we say that it has *Vapnik-Chervonenkis Dimension* equal to N .

We can distinguish two flavors for classification:

- **Crisp:** the classifier assigns to each individual one label.
- **Probabilistic:** the classifier assigns a probability for each of the possible labels.

A workflow for classification

1. Learning the model for the given set of classes
 - a training set is available, containing a number of individuals
 - for each individual the value of the class label is available (also named ground truth)
 - the training set should be representative as much as possible and should be obtained by a random process
 - the model is fit learning from data the best parameter setting
2. Estimate the accuracy of the model
 - a test set is available, for which the class labels are known

- the model is run by a classification algorithm to assign the labels to the individuals
- the labels assigned by the model are compared with the true ones, to estimate the accuracy

1.1 Decision Trees

A run-time classifier structured as a decision tree is a tree-shaped set of tests, the decision tree has inner nodes and leaf nodes (predict class of element x).

Given a set X of elements for which the class is known, grow a decision tree as follows:

1. if all the elements belong to class c or X is small generate a leaf node with label c .
2. otherwise choose a test based on a single attribute with two or more outcomes, make this test the root of a tree with one branch for each of the outcomes of the test, partition X into subsets corresponding to the outcomes and apply recursively the procedures to the subsets.

There are problems to solve:

1. which attribute should we test?
2. which kind of test?
3. what does it mean X is small, in order to choose if a leaf node is to be generated also if the class in X is not unique?

How much a pattern is interesting?

Supervised learning goal is to design an algorithm to finding interesting patterns, in order to forecast the values of an attribute given the values of other attributes (in our case, find patterns to guess the class). But how distinguish real patterns from illusions?, how choose useful patterns? There are several methods, one of them is based on **information theory**.

Consider the bit transmission example:

1. Given a variable with 4 possible values and a given probability distribution $P(A)=0.25$, $P(B)=0.25$, $P(C)=0.25$, $P(D)=0.25$
 - an observation of the data stream could return BAACBADCDADDDA...
 - the observation could be transmitted on a serial digital line with a two-bit coding $A=00$, $B=01$, $C=10$, $D=11$
 - the transmission will be 0100001001001110110011111100...
2. What if the probability distributions are uneven? $P(A)=0.5$, $P(B)=0.25$, $P(C)=0.125$, $P(D)=0.125$

- of course, the coding shown above is possible, requiring two bits per symbol
- is there a coding requiring only 1.75 bit per symbol, on the average?
A=0, B=10, C=110, D=111

In general given a source X with V possible values, with probability distribution $P(v_1)=p_1, P(v_2)=p_2, \dots, P(v_V)=p_V$, the best coding allows the transmission with an average number of bits given by

$$H(X) = -\sum_j p_j \log_2(p_j) \quad (1.1)$$

$H(X)$ is the *entropy* of the information source X .

High entropy means that the probabilities are mostly similar, low entropy means that some symbols have much higher probability, higher number of allowed symbols (i.e. of distinct values in an attribute) gives higher entropy.

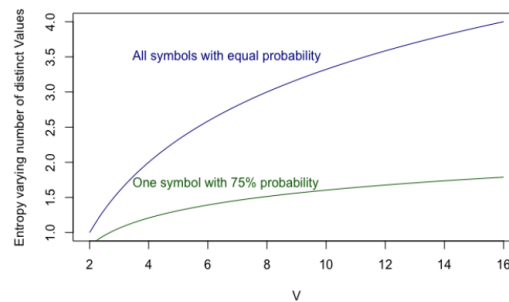


Figure 1.2

Note: in a binary source with symbol probabilities p and $1-p$ when p is 0 or 1 the entropy goes to 0, see at the Figure 1.3.

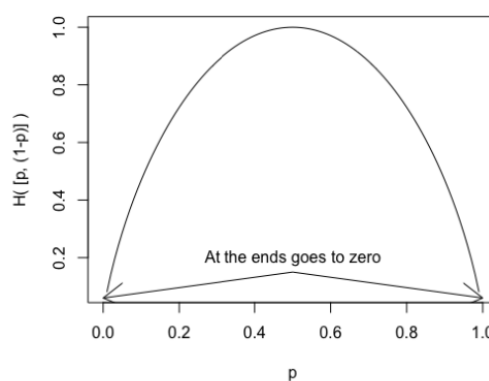


Figure 1.3: Entropy of a binary source

Entropy from Iris dataset

Let's consider only the fourth data column and the target, only the rows with classes 1 or 2, as in Figure 1.4.

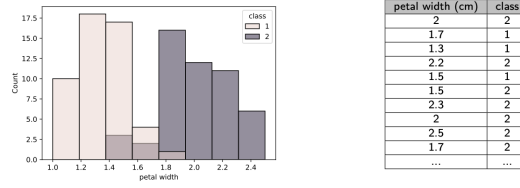


Figure 1.4: $N = 100$, $p_{class=1} = 0.5$, $p_{class=2} = 0.5$, $H_{class} = -(p_{class=1} * \log_2(p_{class=1}) + p_{class=2} * \log_2(p_{class=2})) = 1$

Splitting the dataset in two parts according to a threshold on a numeric attribute the entropy changes, and becomes the weighted sum of the entropies of the two parts, where the weights are the relative sizes of the two parts. Let $d \in D$ be a real-valued attribute, let t be a value of the domain of d , let c be the class attribute, so we define the entropy of c w.r.t. d with threshold t as $H(c|d : t) = H(c|d < t) * P(d < t) + H(c|d \geq t) * P(d \geq t)$.

It is the reduction of the entropy of a target class obtained with a split of the dataset based on a threshold for a given attribute. We define $IG(c|d : t) = H(c) - H(c|d : t)$ it is the information gain provided when we know if, for an individual, d exceeds the threshold t in order to forecast the class value. We define $IG(c|d) = \max_t IG(c|d : t)$.

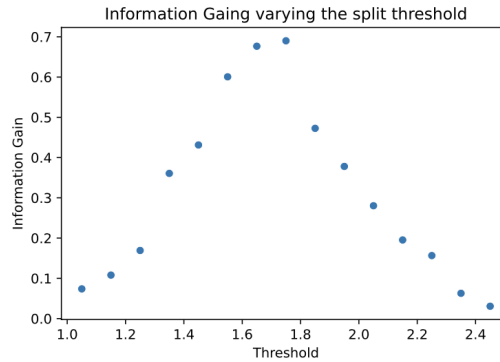


Figure 1.5:

How can we use the information gain?

Predict the probability of long life given some historical data on person characteristics and life style: $IG(LongLife|HairColor) = 0.01$, $IG(LongLife|Smoker) = 0.2$, $IG(LongLife|Gender) = 0.25$, $IG(LongLife|LastDigitSSN) = 0.00001$. Note that correlations between attributes is an important issue: it is not considered here.

Looking back at the question of which attribute should we test, we can say that we should test the attribute which guarantees the maximum IG for the class attribute in the current data set X , partition X according to the test outcomes and recursion on the partitioned data.

Train-Test splitting

The supervised data set will be split in (at least) two parts: the *training set* used to learn the model and the *test set* used to evaluate the learned model on fresh data. The split have to be done randomly, otherwise the assumption that the two parts have similar characteristics could not be true; the proportion of the split is decided by the experimenter.

Iris Dataset - Predicting attribute: Sepal Length

Best threshold: 5.75

Information Gain: $1.58 - (40 * 1.04 + 35 * 0.90)/75 = 0.61$

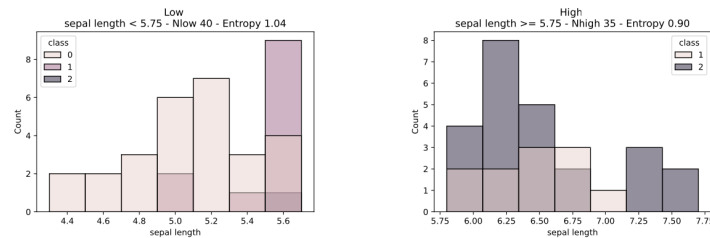


Figure 1.6

Best threshold: 3.25 – Information Gain: $1.58 - (52 * 1.44 + 23 * 0.56)/75 = 0.41$

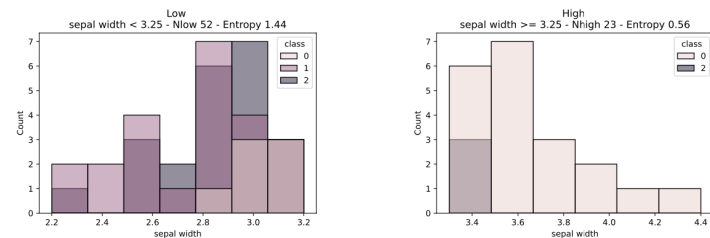


Figure 1.7

Best threshold: 2.6 – Information Gain: 0.94

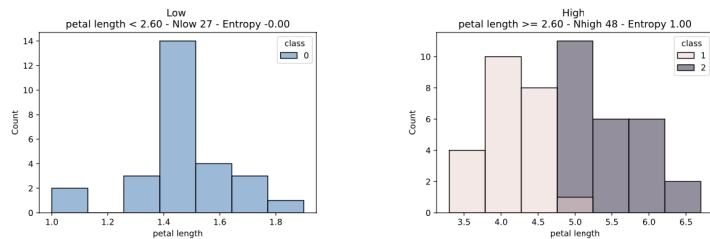


Figure 1.8

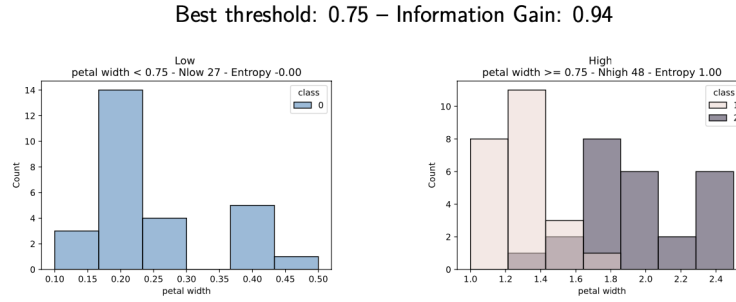


Figure 1.9:

One-stump decision

Choose the attribute giving the highest IG, partition the dataset according to the chosen attribute and choose as class label of each partition the majority.

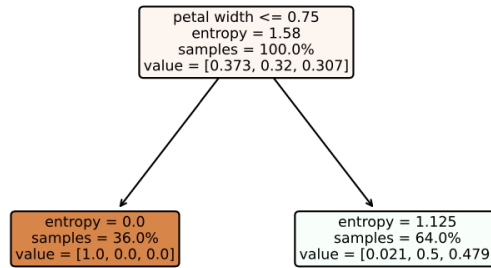


Figure 1.10

Build a new tree starting from each subset where the minority is non-empty

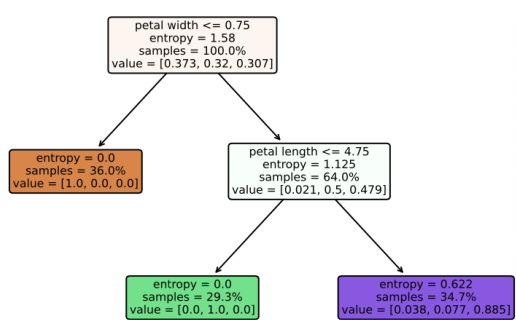


Figure 1.11

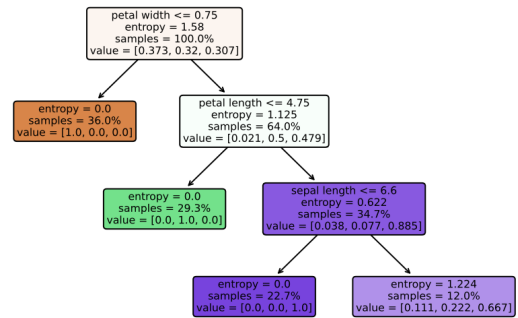


Figure 1.12

Observation: The weighted sum of the entropy of the descendant nodes is always smaller than the entropy in the ancestor node, even if one of the descendant has higher entropy (see Figure 1.12). Consider the three bottom right nodes (a=ancestor, ld=left descendant, rd=right descendant) $ent_a = 0.622 > (ent_{ld} * samp_{ld} + ent_{rd} * samp_{rd}) / samp_a = (0 * 22.7 + 1.224 * 12) / 34.7 = 0.39$.

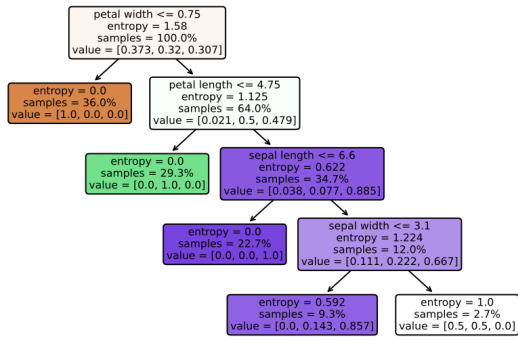


Figure 1.13

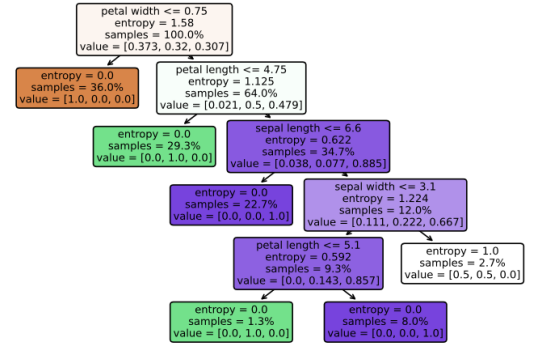


Figure 1.14

Most of the leaves are pure, recursion is impossible. One of the leaves is not pure, but no more tests are able to give positive information gain, recursion is impossible; it is labelled with the majority class, or, in case of tie, with one of the non-empty classes. We note that the error rate on the training set is 1.35%. 1 of the 75 examples in the training set is not correctly classified by the learned decision tree and it is one of the two items in the rightmost leaf.

Building a Decision Tree with binary splits

```

procedure BUILDTREE(dataset  $\mathcal{X}$ , node  $p$ )
  if all the class values of  $\mathcal{X}$  are  $c$  then
    return node  $p$  as a leaf, label of  $p$  is  $c$ 
  if no attribute can give a positive information gain in  $\mathcal{X}$  then
    say that the majority of elements in  $\mathcal{X}$  has class  $c$ 
    return node  $p$  as a leaf, label of  $p$  is  $c$ 
  find the attribute  $d$  and threshold  $t$  giving maximum information gain in  $\mathcal{X}$ 
  create two internal nodes descendant of  $p$ , say  $p_{left}$  and  $p_{right}$ 
  let  $\mathcal{X}_{left}$  = selection on  $\mathcal{X}$  with  $d < t$ 
  BUILDTREE( $\mathcal{X}_{left}$ ,  $p_{left}$ )
  let  $\mathcal{X}_{right}$  = selection on  $\mathcal{X}$  with  $d \geq t$ 
  BUILDTREE( $\mathcal{X}_{right}$ ,  $p_{right}$ )

```

Figure 1.15: Decision tree with binary split

Training and Test Set Error

Executing the generated decision tree on the training set itself, obviously hiding the class attribute, we can count the number of discrepancies between the true and the predicted class; this is the training set error. The model is not able to reproduce exactly the ground truth (Y) and the training error can be non-zero due to the limits of decision trees in general but also due to insufficient information in the predicting attribute. Referring to the 1.35% of error obtained from the Iris DT, is it interesting? It is probably the **lower limit** of the error we can expect when classifying new data, so we are much more interested to an **upper limit**, or to a more significant value. The test set error is more indicative of the expected behaviour with new data.

From the Iris dataset we can find an error on the Test set of 17.33%, why the test set error is so much worse?

1.1.1 Overfitting

Definition: overfitting happens when the learning is affected by noise. When a learning algorithm is affected by noise, the performance on the test set is (much) worse than that on the training set.

More formally a decision tree is a hypothesis of the relationship between the predictor attributes and the class. If h =hypothesis, $error_{train}(h)$ =error of the hypothesis on the training set, $error_X(h)$ = error of the hypothesis on the entire dataset. h overfits the training set if there is an alternative hypothesis h' such that $error_{train}(h) < error_{train}(h')$ and $error_X(h) > error_X(h')$.

Noise means that individuals in the test set can have *bad values in the predicting attributes and/or in the class label*. Another cause for overfitting is *lack of representative instances*.

A good hypothesis has low generalization error i.e. it works well on examples different from those used in training.

Remind: simple theories are preferable to complex ones, a long hypothesis that fits the data is more likely to be a coincidence. Pruning a decision tree is a way to simplify it. From the previous example of Iris dataset, after pruning, we can obtain TrainingSetError=4% and TestSetError=8%, the second is much lower than before.

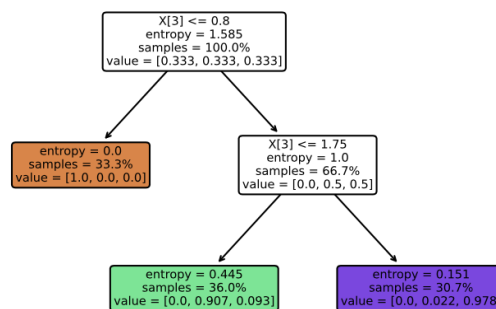


Figure 1.16: Iris classification with pruned tree

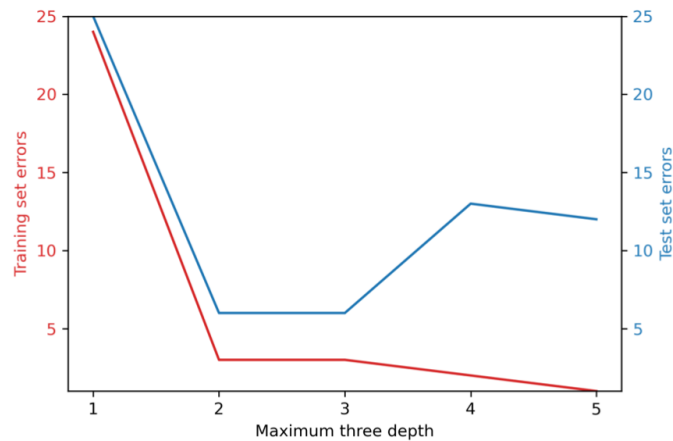


Figure 1.17: Training and test errors varying tree depth

More in general we have graphics from Figure 1.18. The best position in the graph is at the center of it.

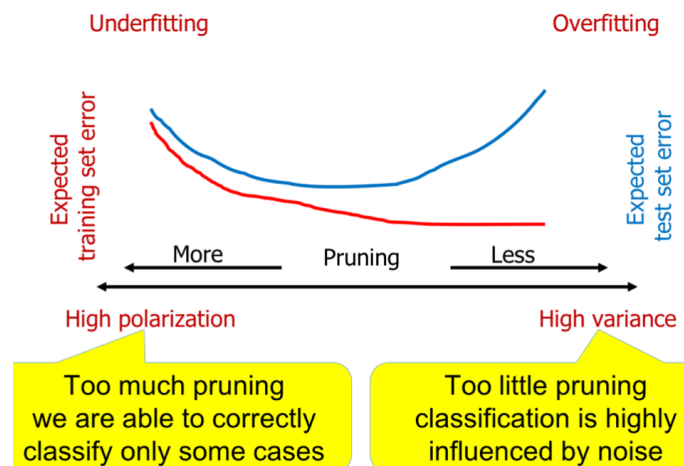


Figure 1.18: General effect of model simplification with pruning

Choice of the attribute to split the dataset

Looking for the split generating the maximum purity, we need a measure for the purity of a node: a node with two classes in the same proportion has low purity despite to a node with only one class that has highest purity.

1.1.2 Impurity functions

In advance to the information gain, already seen, we have other functions.

Gini Index

Consider a node p with C_p classes, Which is the frequency of the wrong classification in class j given by a random assignment based only on the class frequencies

in the current node? For class j the Gini Index is the total probability of wrong classification and is given by $\sum_j f_{p,j} * (1 - f_{p,j}) = \sum_j f_{p,j} - \sum_j f_{p,j}^2 = 1 - \sum_j f_{p,j}^2$, where $f_{p,j}$ is the frequency, $1 - f_{p,j}$ is the frequency of the other classes, and $f_{p,j} * (1 - f_{p,j})$ is the probability of wrong assignment. The maximum value is when all the records are uniformly distributed over all the classes: $1-1/C_p$, the minimum value is when all the records belong to the same class: 0.

When a node p is split into ds descendants, say $p_1, \dots, p_{ds}$, let $N_{p,i}$ and N_p be the number of records in the i -th descendant node and in the root, respectively, we choose the split giving the maximum reduction of the Gini Index

$$GINI_{split} = GINI_p - \sum_{i=1}^{ds} \frac{N_{p,i}}{N_p} GINI(p_i).$$

Misclassification error

If a node is a leaf, we find the highest label frequency; this frequency is the accuracy of the node and this label is the output of the node. The misclassification error is the complement to 1 of the accuracy. Since the most frequent class determines the node label, the complement is the error: the maximum value ($1-1/C_p$) is when all the records are uniformly distributed over all the classes, The minimum value (0) is when all the records belong to the same class. The choice of the split is done in the same way as for the Gini index $ME(p) = 1 - \max_j f_{p,j}$.

The behavior of ME is linear, therefore an error in the frequency is completely transferred into the impurity computation. Entropy and Gini have varying derivative, with the minimum around the center, they are more robust w.r.t. errors in the frequency, when the frequencies of the two.

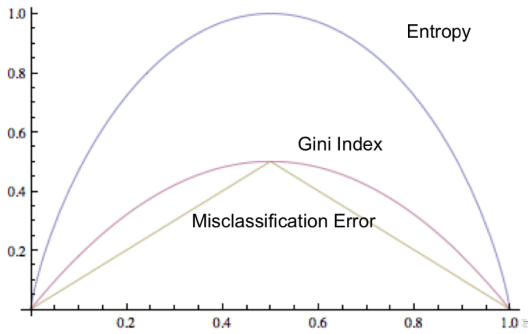


Figure 1.19: Comparison of the impurity functions for two classes with frequencies f and $1-f$

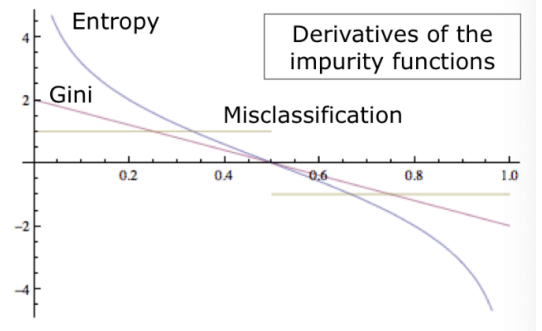


Figure 1.20: Derivatives of the impurity functions

1.1.3 Building DT

Exist several variants of algorithms for building DT, depending on tree construction strategy, partition strategy and pruning strategy. We could consider tests based on linear combinations of numeric attributes (multivariate tests) but this is

very complex.

Complexity of DT induction

If we have N instances and D attributes in X the tree height is $O(\log N)$ (increases slowly), each level of the tree requires the consideration of all the dataset (considering all the nodes) and all the attribute, so the overall cost is $O(DN \log N)$. In addition the binary split of numeric attributes costs $O(N \log N)$, without increment of complexity, pruning requires the consideration of each node, but nodes are $2N-1$, at most, it requires to consider globally all instances at each level, generating an additional $O(N \log N)$, which does not increase complexity.

Characteristic of DT induction

It is a non-parametric approach to build classification models, it does not require any assumption on the probability distributions of classes and attribute values (less assumption on data). Finding the best DT is NP-complete, the heuristic algorithms allow to find sub-optimal solutions in reasonable times. The run-time use of a DT to classify new instances is extremely efficient: $O(h)$, where h is the height of the tree. It is robust w.r.t. noise in the training set (i.e. wrong class labels), if the overfitting is avoided with appropriate pruning. In case of redundant attributes they do not cause any difficulty: if I have one attribute that brings more or less the same information of someone else I can choose randomly one of them and I get the same information. If one is chosen for a split, most likely the other will never provide a good increment of node purity, and will never be chosen. We can say in advance that the nodes at a high depth are easily irrelevant (and therefore pruned), due to the low number of training records they cover. In practice the impurity measure has low impact on the final result meanwhile the pruning strategy has high impact on the final result.

1.2 Model selection: evaluation of a classifier

We have to find a candidate for classification models but in general we have more than one model, and each one allows a number of different hyperparameters setting, raging in some space. Which of the available models for classification is the best one? Which of the available algorithms is the best one? Which is the best hyperparameters configuration? We could have that in principle a model A seems to be better than a model B but after tuning hyperparameters this may not be longer true. At the end we should evaluate the performances and select the best model and the best hyperparameters setting.

The evaluation is drive by performance measures and there exist several possible measures but the choice must be driven by the requirements of the specific problem at hand.

The Oil Slick example

We want detect oil slick (failures, illegal dumping) from satellite images, for early alarm.

Radar satellite images are made of dark regions whose size and shape depend on weather and sea conditions but manual detection by experts is definitely expensive and slow. Moreover there is a scarcity of training data (oil spills are, fortunately, rare). By the unbalanced nature of data the negative examples (non-spills) are pre-dominant over the positive ones.

An automatic hazard detection system has been developed and marketed: pre-selection of images for final manual processing, it is necessary a tradeoff between undetected spills and false alarms and the evaluation of performance guides the tradeoff.

The training set

In supervised learning the training set performance is **overoptimistic** (I overestimate the performance), thus we need a lower bound for performance obtained by independent tests. Supervised data are usually scarce, we need to balance the use of them between train, validation, to tune the parameter (sometimes it is omitted) an test. This allow to evaluate how much the theory fits the data and evaluate the cost generated by prediction errors.

Learning and evaluation

Empirically (and intuitively) the more training data we use the best performance we should expect; statistically, we should expect a larger covering of the situation that can occur when classifying new data. We must consider the effect of random changes. The evaluation is independent from the algorithm used to generate the model, it depends only on the generated predictions and the ground truth.

1.2.1 Error estimation

let us suppose that the test set is a good representation, on the average, of the entire dataset X (i.e. run-time); the relationship between the training set and X will be subject to probabilistic variability.

The evaluation can be done either at different levels: *general* on the whole performance of the classifier or *local* on the local performance of a component of the model, i.e. a node of a decision tree. If the test set error ratio is e , we should expect a run-time error $e \pm \text{confidence-interval}$.

Forecasting each element of the test set is like one experiment of a Bernoulli process: good prediction $\Rightarrow$ success (S), bad prediction $\Rightarrow$ error (E). It is the same as N independent binary random events of the same type. We define the **empirical frequency of error** as $e = E/N$ ($s = 1 - e$).

Deviations of the empirical frequency from the true frequency are due to noise, which is usually assumed to have a normal distribution around the true probability (for $N \geq 30$). We choose the **confidence level**, i.e. the probability that the true frequency of success (error) is below (above) the pessimistic frequency that we will compute.

The probability that a variable with normal distribution and given mean and deviance falls in the tail is:

$$P\left(z_{\alpha/2} \leq \frac{e - p}{\sqrt{p(1-p)/N}} \leq z_{1-\alpha/2}\right) = 1 - \alpha \quad (1.2)$$

With a little of algebra we can compute the pessimistic values for ps and pe:

$$s_{min} = \frac{1}{1 + \frac{1}{N}z^2} \left[s + \frac{1}{2N}z^2 - z\sqrt{\frac{1}{N}s(1-s) + \frac{1}{4N^2}z^2} \right] \quad (1.3)$$

$$e_{max} = \frac{1}{1 + \frac{1}{N}z^2} \left[e + \frac{1}{2N}z^2 + z\sqrt{\frac{1}{N}s(1-s) + \frac{1}{4N^2}z^2} \right] \quad (1.4)$$

If N is big then the quantity to add to e to achieve the pe are small ($N \rightarrow \infty \Rightarrow e \rightarrow 0$).

What is α ? α is the probability of a wrong estimate.

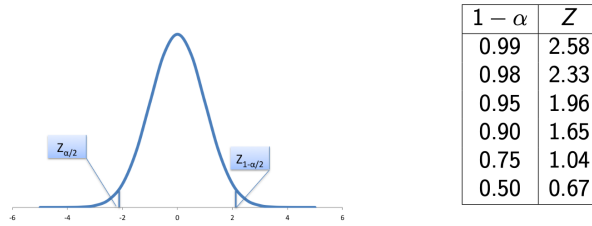


Figure 1.21: Confidence level in error estimation

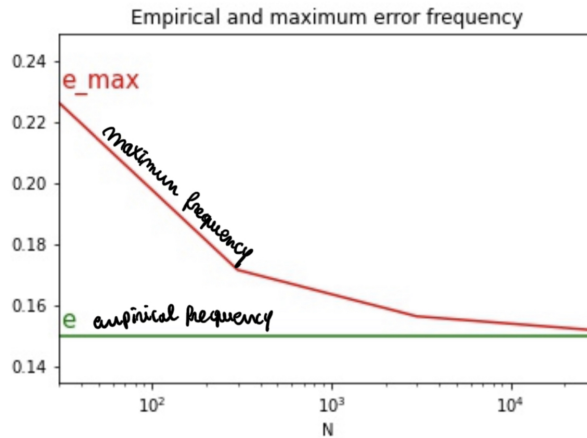
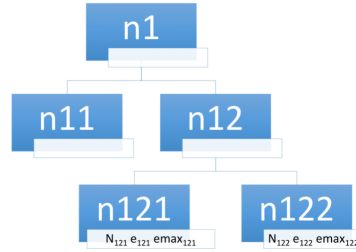


Figure 1.22: Confidence level in error estimation. Increasing N, with constant empirical frequency, the uncertainty for p narrows.

1.2.2 Statistical pruning of DT with error estimation

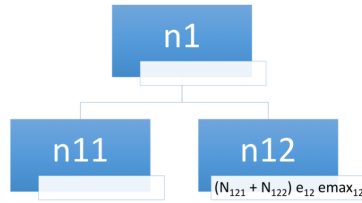
The C4.5 strategy:

- Consider a subtree near the leaves
- Compute its maximum error e_l ($e_{max}leaves$) as a weighted sum of the maximum error of the leaves
- Compute the maximum error e_r ($e_{max}root$) of the root of the subtree transformed into a leaf
- Prune if $e_r \leq e_l$
- With pruning, the error frequency increases, but the number of records in node also increases, therefore the maximum error can decrease



N_x = Records in node f_x = Error frequency in node e_x = Maximum error frequency in node

Figure 1.23: DT before pruning



$e_x - f_x$ increases as N_x decreases

Pruning is done if $(N_{121} + N_{122}) * e_{12} < N_{121} * e_{121} + N_{122} * e_{122}$

Figure 1.24: DT after pruning

1.2.3 Other pruning techniques

Scikit-Learn allows to adjust pruning with one or more of the hyperparameters below

- **max_depth** - the maximum depth allowed for the tree; it is a *horizontal cut*, pruning all the branches below a given depth
- **min_samples_split** - either the minimum absolute number of samples or the minimum fraction of samples (with respect to the entire population in the dataset) in a node to make a split, if the threshold is not exceeded the node becomes a leaf
- **min_samples_leaf** - the minimum number of samples (or fraction, as above) required to be at a leaf node
- **min_impurity_decrease** - a node will be split if this split induces a decrease of the impurity greater than or equal to this value; if the weighted sum of the descendant leaves do not decrease from the node under consideration more than this threshold then the node becomes a leaf

1.2.4 Testing a classifier

The error frequency is the simplest indicator of the quality of a classifier and it is defined as the sum of errors on any class divided by the number of tested records. From now on, for simplicity, we will use the empirical error frequencies (remember that in real cases the maximum error frequencies should be used instead).

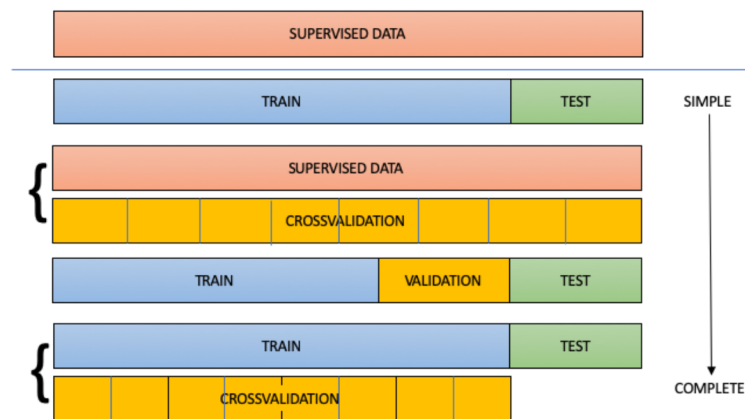
Accuracy and other more sophisticated indicators are used to: compare different classifiers or parameter settings and estimate the run-time performance we can expect, and therefore the cost of errors.

The hyperparameters

Every machine learning algorithm has one or more parameters than influence its behaviour, they are usually called hyperparameters. Several train/test loops are in general necessary to find the best set of values for the hyperparameters. It is crucial to obtain a highly reliable estimate of the run-time performance. Sometimes it is necessary to find the best compromise between the optimisation step and the quality of the result.

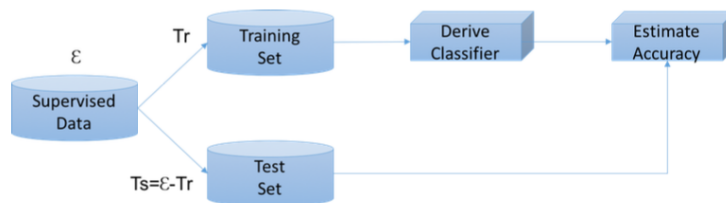
Testing strategies

Problem: in every step the data should be representative of the data that will be classified run-time. We have two possibilities to overcome this problem and are the Holdout and Cross Validation.



HOLDOUT

It splits data into training set and test set



A typical value of the training/test ratio is 2/1. The split should be as random as possible.

It may happen that the proportion of classes in the supervised dataset X is altered in the Training and Test sets, to prevent such cases the statistical sampling technique of *stratification* ensures the maintenance of the proportion of classes. In this setting, the test set is used to obtain an estimation of the performance measures with new data.

CROSS VALIDATION

- The training set is randomly partitioned into k subsets; if necessary uses stratified partitioning
- k iterations using one of the subsets for test and the others for training
- combine the result of tests
- generate the final model using the entire training set
- optimal use of the supervised data: each record is used $k-1$ times for training and once for testing
- typical value: $k=10$

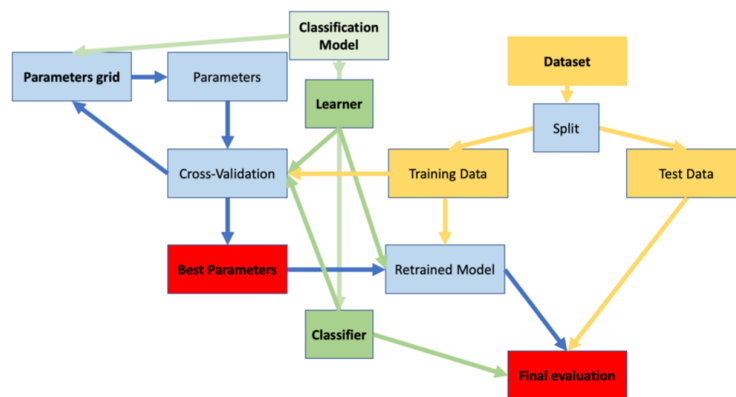
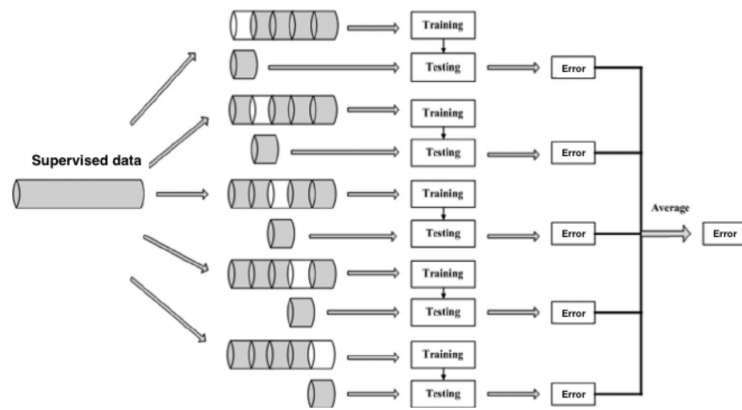


Figure 1.25: Cross-validation workflow

- ☹ The train/test loop is repeated k times
- ☺ The estimate of the performance is averaged on k runs
⇒ more reliability
- ☺ All the examples are used once for testing
- ☺ The final model is obtained using all the examples
⇒ best use of the examples

Figure 1.26: Cross Validation – pros and cons

1.2.5 Performance measures of a classifier

Consider a binary prediction, for simplicity: Positive/Negative

		Predicted class	
		P	N
True class	P	TP	FN
	N	FP	TN

Figure 1.27

We define the success-rate = accuracy = $\frac{TP+TN}{N_{test}}$ and the error-rate=1-success-rate.

Is accuracy enough? Is the accuracy the only performance indicator for a classifier? Other possible indicators are velocity (from the workin point of view), robustness w.r.t. noise (i.e. training data with bad class label), scalability (ability to work with large amount of data), interpretability (if I look in the internal nodes I see a clear path).

A classification error can have different consequences, depending on the class of the individual. when forecasting an illness a false positive can be less dangerous than a false negative.

A summary of measures

Precision = $\frac{TP}{TP+FP}$, the rate of true positives among the positive classifications.

Recall = $\frac{TP}{TP+FN}$, the rate of the positives that I can catch (a.k.a. Sensitivity).

Specificity = $\frac{TN}{TN+FP}$, the rate of the negatives that I can catch.

Accuracy = $sens \frac{pos}{N} + spec \frac{neg}{N}$, the weighted sum of sensitivity and specificity.

F₁score = $2 \frac{prec \cdot rec}{prec + rec}$, the armonic mean of precision and recall. $0 \leq F_1score \leq 1$. It increases when precision and recall are balanced.

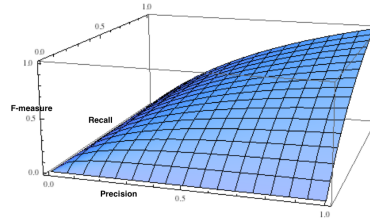


Figure 1.28: F1 score

Multi-dimensional case

The table in figure 1.27 is easily extended and is called **confusion matrix**. Each cell contains the number of test records of class i and predicted as class j. The numbers in the main diagonal are the “true” predictions. The accuracy is the sum of the elements in the main diagonal divided by the number of experiments.

Confusion matrix

Consider a confusion matrix with three classes, say a, b, c

		Predicted class			
		a	b	c	Total
True class	a	TP_a	FP_{a-b}	FP_{a-c}	T_a
	b	FP_{b-a}	TP_b	FP_{b-c}	T_b
	c	FP_{c-a}	FP_{c-b}	TP_c	T_c
	Total	P_a	P_b	P_c	N

Let be:

- T_x = true number of x labels in the dataset
- P_x = total number of x predictions by a given classifier, say $\bar{C}$
- TP_x = number of true predictions for class x given by classifier $\bar{C}$
- FP_{i-j} = number of false prediction for class i predicted as j

Then:

- accuracy = $\frac{\sum_i TP_i}{N}$
- precision_i = $\frac{TP_i}{P_i}$
- recall_i = $\frac{TP_i}{T_i}$

The definition of precision, recall and F_1 for each class is easily derived from the original one seen before. It is useful to give also global definitions, which combine the values of the single classes. There are several strategies for this combination, for example:

- **micro**: uses the global number of TP, FN, FP and calculates the measure directly: in practice, it is always the global accuracy. precision-micro=recall-micro= $\frac{\sum TP_i}{\sum TP_i + \sum FP_i}$.
- **weighted**: calculates the score for each class independently but when it adds together uses a weight that depends on the number of true labels of each class. precision-weighted= $\frac{\sum prec_i T_i}{N}$, recall-weighted= $\frac{\sum rec_i T_i}{N}$.
- **macro**: calculates the score separated by class but not using weights for the aggregation. precision-macro= $\frac{\sum prec_i}{\text{number of classes}}$, recall-macro= $\frac{\sum rec_i}{\text{number of classes}}$.

1.2.6 κ statistic

Consider the confusion matrix in figure of classifier $\bar{C}$ on a given dataset 1.30. We have 140 correct predictions. The true absolute frequency of classes is the right marginal 120,60,20. The predicted absolute frequency of classes is the bottom marginal 100,60,40.

		Predicted class			
		a	b	c	Total
True class	a	88	14	18	120
	b	10	40	10	60
	c	2	6	12	20
	Total	100	60	40	200

Figure 1.29

If now we consider a random classifier $R_{\bar{C}}$ producing the same proportion of classes as $\bar{C}$, the horizontal margin is the same. The rows have all the same proportion as the horizontal margin. We have that 82 predictions are exact by chance (the sum of the element in the main diagonal).

		Predicted class			
		a	b	c	Total
True class	a	60	36	24	120
	b	30	18	12	60
	c	10	6	4	20
	Total	100	60	40	200

Figure 1.30

The improvement of $\bar{C}$ over $R_{\bar{C}}$ is $140-82=58$. The improvement of the perfect classifier is $200-82=118$. We define $\kappa(\bar{C})$ the improvement of the classifier at hand w.r.t. the improvement of the perfect classifier and we obtain that $\kappa(\bar{C}) = 58/118 = 0.492$

The κ statistic evaluates the concordance between two classifications, in our case between the predicted and the true one. We have that:

- Probability of concordance: $Pr(c) = \frac{TP_a + TP_b + TP_c}{N}$
- Probability of random concordance: $Pr(r) = \frac{T_a P_a + T_b P_b + T_c P_c}{N^2}$
- κ is the ratio between the concordance exceeding the random component and the maximum surplus possible, $-1 \leq \kappa = \frac{Pr(c) - Pr(r)}{1 - Pr(r)} \leq 1$. If $\kappa = 1$ we have total agreement (it is the perfect classifier), $\kappa = -1$ total disagreement, there is a perfect swap between predictions and true labels, $\kappa = 0$ random agreement.

1.2.7 The cost of errors

Our decisions are driven by predictions. Bad predictions imply a cost.

Examples

- grant a loan to a person who turns out to be a bad payer costs more than denying a loan to a person that could be a good payer
- a false "oil spill" alarm is less expensive than an undetected spill
- a wrong "fault prediction" in an industrial plant is in general less expensive than an unexpected fault disabling the plant and creating damages
- in direct marketing, sending advertisement material without redemption is less harmful than the loss of business if a promising customer is ignored

Cost sensitive learning

Weight the errors. Alternatives:

1. Alterate the proportion of classes in the supervised data, duplicating the examples for which the classification error is higher. In this way, the classifier will become more able to classify the classes for which the classification error cost is higher. This solution is useful also when the classes are imbalanced, that is the frequencies of the class labels in X are not equal.

2. Some learning schemes allow to add weights to the instances. E.g. the `DecisionTreeClassifier` of Scikit-Learn has the hyperparameter `class-weight`: it allows to define a dictionary, with one key per distinct class, specifying the relative weight to be assigned to each class, the optimisation of the fitting will be adjusted accordingly.

1.3 Evaluation of a probabilistic classifier

Many classifiers produce, rather than a class label (crisp prediction), a tuple of probabilities, one for each possible class, (probabilistic, or soft prediction).

The adequacy of one output or the other depends on the application domain: when an immediate decision is required the crisp output is necessary, while when the classification is part of a process including several evaluation/action steps the probabilistic output can be more appropriated.

Crisp values sometimes hide probabilities. For example, when a leaf of a decision tree has non-zero counts for the minority classes, a less-than-one probability can be assigned on the basis of on the fraction of the elements belonging to the majority class. Since in this case it is quite common to have leaves with a small number of examples and/or minority classes with frequencies near to zero, smoothing techniques are used to adjust the probabilities (e.g. smoothing).

Probabilities can be converted to a crisp value with different techniques, depending on the number of classes (binary or multiclass):

- **binary** – set a *threshold* for the positive class.
- **multiclass** – output the class with the maximum probability

1.3.1 Lift Chart

It is used to evaluate various scenarios, depending on the application. Consider a dataset with 1000 positives and apply a probabilistic classification scheme. Sort all the classified elements for decreasing probability of positive class. Make a bi-dimensional chart with axes x = sample size, y = number of positives in sample. Only the rank is important, not the specific probability.

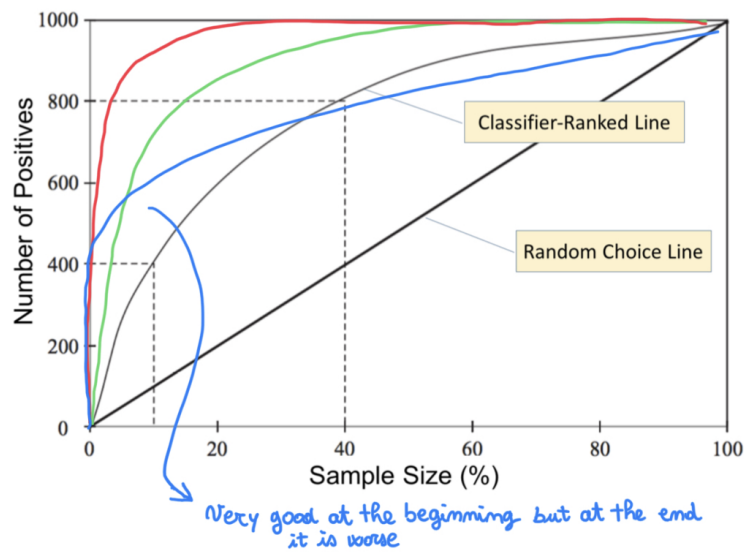


Figure 1.31: The straight line plots the number of positives obtained with a random choice of a sample of test data. The curve plots the number of positives obtained drawing a fraction of test data with decreasing probability. The larger the area between the two curves, the best the classification model.

1.3.2 ROC curve

Acronym for **Receiver Operator Characteristic**.

History: interpretation of radar signals during world war 2. It was necessary to distinguish enemy planes from allies. Tradeoff between hit rate and false alarm in a noisy channel. The noise can be such that the recognition of the transmission is altered. The noise alters the two levels according to a gaussian distribution. Problem: set the positive/negative threshold in order to maximize the tradeoff above, according to application-dependent requirements.

See at figure 1.32. With less noise the two gaussian curves are better separated. Decreasing the threshold value generates a movement towards right of the threshold point in the ROC curve, i.e. it increases both the rate of true positives and false positives caught. The area between the non-discrimination line and the ROC curve is a quality index of the line. The maximum area is the upper left triangle.

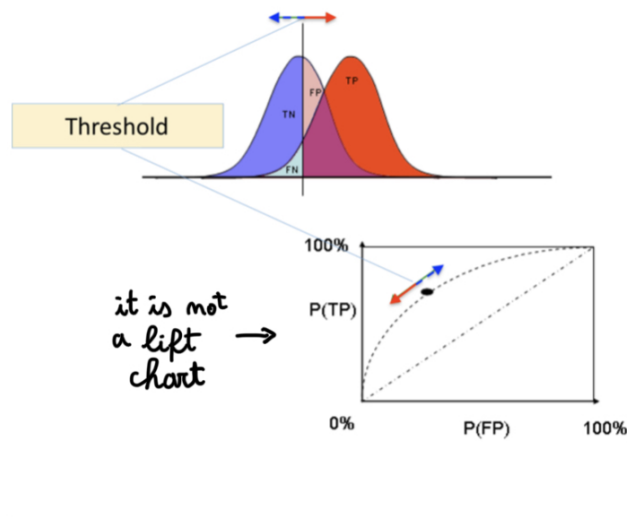


Image from Wikipedia

TN = blue + cyan = probability of a negative to be caught

FN = cyan = probability of a negative to be missed

FP = pink + purple = probability of a positive to be missed

TP = red + purple = probability of a positive to be caught

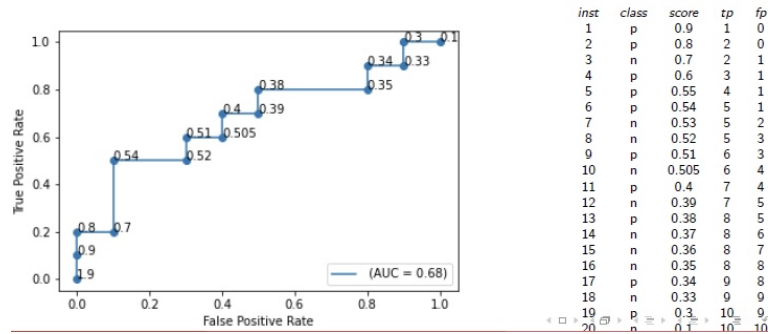
Figure 1.32: Axis y = Probability of Positive. Axis x = Probability of False Positive

ROC for soft classifier

The soft classifier can be converted into a crisp one by choosing a threshold $\Rightarrow$ predict positive if the probability of the test record exceeds the threshold. Varying the threshold the behavior of the classifier changes, by changing the ratio of TP and FP.

Threshold steps allow to track the ROC curve:

- sort the test elements by decreasing positive probability
- set the threshold to the highest probability, set TP and FP to zero
- repeat:
 - update the number of TP and FP with probability from the threshold to 1
 - draw a point in the curve
 - move to next top probability of positive
- end repeat



Decreasing the threshold FP and TP increase good performance is when TP increases faster than FP decreasing the threshold recall increases and precision decreases in an ROC curve the quality of the classifier is summarized by the Area Under Curve (AUC) the bigger, the better.

In the two figures below we have two extreme cases:

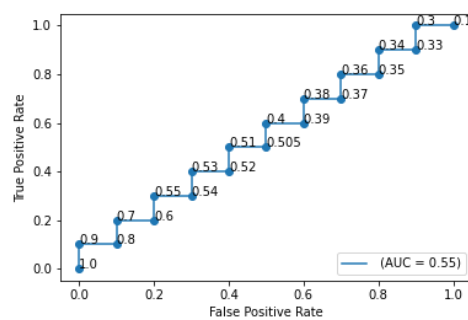


Figure 1.33: Random

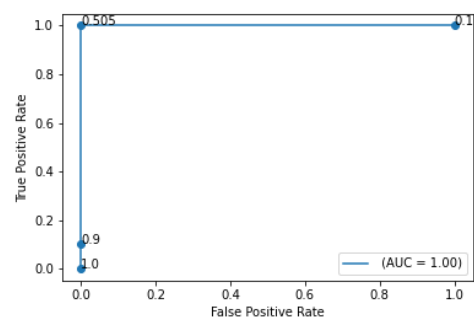
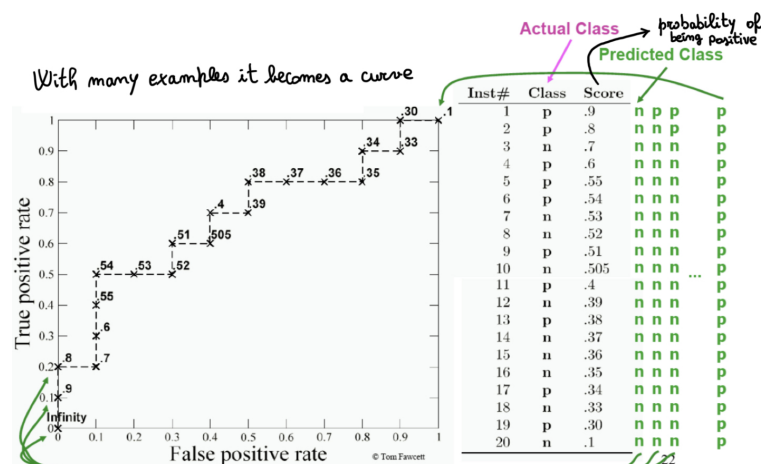
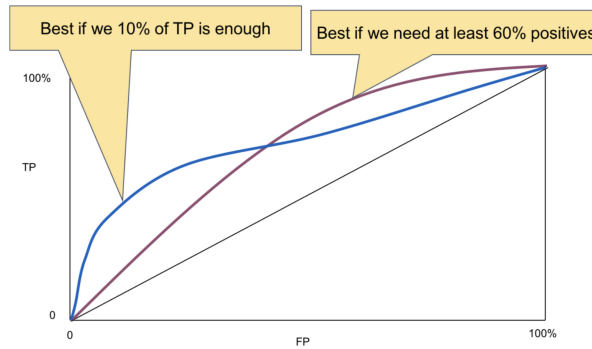


Figure 1.34: Perfect

There is not a best model. The goodness of a method depends on the data.





1.4 Statistical modeling - Naive Bayes Classifier

Called *naive* because it is based on the assumption that each attribute is independent from the others, given the class (very strong assumption, but the method still works): A NBC considers the contributions of all the classifiers. It estimates the probabilities with the frequencies, as usual.

$$Pr(d1 = v1, d2 = v2 | c = cx) = Pr(d1 = v1 | c = cx) \times Pr(d2 = v2 | c = cx) \quad (1.5)$$

The weather/play data

	Outlook		Temperature		Humidity		Windy		Play				
	yes	no	yes	no	yes	no	yes	no	yes	no			
sunny	2	3	hot	2	2	high	3	4	false	6	2	9	5
overcast	4	0	mild	4	2	normal	6	1	true	3	3		
rainy	3	2	cool	3	1								
sunny	2/9	3/5	hot	2/9	2/5	high	3/9	4/5	false	6/9	2/5	9/14	5/14
overcast	4/9	0/5	mild	4/9	2/5	normal	6/9	1/5	true	3/9	3/5		
rainy	3/9	2/5	cool	3/9	1/5								

Outlook: sunny, Temperature: cool, Humidity: high, Windy: true, Play: ?
Treat the five features and the overall likelihood that play is yes or no as equally important. They are independent pieces of evidence, the overall likelihood is obtained by multiplying the probabilities (i.e. the frequencies): likelihood of yes = $2/9 \times 3/9 \times 3/9 \times 3/9 \times 9/14 = 0.0053$, likelihood of no = $3/5 \times 1/5 \times 4/5 \times 3/5 \times 5/14 = 0.0206$.
Normalizing to 1 $Pr(yes) = \frac{0.0053}{0.0053+0.0206} = 20.5\%$, $Pr(no) = \frac{0.0206}{0.0053+0.0206} = 79.5\%$.

Bayes' Theorem

Given a hypothesis H and an evidence E that bears on that hypothesis

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

The hypothesis is the class, say c, the evidence is the tuple of values of the element to be classified. We can split the evidence into pieces, one per attribute, and, if the attributes are independent inside each class:

$$P(c|E) = \frac{P(E_1|c) \times P(E_2|c) \times P(E_3|c) \times P(E_4|c) \times P(c)}{P(E)}$$

1.4.1 The Naive Bayes method

- Compute the conditional probabilities from examples
- Apply the theorem
- The denominator is the same for all the classes, and

What if value v of attribute d never appears in the elements of class c ?

In this case $Pr(d = v|c) = 0$ and this makes the probability of the class for that evidence drop to zero. In practice, this case is quite common, in particular in a domain with many attributes and many distinct values. Which is the solution to this problem?

Values not represented in a class – Laplace smoothing

Let's start ignoring the details of the dataset, we consider only the value domains, and we know that for a given attribute d there are distinct values.

Then a simple guess for the frequency of each distinct value of d in each class is $1/V_d$. In this way we consider only the prior probabilities. We can smooth the computation of the posterior probabilities of values inside a class balancing it with the prior probability. Let's introduce:

- α smoothing parameter, typical value is 1
- $af_{d=vi,c}$ Absolute frequency of value vi in attribute d over class c
- V_d number of distinct values in attribute d over the dataset
- af_c Absolute frequency of class c in the dataset

We define the **smoothed frequency** $= sf_{d=vi,c} = \frac{af_{d=vi,c} + \alpha}{af_c + \alpha V_d}$. With $\alpha = 0$ we obtain the standard, unsmoothed formula, i.e. the relative frequency of the value of that attribute in that class. Higher values of α give more importance to the prior probabilities for the values of d w.r.t. the evidence given by the examples ($\alpha \rightarrow \infty \Rightarrow sf \rightarrow 1/V$).

When for some class the posterior probability of a value is zero, the prior probability still gives an approximate guess.

Smoothing is necessary when some frequencies are zero. Smoothing is useful especially when you do not have a lot of data and you need to account for some uncertainty. In other words, smoothing can reduce overfitting.

If we decide to apply smoothing, we must do it for the entire dataset: it is a hyperparameter for Naive Bayes.

Missing Values

Normally missing values are untreatable, and instances with missing values must be discarded. Instead, in a naive bayes model, missing values are just something we don't know probabilistically.

In train instance the record is simply not included in the frequency counts for that attribute.

In test instance the calculation of the likelihood simply omits this attribute.

Numeric Values

The method based on frequencies is inapplicable. So then we assume a Gaussian distribution. Instead of the fractions of counts, we compute, from the examples the mean μ and the variance σ of the values of each numeric attribute inside each class. For a given attribute and a given class, the distribution is supposed to be

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

The weather/play data with numeric values

Numbers of cases, fractions and descriptive statistics of the Weather/Play dataset.

Outlook			Temperature		Humidity		Windy		Play				
	yes	no	yes	no	yes	no	yes	no	yes	no			
sunny	2	3	83	85	86	85	false	6	2	9	5		
overcast	4	0	70	80	96	90	true	3	3				
rainy	3	2	68	65	80	70							
			64	72	65	95							
			69	71	70	91							
			75		80								
			75		70								
			72		90								
			81		75								
sunny	2/9	3/5	mean	73	74.6	mean	79.1	86.2	false	6/9	2/5	9/14	5/14
overcast	4/9	0/5	stdev	6.2	7.9	stdev	10.2	9.7	true	3/9	3/5		
rainy	3/9	2/5											

We are considering a yes outcome when the temperature is 66.
Plug the the value under consideration, the mean and the stdev in the gaussian probability density formula.

$$f(\text{temperature} = 66|\text{yes}) = \frac{1}{\sqrt{2\pi} \cdot 6.2} e^{-\frac{(66-73)^2}{2 \cdot 6.2^2}} = 0.0340$$

$$f(\text{humidity} = 90|\text{yes}) = 0.0221$$

Outlook: sunny, Temperature: 66, Humidity: 90, Windy: true, Play: ?

likelihood of yes = $2/9 \times 0.0340 \times 0.0221 \times 3/9 \times 9/14 = 0.000036$

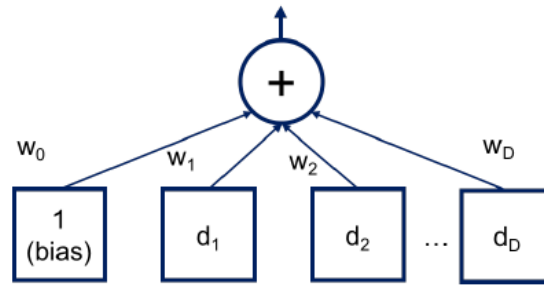
likelihood of no = $3/5 \times 0.0221 \times 0.0381 \times 3/5 \times 5/14 = 0.000108$

$$\Pr(\text{yes}) = \frac{0.000036}{0.000036 + 0.000108} = 25.0\%$$

$$\Pr(\text{no}) = \frac{0.000108}{0.000036 + 0.000108} = 75.0\%$$

1.5 The Linear Perceptron

Also called "Artificial Neuron". In practice it is a linear combination of weighted inputs.



Consider the task is to separate examples of two classes as in figure 1.35

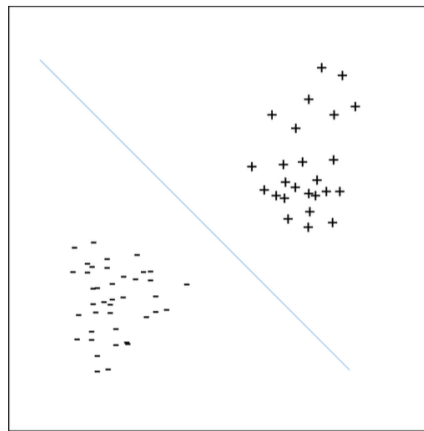


Figure 1.35

For a dataset with numeric attributes we want to learn a hyperplane such that all the positives lay on one side and all the negatives on the other.

How can we learn this hyperplane?

The hyperplane is described by a set of weights $w_0, \dots, w_D$ in a linear equation on the data attributes $x_0, \dots, x_D$; we add a fictitious attribute $x_0 = 1$ to allow a hyperplane that does not pass through the origin. There are either none or infinite such hyperplanes:

$$w_0 * x_0 + w_1 * x_1 + \dots + w_D * x_D \quad \begin{cases} > 0 \Rightarrow \text{positive} \\ < 0 \Rightarrow \text{negative} \end{cases}$$

ALGORITHM

```

set all weights to zero
while there are examples incorrectly classified do
  for each training instance x do
    if x is incorrectly classified then
      if class of x is positive then
        add the x data vector to the vector of weights
      else
        subtract the x data vector from the vector of weights

```

Figure 1.36: Learning the hyperplane

Linear perceptron convergence

Each change of weights moves the hyperplane towards the misclassified instance, consider the equation after the weight change for a positive instance x which was classified as negative

$$(w_0 + x_0) \times x_0 + (w_1 + x_1) \times x_1 + \dots + (w_D + x_D) \times x_D$$

The result of the equation is increased by a positive amount $x_0^2 + \dots + x_D^2$. Therefore the result will be less negative or, possibly, even positive.

Analogously for a negative instance which was classified as positive.

The corrections are incremental and can interfere with previous updates. The algorithm converges if the dataset is linearly separable, otherwise it does not terminate.

For practical applicability it is necessary to set an upper bound to the iterations.

1.6 Support Vector Machines (SVM)

It is a tool to solve the problems of **non separability**: if we cannot find the hyperplane. The data are not *linearly separable*, i.e. the boundary between classes is some hyper-surface more complex than a hyperplane.

A solution could be to give up the linearity.

$$w_1 * x_1^3 + w_2 * x_1^2 * x_2 + w_3 * x_1 * x_2^2 + w_4 * x_3^3$$

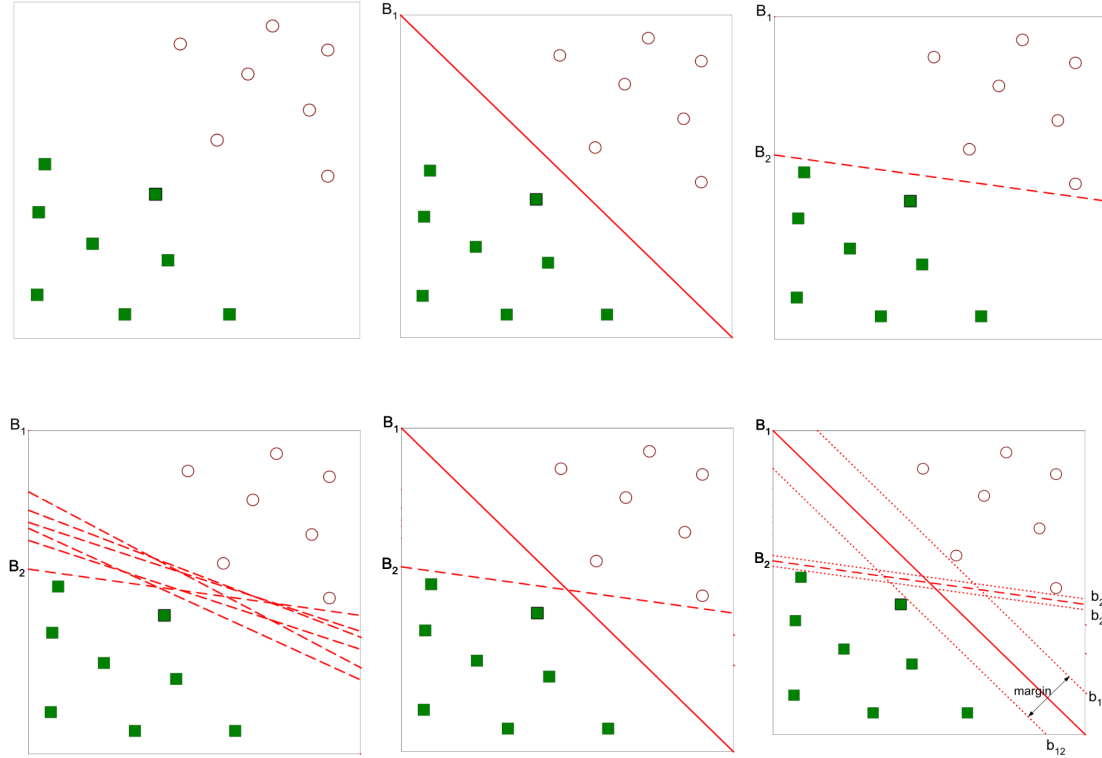
This method becomes soon intractable for any reasonable number of parameters (with 10 variables and limiting to factors with maximum order 5 we would need something like 2000 coefficient). Moreover, the method would be extremely **prone to overfitting**, if the number of parameters approaches the number of examples.

The solution of this problem is then Computational learning theory: new efficient separability of non-linear functions that use **kernel functions**. It does not guarantee to find the best solution but finds optimal parameters.

The search of a prediction function is modeled as a function estimation problem.

1.6.1 Maximum Margin Hyperplane

In the linear perceptron we have infinite solutions. What to do if we would find the best solution?



Given a solution (a line), we define the **margin** as the distance to a parallel solution that touches a data point. From the figures above we can intuitively notice that B_1 is better than B_2 because it has higher marginal; it means that it is more likely that different data fall in the correct side.

The linear perceptron accepts any hyperplane able to separate the classes of the training examples, but the **maximum margin hyperplane** gives the greatest separation between the classes. The best solution will so have the maximum margin.

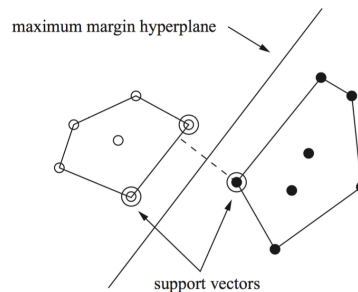


Figure 1.37

The convex hull of a set of points is the tightest enclosing convex polygon if the dataset is linearly separable the convex hulls of the classes do not intersect.

The maximum margin hyperplane is as far as possible from both the hulls. It is the perpendicular bisector of the shortest line connecting the hulls.

In general a subset of the points is sufficient to define the hull; those are the **support vectors** (circled in the figure 1.37). Support vectors are the elements of the training set which would change.

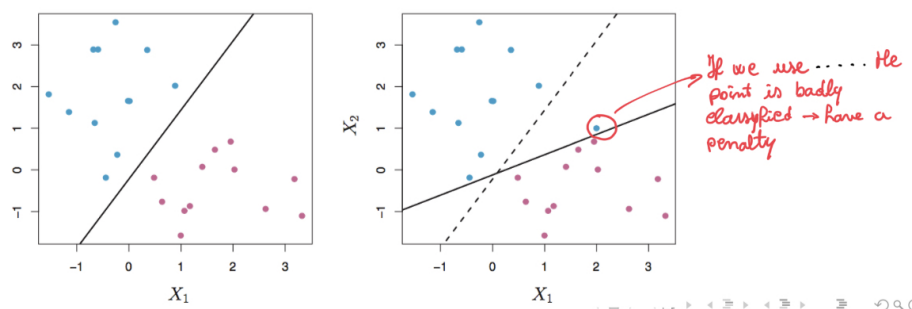
Finding the support vectors and the maximum margin hyperplane belongs to the well known class of constrained quadratic optimization problems:

$$\begin{aligned} & \max_{w_0, w_1, \dots, w_D} M \\ & \text{subject to } \sum_{j=1}^D w_j^2 = 1 \\ & c^x * (w_0 + w_1 x_1^x + \dots + w_D x_D^x) > M, \forall x = 1, \dots, N \end{aligned}$$

where the class c^x of example x is either -1 or 1 and M is the margin.

Soft margin

It is quite common that a separating hyperplane does not exist. Then we have to find an hyperplane which almost separate the classes. Disregard examples which generate a very narrow margin.

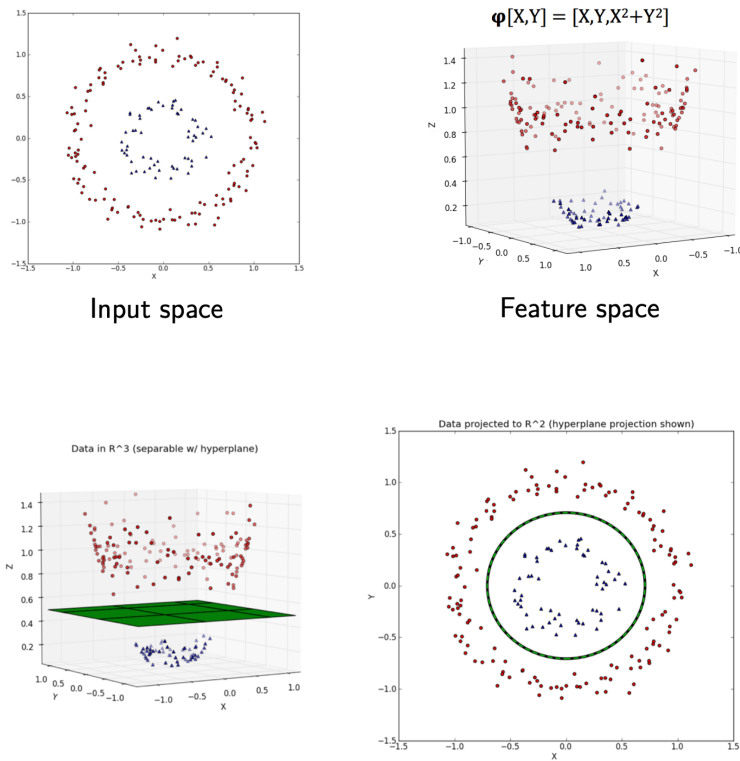


Advantages:

- Greater robustness to individual observations
- Better classification of most of the training observations
- Obtained by adding a constraint to the optimization problem expressed by a single numeric parameter, usually called C in the literature
 - C , the penalty parameter of the error term, controls the amount of overfitting
 - C tuning is critical

1.6.2 Non-linear class boundaries

The support vector method avoids the problem of overfitting. The nonlinearity of boundaries can be overcome with a non-linear mapping. The data are mapped into a new space, usually called feature space, such that a linear boundary in the feature space can correspond to a non-linear boundary in the original space. The feature space can have a number of dimensions higher than the original one.



The kernel trick

The separating hyperplane computation requires a series of dot product computations among the training data vectors.

Defining the mapping on the basis of a particular family of functions, called kernel functions, or simply kernels, the mapping does not need to be explicitly computed, and the computation is done in the input space. This avoids an increase in the complexity.

Some kernel functions:

linear	$\langle x, x' \rangle$
polynomial	$(\gamma \langle x, x' \rangle + r)^{dg}$
rbf	$\exp(-\gamma \ x - x'\ ^2)$
sigmoid	$\tanh(\langle x, x' \rangle + r)$

where γ , dg , r are parameters specified in the documentation of the learning tools.

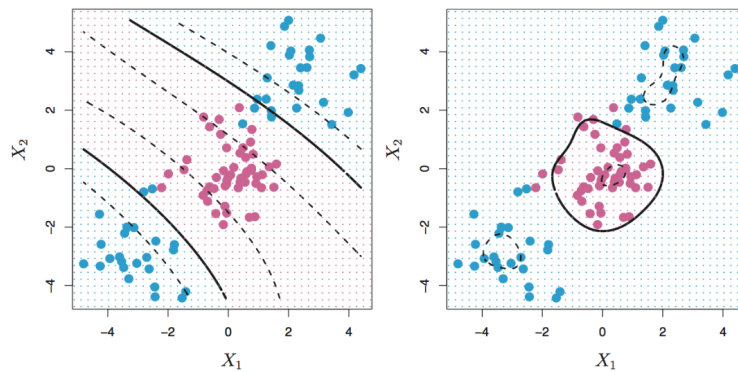


Figure 1.38: Decision boundaries for polynomial kernel of degree (left) and radial based kernel (RBF, right) In this case they seem to be both appropriate, but the generalization capabilities change, depending on where new data could be expected

1.6.3 SVM Complexity

The time complexity is mainly influenced by the efficiency of the optimization library.

The popular libSVM library scales from $O(D * N^2)$ to $O(D * N^3)$, depending on the effectiveness of data caching in the library, which is data dependent; in case of sparse data it is reduced. D is the number of features which becomes important when features grow.

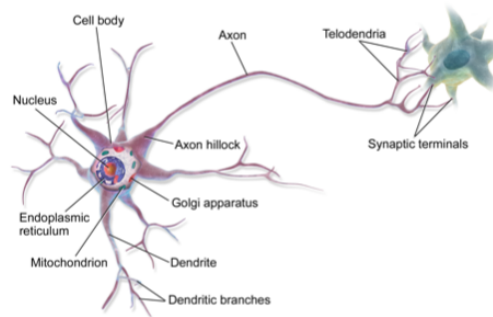
1.6.4 Final remarks on SVM

- Learning is in general slower than simpler methods, such as decision trees
- Tuning is necessary to set the parameters (not discussed here)
- The results can be very accurate, because subtle and complex decision boundaries can be obtained
- Explicitly based on a theoretical model of learning
- Are not affected by local minima
- Do not suffer from the curse of dimensionality: do not use any notion of **distance**
- SVMs do not directly provide probability estimates, these can be calculated using rather expensive estimation techniques (see `scikit-learn` documentation in References below)
 - nevertheless, SVM can produce a **confidence score** related to the distance of an example from the separation hyperplane

1.7 Neural Networks

What happens if we arrange many perceptron-like elements in a hierarchical structure? We can see each perceptron like a neuron (sum of different inputs $\rightarrow$ one output), so the term Neural Networks is inspired to the complex interconnections of neurons in animal brains.

A neuron is a signal processor with threshold and signal transmission from one neuron to another is weighted (weights change over time, also due to learning). Neural networks are another way to overcome the limit of linear decision boundary of linear perceptron.



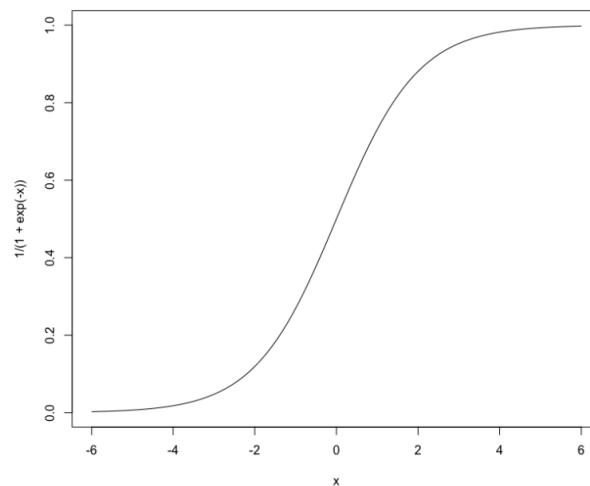
1.7.1 Multi-layer perceptron

The signals transmitted are modeled as real numbers. The threshold of the biological system is modeled as a mathematic function: continuous and differentiable, superiorly and inferiorly limited and that can be expressed in terms of the function itself (for example the sigmoid or the arctangent).

SIGMOID

Also called squashing function, it maps reals into $]0, 1[$. It is continuous, differentiable and non-linear.

$$\frac{1}{1 + e^{-x}}$$



Results with the linear perceptron were non satisfactory because of linearity, in addition to the problem of separability. We know that in a linear system $f(x_1 + x_2) = f(x_1) + f(x_2)$ and if x_2 is generated by noise, it is completely transferred to the output. In a non-linear system, in general, $f(x_1 + x_2) \neq f(x_1) + f(x_2)$.

Feed-forward multi-layered network

Inputs feed an input layer; one input node for each dimension in the training set; one input node for each dimension in the training set. Then the input layer feeds (with weights) a hidden layer and this feeds (with weights) in turn an output layer. The number of nodes in the hidden layer are a parameter of the network, instead the number of nodes in the output layer is related to number of different classes in the domain (one node if there are two classes, one node per class in the other cases). The more nodes in hidden layer the more complex is the separation surface, the more complex the decision function, the more powerful to divide different classes.

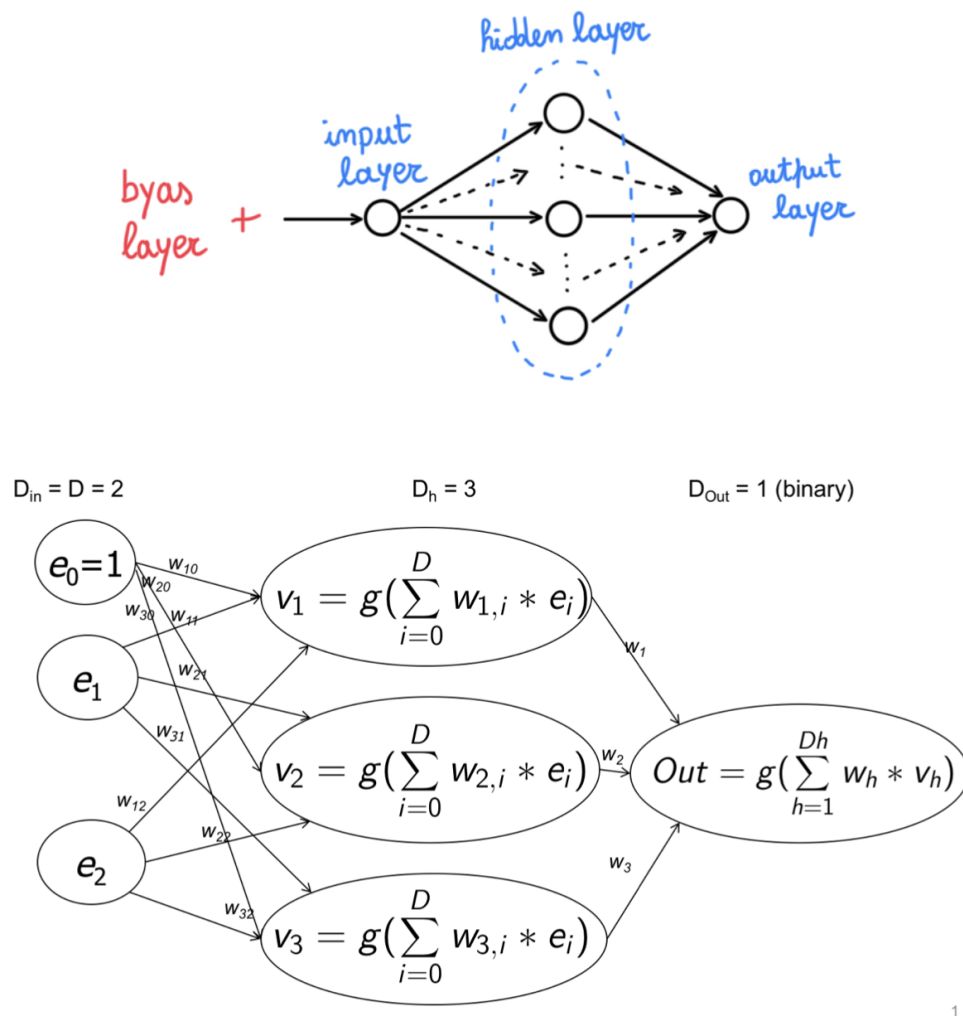


Figure 1.39: It is a binary classifier for data with two variables

Looking at figure 1.39:

- $g(\cdot)$ is the transfer function of the node, e.g. the sigmoid
- the unitary input e_0 is added for dealing with the bias, as in the case of the linear perceptron
- the weights are for each edge connecting two nodes

- feed-forward defines which oriented edges are present:
 - edges connect only a node in a layer to a node in the following layer: input to hidden and hidden to output
 - each node of one layer is connected to all the nodes of the following layer
- in this way the signal flows from the input to the output, without loops

1.7.2 Training the Neural Network

How can we learn the weight?

```

set all weights to random values
while termination condition is not satisfied do
  for each training instance  $x$  do
    feed the network with  $x$  and compute the output  $nn(x)$ 
    compute the weight corrections for  $nn(x) - x_{Out}$ 
    propagate back the weight corrections
  
```

In analogy with learning in the animal domain, the examples must repeatedly feed the network. The weights encode the knowledge given by the supervised examples.

The encoding is not easily understandable: it looks like a structured set of real numbers.

Convergence is not guaranteed.

Important issues are computing the weight corrections, the preparation of the training examples (standardize the attributes to have zero mean and unit variance) and the termination condition.

1.7.3 Computing the weight corrections

Let $\mathbf{x}$ and y be the input vector and the desired output of a node, respectively. Let $\mathbf{w}$ be the input weight vector of a node. The error is:

$$E(\mathbf{w}) = \frac{1}{2}(y - Transfer(\mathbf{w}, \mathbf{x}))^2$$

We can map the error function as a function of weights, see figure 1.40.

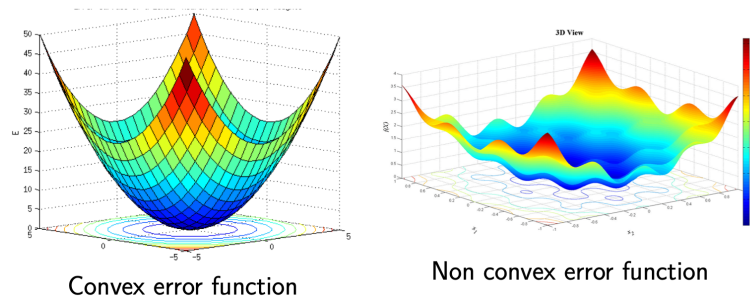


Figure 1.40: Error functions

We want to minimize the error so we need its gradient with respect to the weights. For the sigmoid we have

$$\frac{d\text{sgm}(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = (1 - \text{sgm}(x))\text{sgm}(x)$$

substitute x with w .

The weight is changed subtracting the partial derivative multiplied by a **learning rate** constant; this influences the convergence speed and can be adjusted as a tradeoff between speed and precision. The learning rate represents how much of the correction is transferred to the weights. The subtraction moves towards smaller errors.

The derivatives of the input weights of the nodes of a layer can be computed if the derivatives for the following layer are known.

What we do in practice is:

$$w_{ij} \leftarrow w_{ij} - \lambda \frac{\partial E(w)}{\partial w_{ij}}$$

1.7.4 Training algorithm

```

set all weights to random values
while termination condition is not satisfied do
  for each training instance  $x$  do
    1 – feed the network with  $x$  and compute the output  $nn(x)$ 
    2 – compute error prediction at output layer  $nn(x) - x_{Out}$ 
    3 – compute derivatives and weight corrections for output layer
    4 – compute derivatives and weight corrections for hidden layer
  Steps 1 and 2 are forward, steps 3 and 4 are backward

```

Learning modes

STOCHASTIC:

- each forward propagation is immediately followed by a weight update
- introduces some noise in the gradient descent process, since the gradient is computed from a single data point
- reduces the chance of getting stucked in a local minimum
- good for online learning

BATCH:

Many propagations occur before updating the weights, accumulating errors over the samples within a batch. Generally yields faster and stable descent towards the local minimum, since the update is performed in the direction of the average error.

A learning round over all the samples of the network is called epoch. In general, after each epoch the network classification capability will be improved so several epochs will be necessary. After each epoch the starting weights will be different.

Design choices

How to design the network?

The structure of input and output layers is determined by the domain (the training set). The number of nodes in the hidden layer can be changed. Also the learning rate can be changed in different epochs: in the beginning a higher learning rate can push faster towards the desired direction, in later epochs a lower learning rate can push more precisely towards a minimum.

Stop criteria

All the weight updates in the epoch have been small, when the classification error rate goes below a predefined target a timeout condition is reached.

Risks

Local minima are possible, as usual in gradient tracking methods.

Overfitting is possible, if the network is too complex w.r.t. the complexity of the decision problem.

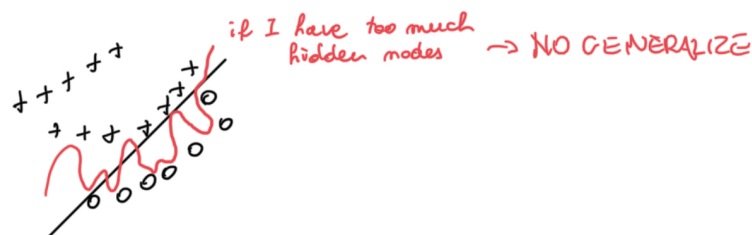


Figure 1.41: Example of overfitting (red curve)

Regularization

How to overcome the problem of overfitting?

There are many techniques used in many machine learning to improve the generalisation capabilities of a model. They Modify the performance function, which is normally chosen to be the sum of squares of the network errors on the training set. In essence:

- improvement of performance is obtained by reducing a loss function (i.e. the sum of squared errors)
- regularisation corrects the loss function in order to smooth the fitting to the data
- the amount of regularisation must be tuned

1.7.5 Examples

- Recognition of characters with noise
- The images are 7x5 arrays of floating points
 - before noise -1 for black, 1 for white
- 35 input nodes
- up to 60 hidden nodes
- 26 output nodes
 - only one at a time should give a value near 1, all the others should be near 0
 - a good alternative could be a 5 bit coding

We have 60 hidden nodes $\sim$ 2 per class and 1 output for each character.

The error rate is:

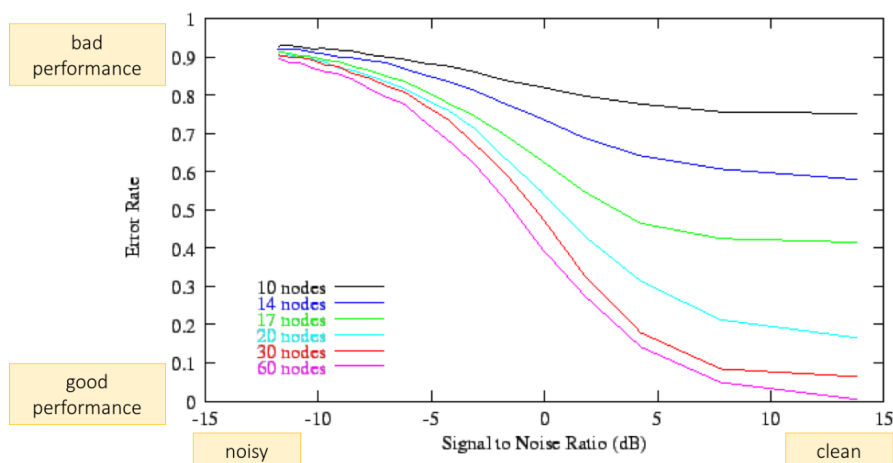


Figure 1.42: The original input was made by bits -1 and 1, due to noise they change. Notice that the choice of 60 nodes is based on this graph.

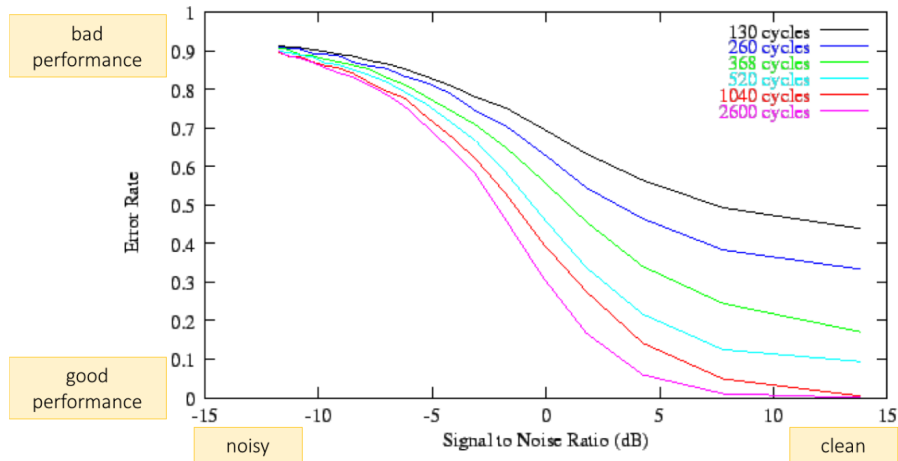


Figure 1.43: With the choice of 60 hidden nodes we obtain this graph. Cycles stands for epochs.

1.8 K Nearest Neighbours Classifier

The methods seen encode the learn in a model, KNN keeps all the training data which constitute the model itself.

Future predictions by computing the similarity between the new sample and each training instance; can use any similarity function. Picks the K entries in the database which are closest to the new data point and the majority vote. The main parameters are the number of neighbours to check and the metric used to compute the distances (the Mahalanobis distance has good performance).

1.9 Multi-class classification

Several classifiers (e.g. SVM and linear perceptron), generate a binary classification model.

We have two ways to deal with multi-class classification:

- transform the training algorithm and the model (sometimes at the expenses of an increased size of the problem)
- use a set of binary classifiers and combine the results (at the expenses of an increased number of problems to solve). In this case we have two strategies, the **one-vs-one** and the **one-vs-all**.

1.9.1 One-vs-one strategy

Consider all the possible pairs of classes and generate a binary classifier for each pair, $C(C-1)/2$ pairs.

each binary problem considers only the examples of the two selected classes.

At prediction time apply a voting scheme: an unseen example is submitted to the $C(C-1)/2$ binary classifiers, each winning class receives a +1 and the class with the highest number of +1 wins.

Consider three classes A, B, C. I combine two by two and I obtain 3 different classifiers

PAIRS
 $A-B \rightarrow A$
 $A-C \rightarrow A$
 $B-C \rightarrow B \text{ or } C \text{ randomly}$

So I expect to have an A.

1.9.2 One-vs-rest strategy

Consider C binary problems where class c is a positive example, and all the others are negatives. Build C binary classifiers.

At prediction time apply a voting scheme: an unseen example is submitted to the C binary classifiers, obtaining a confidence score, the confidences are combined and the class with the highest global score is chosen.

Consider three classes A, B, C and we obtain

$A - \neg A \rightarrow A$
 $B - \neg B \rightarrow \neg B$
 $C - \neg C \rightarrow \neg C$

So I expect to have an A.

OVO vs OVR

OVO requires solving a higher number of problems, even if they are of smaller size.

OVR tends to be intrinsically unbalanced. If the classes are evenly distributed in the examples, each classifier has a proportion positive to negative 1 to C-1.

1.10 Ensemble methods

Instead of using a single classifier we can make work together more of them, so we can train a set of base classifiers and the final prediction is obtained taking the

votes of the base classifiers In general ensemble methods tend to perform better than a single classifier.

Why ensemble methods should perform better?

Let us consider 25 binary classifiers, each of which has error rate $\epsilon = 0.35$, which in principle should be a poor classifier. The ensemble classifier output is the majority of the predictions (13 or more). If the base classifiers are equal, the ensemble error rate is still ϵ but if they are independent, i.e. their errors are uncorrelated, then the ensemble will be wrong only when the majority of the base classifier is wrong: in this case we can compute the error through **binomial coefficients** as $\epsilon_{ensemble} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1 - \epsilon)^{25-i} = 0.06$. What happens in general is described in Figure 1.44. You can notice that for base classifier error more than 0.5 it is convenient to have an ensemble classifier.

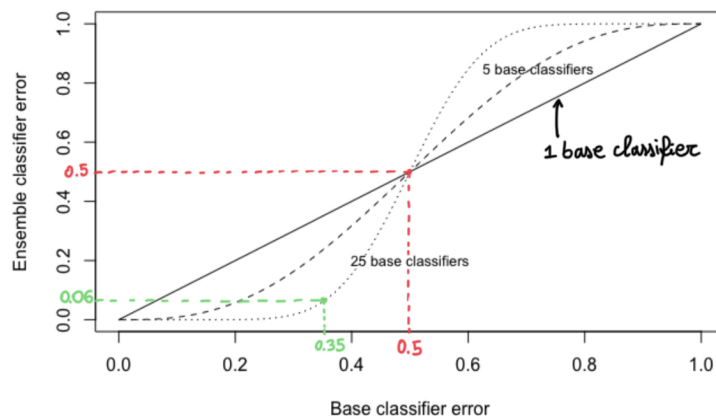


Figure 1.44: Comparison between error of the base classifiers and error of the ensemble classifier

Ensemble methods are useful if the base classifiers are independent and the performance of the base classifier is better than random choice.

Methods for ensemble classifier

How can we have a sort of independence among the different classifiers?

- **Manipulating the training set:** Data are resampled according to some sampling strategy.

Bagging: repeatedly samples with replacement according to a uniform probability distribution. Replacing allows at each iteration to have the same probability to pickup one value.

Boosting: iteratively changes the distribution of training examples so that the base classifier focus on examples which are hard to classify.

Adaboost: the importance of each base classifier depends on its error rate

- **Manipulating the input features:** subset of input features can be chosen either random or according to suggestions from domain experts (we need of a large number of features). For example *Random forest* uses decision trees as base classifiers and frequently produces very good results.
- **Manipulating the class labels:** It is useful when the number of classes is high. For each base classifier, randomly partition class labels into two subsets, say A1, A2 and re-label the dataset, then train binary classifiers with respect to the two classes. At testing time, when a subset is selected all the classes it includes receive a "vote" and the class with the top score will win.
- **Error-correcting output coding**

1.11 Forest of randomized trees

Perturb-and-combine techniques specifically designed for trees. A diverse set of classifiers is created by introducing randomness in the classifier construction:

- the classifiers must be independent
- each tree in the ensemble is built from a sample drawn with replacement from the training set
- furthermore, when splitting each node during the construction of a tree, the best split is found either from all input features or a random subset of size max-features:

The prediction of the ensemble is given as the averaged prediction of the individual classifiers.

Bias-vs-Variance tradeoff

Bias is the simplifying assumptions made by the model to make the target function easier to approximate.

Variance is the amount that the estimate of the target function will change, given different training data.

Bias-variance trade-off is the sweet spot where our machine model performs between the errors introduced by the bias and the variance.

Random Forest

The purpose of the two sources of randomness is to decrease the variance of the forest estimator.

Individual decision trees typically exhibit high variance and tend to overfit, the injected randomness in forests yield decision trees with somewhat decoupled prediction errors. By taking an average of those predictions, some errors can cancel out. Random forests achieve a reduced variance by combining diverse trees, sometimes at the cost of a slight increase in bias.

In practice the variance reduction is often significant hence yielding an overall better model.

1.12 Boosting

A weight is associated to each training instance and different classifiers are trained with those weights; the weights are modified iteratively according to classifier performance. Fit a sequence of weak learners on repeatedly modified versions of the data. the predictions from all of them are then combined through a weighted majority vote (or sum) to produce the final prediction. the data modifications at each so-called boosting iteration consist of applying and modifying weights to each of the training samples. initially, the weights $w_1, w_2, \dots, w_N$ are all set to $1/N$, so that the first step simply trains a weak learner on the original data and for each successive iteration, the sample weights are individually modified and the learning algorithm is reapplied to the reweighted data: at a given iteration, the training examples that were incorrectly predicted by the boosted model induced at the previous step have their weights increased, whereas the weights are decreased for those that were predicted correctly. As iterations proceed, examples that are difficult to predict receive ever-increasing influence; each subsequent weak learner is thereby forced to concentrate on the examples that are missed by the previous ones in the sequence

Chapter 2

Regression - Forecasting continuous values

Training regression is something different from training classification. Also this is a supervised task, with the difference that the target variable is numeric. The scope is the same: minimize the error of the prediction with respect to the target but here to compute the error we don't have to just count the frequency of bad predictions.

2.1 Linear Regression

Let's consider a data set X with N rows and D columns, be x_i a D dimensional data element, let call $\bar{y}$ the response vector with N values y_i and be w a D -dimensional vector of coefficients that needs to be learned. We model the dependence of each response value y_i from the corresponding independent variables x_i as $y_i \approx w^T x_i, \forall i \in [1, \dots, N]$, such that the error of modelling is minimised. See for an instance of regression Figure 2.1.

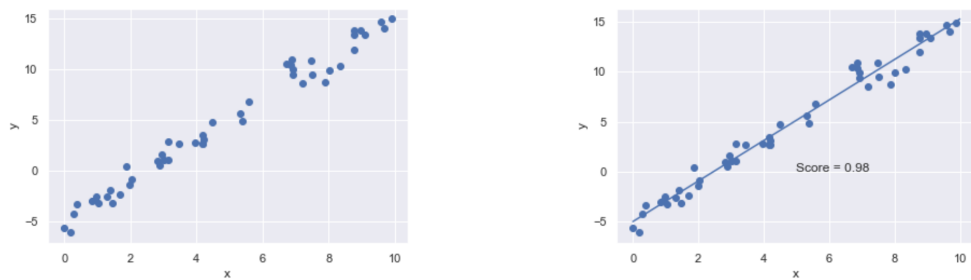


Figure 2.1: One-dimensional data x and response variable y (on lhs) and Regression line: $y=2.03x-5.00$ (on rhs)

OBJECTIVE FUNCTION AND MINIMISATION

$$O = \sum_{i=1}^N (w^T x_i - y_i)^2 = \|X_w^T - y\|^2 = (X_w^T - y)^T \cdot (X_w^T - y) \quad (2.1)$$

Now we want to minimize it so the gradient of O with respect to w is $2X^T(X_w^t - y)$. Constraining the gradient to 0 we obtain the optimisation condition $X^T X_w^T = X^T y$. If the symmetric matrix $X^T X$ is invertible the solution can be derived as $w = (X^T X)^{-1} X^T y$ and the forecast is given by $y^f = X \cdot x^t$.

At this time the initial problem is transformed into a matrix calculus problem. We could have some issues related to matrix calculus if $X^T X$ is not invertible.

Quality of the fitting - R^2

What we are interested in is the quality of our results, for this reason we define:

- *Mean of the observed data:* $y^{avg} = \frac{1}{N} \sum_i y_i$
- *Sum of squared residuals:* $SS_{res} = \sum_i (y_i - y_i^f)^2$;
- *Total sum of squares:* $SS_{tot} = \sum_i (y_i - y^{avg})^2$

Since the SS_{res} takes into account the spread of the data we need of a measure of our precision described by a constant, in order to compare data with different size. So we define the **coefficient of determination** $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$, which is an absolute number.

The R^2 compares the fit of the chosen model with that of a horizontal straight line. With perfect fitting the numerator of the second term is zero and $R^2 = 1$, if otherwise the model does not follow the trend of the data the numerator of the second term can reach or exceed the denominator, and R^2 can also be negative.

Overfitting and Regularisation

In presence of high number of features overfitting is possible, so performance on test data becomes much worse. Regularisation reduces the influence of less interesting attributes and therefore reduces overfitting.

What happens if we have data that cannot be fitted with a straight line? For example see the Figure 2.2 and Figure 2.3 (underfitting).

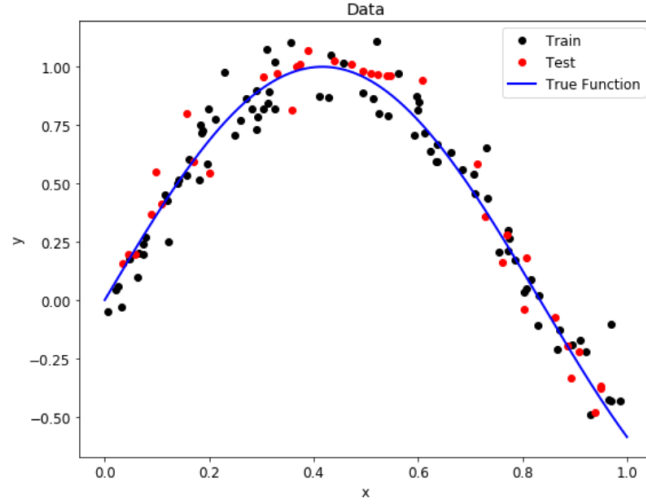


Figure 2.2: Example of data

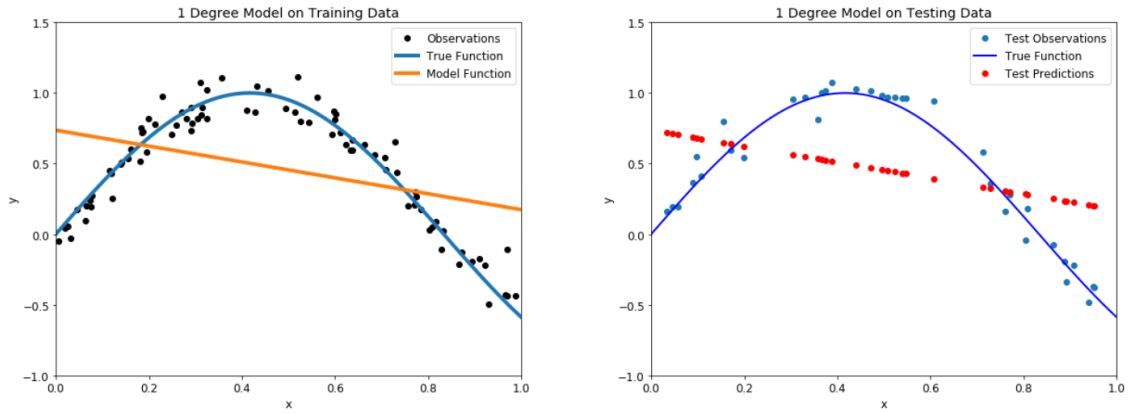


Figure 2.3: This is an example of underfitting

For the reason that we can understand looking at the figures above a better way is fitting using a polynomial model $y = w_0 + w_1x + w_2x^2 + w_3x^3 + \dots$; this is still a linear problem since the coefficients that we want learn are at the exponent 1. Depending on the degree, we could occur in overfitting case, if for example we use too many parameters (the degree is high), see Figure 2.4. See Figure 2.5 for an example of good fitting.

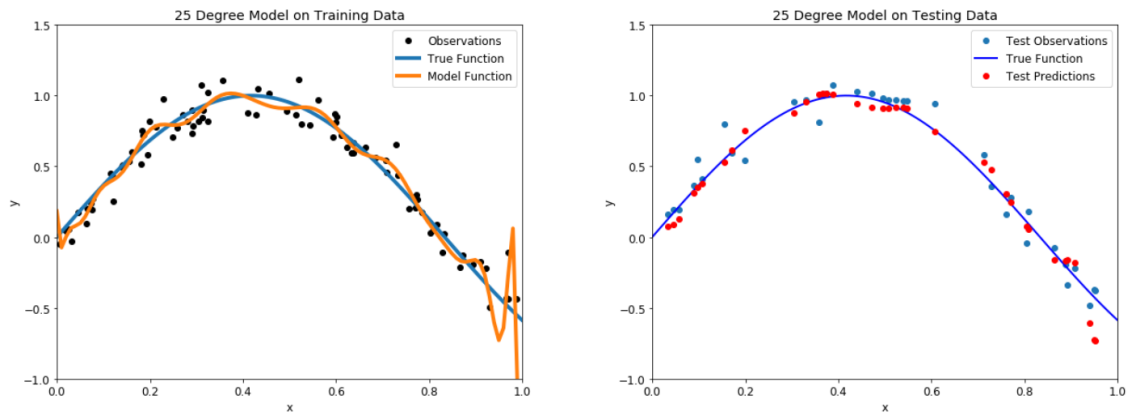


Figure 2.4: This is an example of overfitting

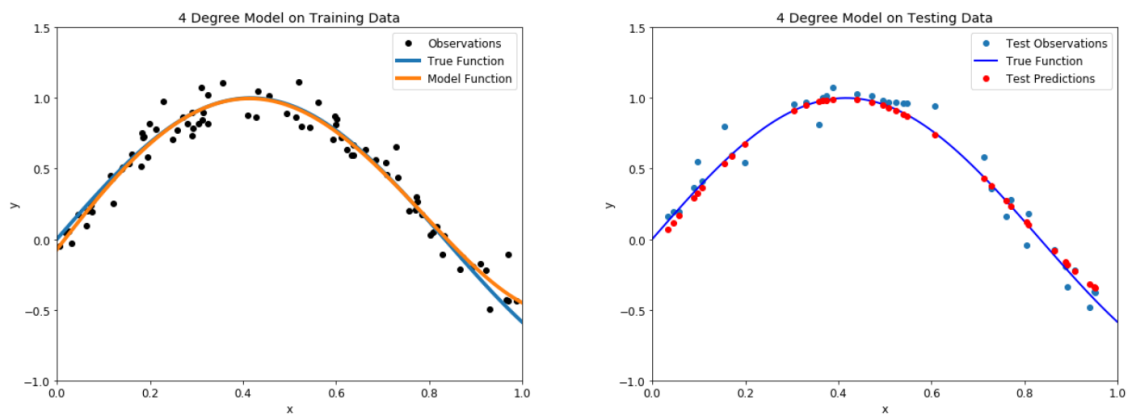


Figure 2.5: This is an example of good fitting

Chapter 3

The Data - Pre-processing and dissimilarities

We know that the workflow of ? is composed of getting data, preparing them, learning from data and at the end predicting. In this chapter we speak about pre-processing data.

3.1 Data pre-processing

Aggregation means combining two or more attributes (or objects) into a single attribute (or object) for the purpose of DATA REDUCTION, reducing the number of attributes or objects, CHANGE OF SCALE and having MORE STABLE DATA, aggregated data tends to have less variability.

Let's consider the example of monthly amount of precipitation measured in different region of Australia. The first histogram in Figure 3.1 it represents the number of regions with a certain standard deviation of the average of monthly precipitation; the histogram is called bimodal since has two local maxima. We notice, looking at the second histogram, that if we change from average monthly to average yearly the distribution becomes more regular: this is the result of aggregation that makes data more reasonable.

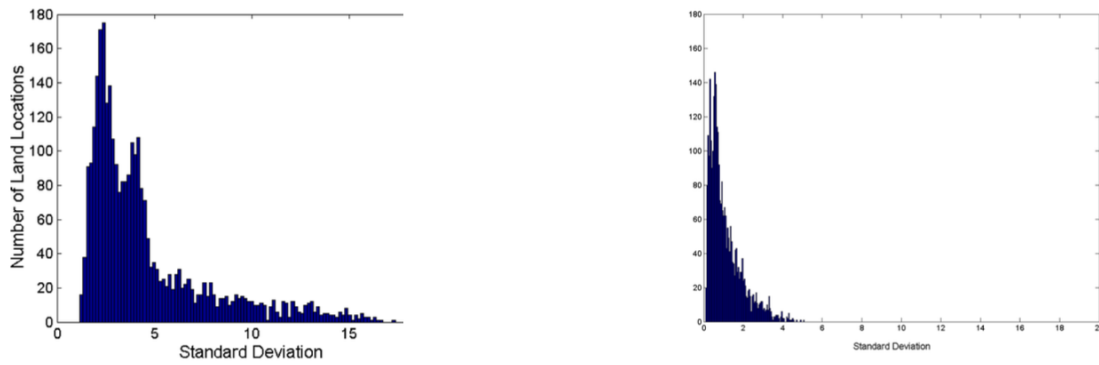


Figure 3.1: Standard deviation of average monthly precipitation (lhs) and standard deviation of average monthly precipitation (rhs).

3.1.1 Sampling

For both preliminary investigation and final data analysis, from a statistician perspective obtaining the entire data set could be impossible or too expensive, from a data processing perspective, processing the entire data set could be too expensive or time consuming.

Using a sample will work almost as well as using the entire data sets, if the sample is **representative**. A sample is representative if it has approximately the same property (of interest) as the original set of data.

How can we sample?

There are different types of sampling:

1. SIMPLE RANDOM: a single random choice of an object with given probability distribution
2. with replacement: repetition of independent extractions
3. WITHOUT REPLACEMENT: repetition of extractions, extracted element is removed from the population
4. STRATIFIED: split data into several partitions according to some criteria, then draw the random samples from each partition. It is used when the data set is split into subsets with homogeneous characteristics, i.e. the representativity is guaranteed inside each subset (it is preserved the proportion of the classes). This last property is very important in train-test splitting and in cross validation.

Which is the difference between sampling with and without replacement?

They are nearly equivalent if sample size is a small fraction of data set size; with replacement, in a small population a small subset could be underestimated. In general sampling with replacement is much easier to implement and to be interpreted from a statistical point of view since extractions are statistically independent.

Sample size

How/When a sample is significant?

Statistics provides techniques to assess optimal sample size and sample significance, a tradeoff between data reduction and precision.

Which is the probability of sampling at least one element for each class (with replacement)? If we suppose to have ten classes we can obtain the graph in Figure 3.2.

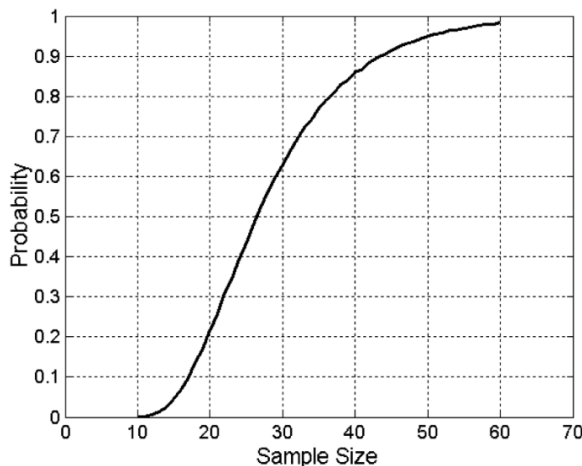


Figure 3.2

This aspect becomes relevant, for example, in a supervised dataset with a high number of different values of the target. If the number data elements is not big enough, it can be difficult to guarantee a stratified partitioning in train/test split or in cross-validation split.

If we consider the example: $N=1000$, $C=10$, the test set size = 300, the cross validation folds = 10, the probability of folds without adequate representation of some classes becomes quite high. One could use only 3 folds in cross-validation.

The curse of dimensionality

Let's consider points in N -dimensional space; I can plot them and I can measure the distance from each point to another. What happens is that when dimensionality of the space is very high the occupation of the space becomes very sparse, so the discrimination on the basis of the distance becomes ineffective.

If we, for example, generate randomly 500 points and we plot the relative difference between the maximum and the minimum distance between pairs of points we obtain the graph in Figure 3.3. For it we see the $\frac{\max - \min}{\min} \rightarrow 0$ for $N \rightarrow \infty$.

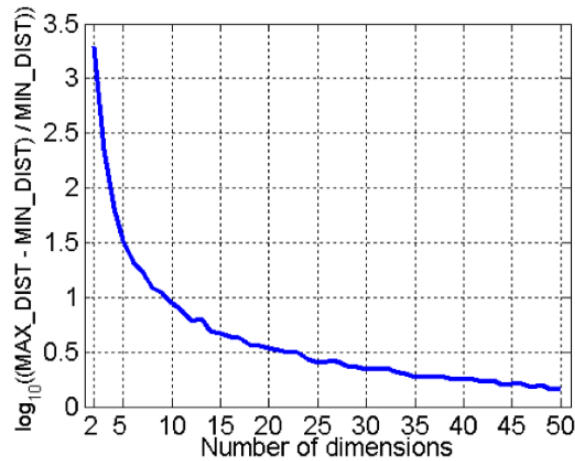


Figure 3.3

For this reason is important to do a dimensionality reduction and also for noise reduction, reducing time and memory complexity of the mining algorithms and for visualization. There are different techniques which we discuss below.

3.1.2 Principal Component Analysis (PCA)

PCA is a non supervised technique.

Consider the case in Figure 3.4. If I drop one of the two dimension, data will be projected into the other axes and I will loose some informations. If I project the data into the line I always loose something but, as they say, I loose less variability. PCA finds projections that capture most of the data variation; it finds the eigenvectors of the covariance matrix which defines a new space. The new dataset will have only the attributes which capture most of the data variation.

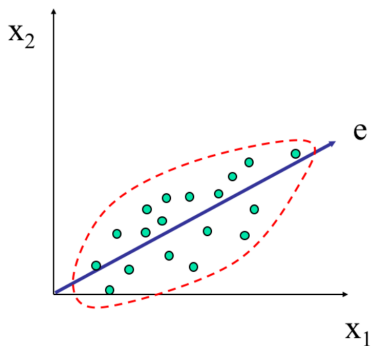


Figure 3.4

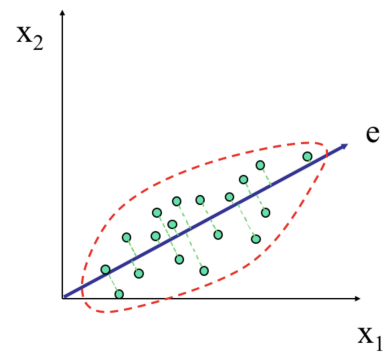


Figure 3.5: Data projection into a subspace

PCA is a global method which consider all the attributes at the same time.

Something different is **Feature subset selection**, a local way to reduce dimensionality that takes into account pairs of attributes. Sometimes we have some redundant attributes which duplicate most of the information contained in other attributes and irrelevant attributes which do not contain any information useful for analysis.

How to deal with this cases? If we want reduce the amount of data we have:

1. Brute force: try all possible feature subsets as input to data mining algorithm and measure the effectiveness of the algorithm with the reduced dataset
2. Embedded approach: feature selection occurs naturally as part of the data mining algorithm (looking at decision trees: attribute not present in it are not relevant in predicting)
3. Filter approach: features are selected before data mining algorithm is run
4. Wrapper approaches: a data mining algorithm can choose the best set of attributes

New features can capture more efficiently data characteristics: feature extraction (in faces recognition, among different images, it is more useful and easy to deal with some characteristic as eye distance than mapping all the pixels), mapping to a new space, ect.

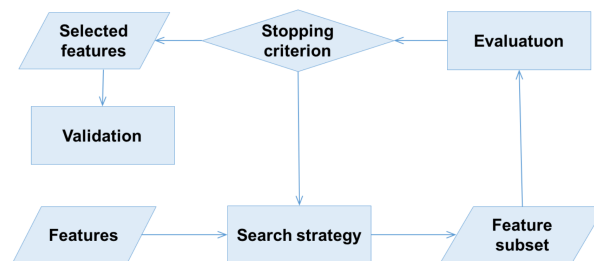


Figure 3.6: An architecture for feature subset selection

3.2 Data type conversion

Why do we need type conversion?

Many algorithms require numeric features and for this reason categorical features must be transformed into numeric and the same for ordinal features preserving the order. Classification requires a target with nominal values since a numerical target can be discretised. Despite discovery of association rules requires boolean features, so numerical feature can be discretised and transformed into a series of boolean features.

Quality	Quality-Awful	Quality-Poor	Quality-OK	Quality-Good	Quality-Great
Awful	1	0	0	0	0
Poor	0	1	0	0	0
Ok	0	0	1	0	0
Good	0	0	0	1	0
Great	0	0	0	0	1

Figure 3.7: An example of binarization of discrete attributes. An attribute allowing V values generate V binary attributes. this is an example of ordinal attribute.

Nominal to numeric

One-Hot-Encoding:

- a feature with V unique values is substituted by V binary features each one corresponding to one of the unique values
- if object x has value v in feature d then the binary feature corresponding to v has True for x , all the other binary features have value False
- True and False are represented as 1 and 0, therefore can be processed by also by procedures working only on numeric data, as is the case for the estimators available in scikit-learn
- `sklearn.preprocessing.OneHotEncoder`

Ordinal to numeric

The ordered sequence is transformed into consecutive integers. By default the lexicographic order is assumed but the user can specify the proper order of the sequence. See `sklearn.preprocessing.OrdinalEncoder`

Numeric to binary with threshold

Not greater than the threshold becomes zero and greater than the threshold becomes one. See `sklearn.preprocessing.Binarizer`

Discretization

Some algorithms work better with categorical data. A small number of distinct values can let patterns emerge more clearly because let the algorithms to be less influenced by noise and random effects.

- Continuous $\rightarrow$ Discrete. We can have more thresholds ora a single one after binarization. See at the figure 3.8.
- Discrete with many values $\rightarrow$ Discrete with less values. This is guided by domain knowledge

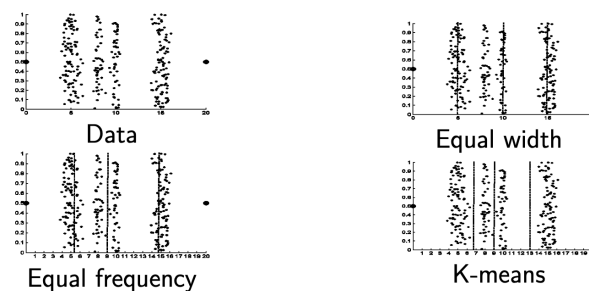


Figure 3.8: Continuous $\rightarrow$ Discrete. Boundaries on x axis – Unsupervised. "Equal frequency" means same amount of point between two consecutive lines

Numeric to k values

The numbers are discretised into a sequence of integers 0 to k-1. Several strategies are available ('uniform', 'quantile', 'means'). See `sklearn.preprocessing.KBinsDiscretizer`

3.3 Similarity and dissimilarity

Instead of using distance we use similarity **Similarity** is a numerical measure of how alike two data objects are. Is higher when objects are more alike. Often falls in the range [0,1]. In general $\text{Sim}(p,q)=1$ only if $p=q$ and $\text{Sim}(p,q)=\text{Sim}(q,p)$. **Dissimilarity** is a numerical measure of how different are two data objects. Lower when objects are more alike (minimum dissimilarity is often 0); the upper limit varies. Proximity refers to a similarity or dissimilarity. Depending on attribute type we can have different definitions, see figure 3.9

Attribute type	Dissimilarity	Similarity
Nominal	$d = \begin{cases} 0 & \text{if } p = q \\ 1 & \text{if } p \neq q \end{cases}$	$s = \begin{cases} 1 & \text{if } p = q \\ 0 & \text{if } p \neq q \end{cases}$
Ordinal Values mapped to integers 0 to V-1	$d = \frac{ p-q }{V-1}$	$s = 1 - \frac{ p-q }{V-1}$
Interval or Ratio	$d = p - q $	$s = \frac{1}{1+d}$ or $s = 1 - \frac{d - \min(d)}{\max(d) - \min(d)}$

Figure 3.9: Similarity and Dissimilarity by Attribute type. p and q are the values of an attribute for two data objects

Algorithms which deal with space and distances are influenced by the scale, despite of decision tree.

Euclidean distance – L_2

It is defined as $dist = \sqrt{\sum_{d=1}^D (p_d - q_d)^2}$, where D is the number of dimensions (attributes) and p_d and q_d are, respectively, the d-th attributes (components) of data objects p and q. Standardization/Rescaling is necessary if scales differ.

What to do if the scales of the two axis are different? The attribute with the highest range is more relevant. The min-max rescaler, $x_1^s = \frac{x_1 - x_1^{min}}{x_1^{max} - x_1^{min}}$, remap the features in the intervall [0,1]; in this way all the feautres have the same importance. See figure 3.10.

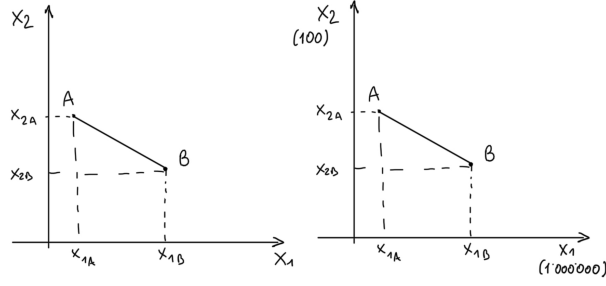


Figure 3.10: In the right plot the distance AB is mostly influenced by $x_{1B} - x_{1A}$

Minkowski distance – L_r

It is defined as $dist = \left(\sum_{d=1}^D |p_d - q_d|^r \right)^{\frac{1}{r}}$, where D is the number of dimensions (attributes) and p_d and q_d are, respectively, the d-th attributes (components) of data objects p and q. r is a parameter which is chosen depending on the data set and the application. Standardization/Rescaling is necessary if scales differ.

- $r=1$ also named city block, Manhattan, L_1 norm: it is the best way to discriminate between zero distance and near zero distance, an ϵ change on any coordinate causes an ϵ change in the distance. It is the less influenced by dimensionality, it works better than euclidean in very high dimensional spaces.
- $r=\infty$ also named Chebyshev, supremum, L_{max} norm, L_∞ norm: considers only the dimension where the difference is maximum. Provides a simplified evaluation, disregarding the dimensions with lower differences. $dist_\infty = \max_d |p_d - q_d|$.

Mahalanobis Distance

Considers data distribution as in figure 3.11. The Mahalanobis distance between two points p and q decreases if, keeping the same euclidean distance, the segment connecting the points is stretched along the direction of greater variation of data. The distribution is described by the covariance matrix of the data set $\Sigma_{ij} = \frac{1}{N-1} \sum_{k=1}^N (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)$, $dist_m = \sqrt{(p-q)\Sigma^{-1}(p-q)^T}$. Consider the example in figure 3.11: A=(0.5,0.5), B=(0,1), C=(1,1).

$$\Sigma = \begin{bmatrix} 0.3 & 0.2 \\ 0.2 & 0.3 \end{bmatrix}$$

The euclidean distances AB and AC are the same but $dist_m(A, B) = 2.236068$ and $dist_m(A, C) = 1$. AC is lower because it is in the direction of the first eigenvalues of the covariance matrix (the direction of maximum variability)

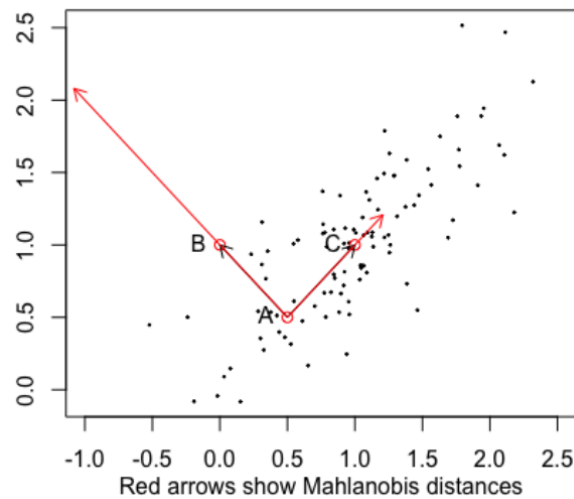


Figure 3.11

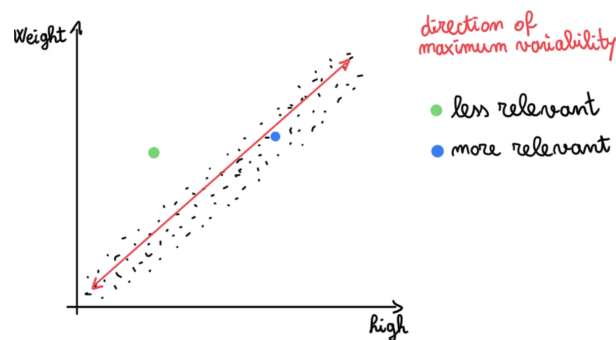


Figure 3.12: More variability $\rightarrow$ more importance, less variability $\rightarrow$ less importance

The Mahalanobis distance is strictly related to PCA.

Covariance matrix

- Variation of pairs of random variables
- The summation is over all the observations
- The main diagonal contains the variances
- The values are positive if the two variables grow together
- If the matrix is diagonal the variables are non-correlated
- If the variables are standardised the diagonal contains “one”
- If the variables are standardised and non correlated, the matrix is the identity and the Mahalanobis distance is the same as the euclidean

3.3.1 Similarity between binary vectors

Consider the counts below:

- Be M_{00} the number of attributes where p is 0 and q is 0
- Be M_{01} the number of attributes where p is 0 and q is 1
- Be M_{10} the number of attributes where p is 1 and q is 0
- Be M_{11} the number of attributes where p is 1 and q is 1

We define the **Simple Matching Coefficient**:

$$SMC = \frac{\text{number of matches}}{\text{number of attributes}} = \frac{M_{00} + M_{11}}{M_{00} + M_{01} + M_{10} + M_{11}},$$

and the **Jaccard Coefficient** (excludes negative matches):

$$JC = \frac{\text{number of 11 matches}}{\text{number of non-both-zero attributes}} = \frac{M_{11}}{M_{01} + M_{10} + M_{11}}$$

If we want not only to see the presence but we want to count also the words we have the cosine similarity: $\cos(p, q) = \frac{p \cdot q}{\|p\| \|q\|}$. We have also a variation of Jaccard for continuous or count attributes (Tanimoto): $T(p, q) = \frac{p \cdot q}{\|p\|^2 + \|q\|^2 - p \cdot q}$

How choose the right proximity measure?

It depends on data: if they are dense and continuous we can use a metric measure, such as the euclidean distance, if they are sparse and asymmetric we can use cosine, jaccard, extended jaccard.

3.3.2 Correlation

Measure of the linear relationship between a pair of attributes. For two given attributes p and q, consider as vectors the ordered lists of the values over all the data records, compute the dot product of the vectors: $p = [p_1, \dots, p_N] \xrightarrow{\text{standardize}} p'$ and $q = [q_1, \dots, q_N] \xrightarrow{\text{standardize}} q'$ and we have $\text{corr}(p, q) = p' \cdot q'$.

If we have Independent variables the correlation is zero (the inverse is not valid in general, it means only that there isn't a linear relationship between the variables)

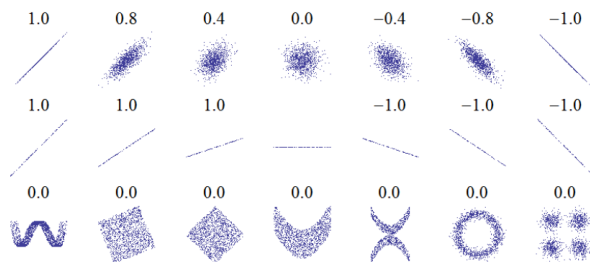


Figure 3.13: Examples of correlation

Correlation between nominal attributes

Symmetric Uncertainty $U(p, q) = 2 \frac{H(p) + H(q) - H(p, q)}{H(p) + H(q)}$, where $H()$ is the entropy of a single attribute while $H(p, q)$ is the joint entropy, computed from the joint probabilities. It is always between 0 ($H(p, q) = H(p) + H(q)$) and 1 (when there is a biunivocal correspondence).

3.4 Data transformations

The features may have different scales and this can alterate the results of many learning techniques (some are sensitive, others not).

For example Machine learning algorithms that use gradient descent as an optimization technique require data to be scaled (linear regression, logistic regression, neural network, etc). The presence of feature value X in the formula will affect the step size of the gradient descent. The difference in ranges of features will cause different step sizes for each feature. Similar ranges of the various features ensure that the gradient descent moves smoothly towards the minima and that the steps for gradient descent are updated at the same rate for all the features.

Attribute transformation

- Map the entire set of values to a new set according to a function as x^k , $\log(x)$, e^x , $|x|$ which in general change the distribution of values
- Standardization: $x \rightarrow \frac{x - \mu}{\sigma}$. if the original values have a gaussian distribution, the transformed values will have a standard gaussian distribution. Translation and shrinking/stretching, no change in distribution.
- MinMax scaling (a.k.a. Rescaling): the domains are mapped to standard ranges; also here we have translation and shrinking/stretching, no change in distribution.

$$x \rightarrow \frac{x - x_{min}}{x_{max} - x_{min}} \text{ (0 to 1) or } x \rightarrow \frac{x - \frac{x_{max} + x_{min}}{2}}{\frac{x_{max} - x_{min}}{2}} \text{ (-1 to 1)}$$

Let's consider the example in the two following figures: we see transformations in action.

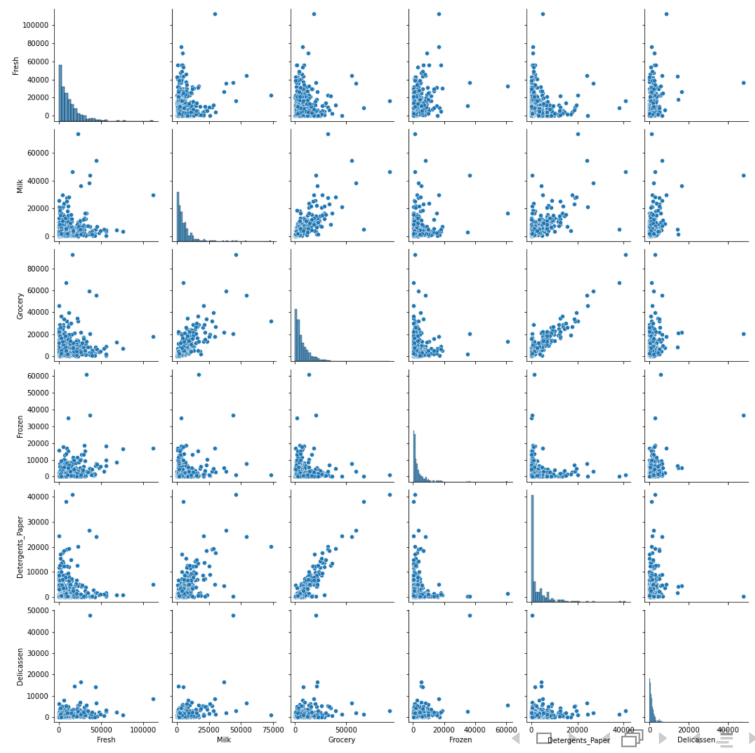


Figure 3.14: Data with skewed distribution

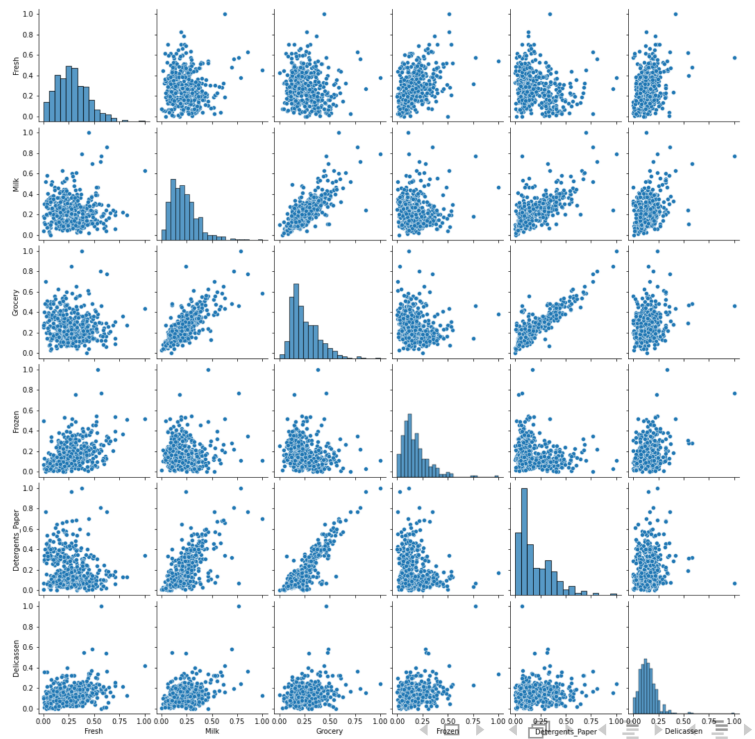


Figure 3.15: After the transformation the data are less skewed

Distance-based algorithms

KNN, K-Means, SVM, distances between points are used to determine their similarity. If we want a fair compatibility of CGPA and Salary we can rescale as in figure 3.16

Example

Original data			Scaled data		
Student	CGPA	Salary	Student	CGPA	Salary
A	3	60	A	-1.18431	1.520013
B	3	40	B	-1.18431	-1.100699
C	4	40	C	0.41612	-1.100699
D	4.5	50	D	1.21635	0.209657
E	4.2	52	E	0.736212	0.471728

Figure 3.16

$$distance(A, B) = \sqrt{(40 - 60)^2 + (3 - 3)^2} = 20$$

$$distance(B, C) = \sqrt{(40 - 40)^2 + (4 - 3)^2} = 1$$

$$distance(A_s, B_s) = \sqrt{(1.1 + 1.5)^2 + (1.18 - 1.18)^2} = 2.6$$

$$distance(B_s, C_s) = \sqrt{(1.1 - 1.1)^2 + (0.41 + 1.18)^2} = 1.59$$

Before the scaling the two distances seemed to be very different, due to the a big numeric difference in the Salary attribute, now they are comparable.

Range-based scaling and standardization

Operate on single features

- Range-based scaling stretches/shrinks and translates the range, according to the range of the feature (there are some variants). It is good when we know that the data are not gaussian, or we do not make any assumption on the distribution. The base variant, the MinMax scaler, remaps to 0, 1
- Standardization subtracts the mean and divides by the standard deviation. the resulting distribution has mean zero and unitary standard deviation, It is good when the distribution is gaussian

3.5 Imbalanced data in classification

The performance minority class (classes) has little impact on standard performance measures. The optimised model could be less effective on minority class (classes) and for this some estimators allow to weight classes. Some performance measures allow to take into account the contribution of minority class (classes).

Cost Sensitive learning

Several classifiers have the parameter class-weight, it changes the cost function to take into account the imbalancing of classes. In practice it is equivalent to over-sampling the minority class, (repeating random examples) in order to produce a balanced training set.

Undersampling

Obtains a balanced training set by randomly reducing the number of examples of the majority class. Obviously part of the knowledge embedded in the training set is dropped out.

Oversampling with SMOTE

- Synthesize new examples from the minority class
- a type of data augmentation
- extract a random example from the minority class is first chosen
- find k of the nearest neighbors are found
- choose a randomly selected neighbor is chosen and a synthetic example is created at a randomly selected point between the two examples in feature space

Chapter 4

Clustering

Given a set of N objects x_i , each described by D values x_{id} we want find a natural partitioning in K clusters and, possibly, a number of noise objects; the result is a clustering scheme, i.e. a function mapping each data object to the sequence $[1, \dots, K]$ (or to noise). We would like that objects in the same cluster are similar so we must look for a clustering scheme which maximizes intra-cluster similarity.

Formally we want find a function $\text{clust}()$ from X to $1, \dots, K$ such that:

- $\forall x_1, x_2 \in X, \text{clust}(x_1) = \text{clust}(x_2)$ when x_1 and x_2 are similar
- $\forall x_1, x_2 \in X, \text{clust}(x_1) \neq \text{clust}(x_2)$ when x_1 and x_2 are not similar

If we consider the data of the figure 4.1, we can see that the clustering function maps points to clusters

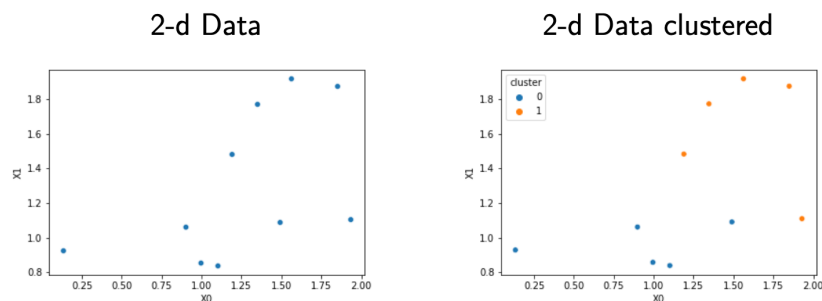


Figure 4.1

But we could do the alternative clustering scheme clust^b in figure 4.2

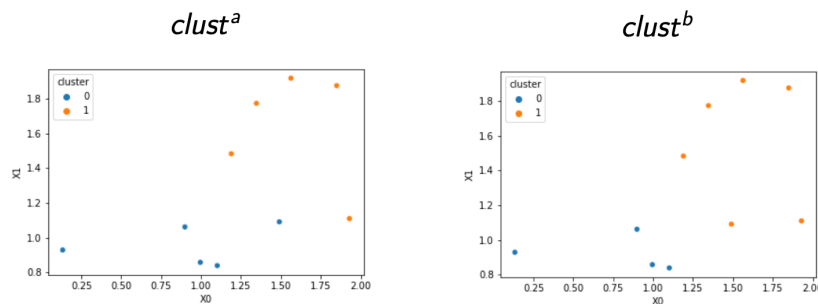


Figure 4.2

Which of the two is better, i.e. maximises intra-cluster similarity? A measure is needed.

Ideas for a measure?

- The sum of the distances between a point and the others in the same cluster? Bigger for bigger clusters.
- The average of the distances between a point and the others in the same cluster? Not influenced by cluster size.
- The sum of the squared distances? More penalisation for sparsity
- We could choose a point which, in some sense, represents the points of the cluster. What about the average of the coordinates?

Centroid

A point with coordinates computed as the average of the coordinates of all the points in the cluster. In physics it is the center of gravity of a set of points of equal mass. For each cluster k and dimension d , the d coordinate of the centroid is $centroid_d^k = \frac{1}{|x_i:clust(x_i)=k|} \sum_{x_i:clust(x_i)=k} x_{id}$. From the previous data the centroids for the two different clustering schemes are in figure 4.3.

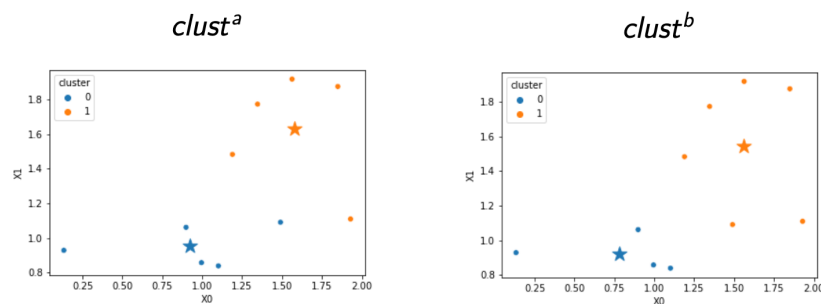


Figure 4.3

4.1 K-means

If we consider the data in figure 4.4 they could be modeled as a five component gaussian mixture, but how do we guess the number five in a D -dimensional space (with $D \geq 2$)?

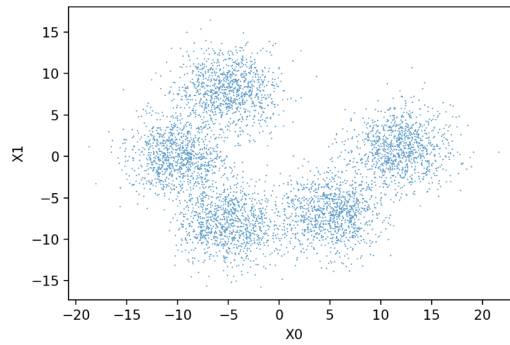


Figure 4.4

How can we "communicate"/transmit the distribution of data? Transmit the coordinates of points allow only two bits per point, so it will be lossy. Thus we need a coding/decoding mechanism. The loss is the sum of the squared errors between the real points and their encoding/decoding. How can we minimize the loss?

First idea is to partition the space into a grid of cells (see figure 4.5 and decode each pair of bits with the center of the grid cell. An improvement could be to decode each pair of bits with the centroid of the points in the grid cell.

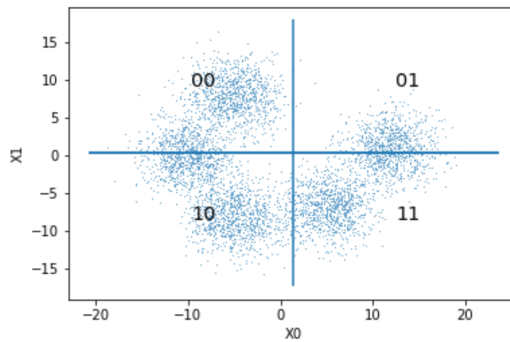


Figure 4.5

K-means works in this way:

1. Ask the user the number of clusters K (5 in the previous example)
2. Random choice of K points as temporary centers (figure 4.6)
3. Each point finds his nearest center and is labelled (i.e. colored) accordingly (figure 4.7)
4. For each center finds the centroid of its points
5. And then move there the center (figure 4.8)

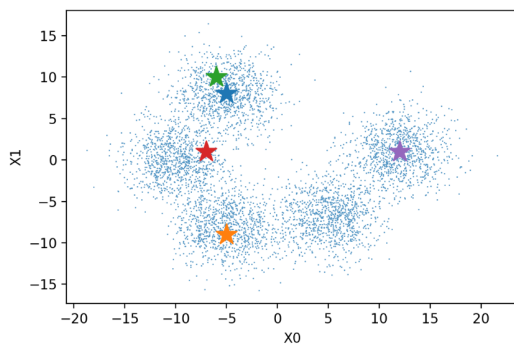


Figure 4.6: Kmeans random centers

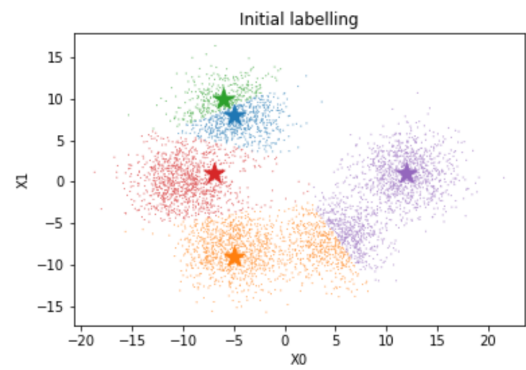


Figure 4.7: Kmean first clustering

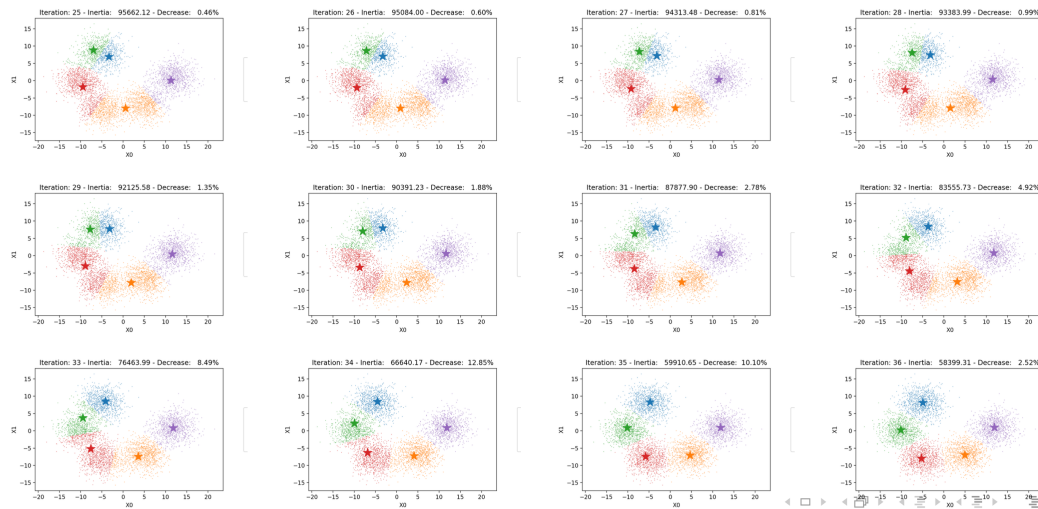


Figure 4.8: KMeans working...

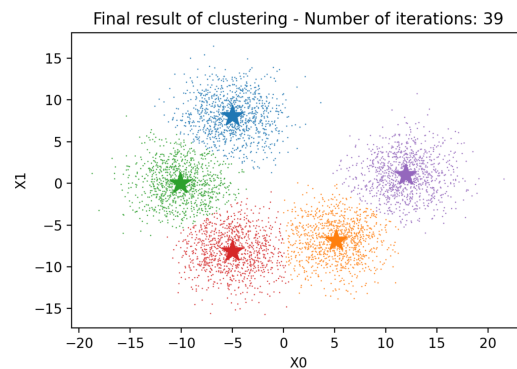


Figure 4.9: KMeans final result

Questions

1. What are we trying to optimize?

2. Is termination guaranteed?
3. Are we sure that the best clustering scheme is found? Which is the definition of best clustering scheme?
4. How should we start?
5. How can we find the number of clusters?

4.1.1 Distortion

Distortion is also frequently called in literature as Inertia.

Given:

a dataset	$\{x_i, i = 1 \dots N\}$
a coding function	$\text{Encode} : \{R\}^D \rightarrow [1..K]$
a decoding function	$\text{Decode} : [1..K] \rightarrow \mathbb{R}^D$
define	$\text{Distortion} = \sum_{i=1}^N (x_i - \text{Decode}(\text{Encode}(x_i)))^2$
shortcut	$\text{Decode}(k) = \mathbf{c}_k$

then
$$\text{Distortion} = \sum_{i=1}^N (x_i - \mathbf{c}_{\text{Encode}(x_i)})^2$$

We would like to minimize the distortion. Which properties are requested to $c_1, \dots, c_k$ for the minimal distortion?

1. x_i must be encoded with the nearest center. Because otherwise the distortion could be reduced by substituting $\text{Encode}(x_i)$ with the nearest center.
 $c_{\text{Encode}(x_i)} = \arg \min_{c_j \in (c_1, \dots, c_k)} (x_i - c_j)^2$
2. The partial derivative of distortion w.r.t. the position of each center must be zero because in that case the function has either a maximum or a minimum

$$\begin{aligned}
 \text{Distortion} &= \sum_{i=1}^N (x_i - \mathbf{c}_{\text{Encode}(x_i)})^2 \\
 &= \sum_{j=1}^K \sum_{i \in \text{OwnedBy}(\mathbf{c}_j)} (x_i - \mathbf{c}_j)^2 \\
 \frac{\partial \text{Distortion}}{\partial \mathbf{c}_j} &= \frac{\partial}{\partial \mathbf{c}_j} \sum_{i \in \text{OwnedBy}(\mathbf{c}_j)} (x_i - \mathbf{c}_j)^2 \\
 &= -2 \sum_{i \in \text{OwnedBy}(\mathbf{c}_j)} (x_i - \mathbf{c}_j)
 \end{aligned}$$

When distortion is minimal we have

$$\mathbf{c}_j = \frac{1}{|\text{OwnedBy}(\mathbf{c}_j)|} \sum_{i \in \text{OwnedBy}(\mathbf{c}_j)} x_i$$

but this is the definition of centroid. Thus each center must be the centroid of the points it owns

It can be proven that after a finite number of steps the system reaches a state where neither of the two operations changes the state.

Algorithm termination

There is only a finite number of ways to partition N objects into K groups. The state of the algorithm is given by the two encode/decode functions. The number of configurations where all the centers are the centroids of the points they own is finite. If after one iteration the state changes, the distortion is reduced. Therefore each change of state bring to a state which was never visited before. In summary, sooner or later the algorithm will stop because there are no new states reachable

Local or global minimum?

Is the ending state the best possible? Not necessarily For example consider the data in figure 4.10. In this case the system cannot move from the state in figure 4.11. In general the function is not monotonic and thus we could find only a global minimum. Everything depends on the initial assignment.

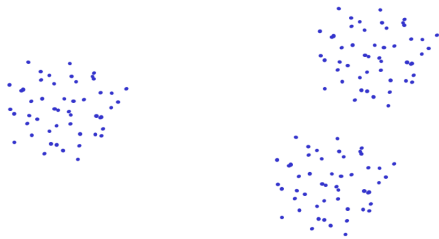


Figure 4.10

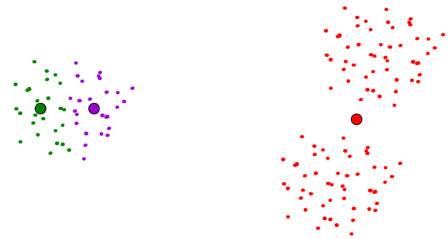


Figure 4.11

Looking for a good ending state

The starting point is important: choose randomly the first starting point and then choose in sequence the $2..K$ starting points as far as possible from the preceding ones. Re-run the algorithm with different starting points

Choose the number of clusters

When we increase the number of clusters we expect that inertia decreases. Inertia is a monotonic function of the number of clusters: if $K=N$ inertia is 0, if $K=1$ inertia is maxima.

You can try various values and use a quantitative evaluation of the quality of the clustering scheme to decide among the different values. The best value finds the optimal compromise between the minimization of intra-cluster distances and the maximization of the inter cluster distances.

The official name of the distortion is **Sum of Squared Errors**

$$\begin{aligned}
SSE &= \sum_{j=1}^K \sum_{i \in \text{OwnedBy}(\mathbf{c}_j)} (\mathbf{x}_i - \mathbf{c}_j)^2 \\
&= \sum_{j=1}^K SSE_j
\end{aligned}$$

A cluster j with high SSE_j has low quality. $SSE_j = 0$ if and only if all the points are coincident with the centroid. SSE decreases for increasing K , is zero when $K=N$. Minimizing SSE is not a viable solution to choose the best K .

Empty clusters

It may happen, at some step, that a centroid does not own any point, this changes the initial requirement of K clusters. Thus we need to choose a new centroid: choose a point far away from the empty centroid or, choose as new centroid a random point in the cluster with the maximum SSE and in this way the cluster with the lowest quality will be split in two.

Outliers

Outliers are points with high distance from their centroid, so they give a high contribution to SSE. they have a bad influence on the clustering results, for this sometimes it is a good idea to remove them (always the choice is related to the application domain).

Complexity

Given T number of iterations, K number of clusters, N number of data points and D number of dimensions, the time complexity is $O(TKND)$

4.1.2 Pros and cons of K-means

STRONG POINT:

- fairly efficient, nearly linear in the number of data points; in general $T, K, D \ll N$

WEAK POINTS:

- in essence it is defined for spaces where the centroid can be computed. When the Euclidean distance is available, also other distance functions work well. But they cannot work with nominal data
- requires the K parameter. nevertheless the best K can be found with iterations
- it is very sensitive to outliers
- does not deal with noise
- does not deal properly with non convex clusters

4.2 Evaluation of a clustering scheme

It is related only to the result, not to the clustering technique, is independent from the method used to generate the classes.

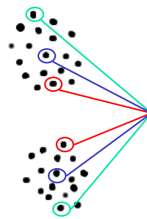
Since clustering is a non supervised method the evaluation is critical, because whenever there is very little apriori information, such as class labels, we need one or more score function to measure various properties of the clusters and of the clustering scheme as a whole. If some supervised data are available, they can be used to evaluate the clustering scheme.

Some issues on the evaluation of clustering are:

- distinguish patterns from random apparent regularities
- find the best number of clusters
- non supervised evaluation
- supervised evaluation
- relative comparison of clustering schemes

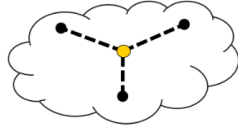
4.2.1 Cohesion and separation - prototype based

These are two measurement criteria: **Cohesions** refers to the proximity of objects in the same cluster, which should be high, **Separation** refers to two different clusters. How to measure it? We could use the distance between the **nearest** objects in the two clusters, or the distance between the **most distant objects** in the two clusters, or the distance between the **centroids** of the two clusters.



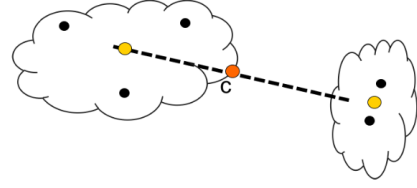
We can also speak about two other measures. One is **Similarity** (or proximity), a two variable function measuring how much two objects are similar, according to the values of their properties, the other is its opposite, **dissimilarity**.

If we look at a single cluster, cohesion is defined as the sum of the proximities between the elements of the cluster itself and the geometric center (the centroid or the medoid; the latter is an element of the dataset whose average dissimilarity with all the elements of the cluster is minimal), see figure 4.12. The separation between two clusters is defined as the proximity between the prototypes, see figure 4.13.



$$\text{Coh}(k_i) = \sum_{x \in k_i} \text{Prox}(x, \mathbf{c}_i)$$

Figure 4.12: Cohesion



$$\text{Sep}(k_i, k_j) = \text{Prox}(\mathbf{c}_i, \mathbf{c}_j)$$

Figure 4.13: Separation

We can also define the **Sum of Squares Between** clusters, which deals with the global separation of a clustering scheme:

$$\begin{aligned} \mathbf{c} &= \text{global centroid of the dataset} \\ \text{SSB} &= \sum_{i=1}^K N_i \text{Dist}(\mathbf{c}_i, \mathbf{c})^2 \end{aligned}$$

Link between cohesion and separation

Be $TSS = \text{Total Sum of Squares}$ the sum of squared distances of the points from the global centroid, thus $TSS = SSE + SSB$. The total sum of squares is a global property of the dataset, independent from the clustering scheme. For a given dataset, minimize $SSE \Leftrightarrow$ maximize SSB .

$$\begin{aligned} TSS &= \sum_{i=1}^K \sum_{x \in k_i} (x - \mathbf{c})^2 = \sum_{i=1}^K \sum_{x \in k_i} ((x - \mathbf{c}_i) - (\mathbf{c} - \mathbf{c}_i))^2 \\ &= \sum_{i=1}^K \sum_{x \in k_i} (x - \mathbf{c}_i)^2 - 2 \sum_{i=1}^K \sum_{x \in k_i} (x - \mathbf{c}_i)(\mathbf{c} - \mathbf{c}_i) + \sum_{i=1}^K \sum_{x \in k_i} (\mathbf{c} - \mathbf{c}_i)^2 \\ &= \sum_{i=1}^K \sum_{x \in k_i} (x - \mathbf{c}_i)^2 + \sum_{i=1}^K \sum_{x \in k_i} (\mathbf{c} - \mathbf{c}_i)^2 \\ &= \sum_{i=1}^K \sum_{x \in k_i} (x - \mathbf{c}_i)^2 + \sum_{i=1}^K |k_i| (\mathbf{c} - \mathbf{c}_i)^2 = SSE + SSB \end{aligned}$$

since $\sum_{x \in k_i} (x - \mathbf{c}_i) = 0$ by definition of $\mathbf{c}_i$

Evaluation of specific clusters and objects

Each cluster can have its own evaluation, the worst clusters can be considered for additional split. A weakly separated pair of clusters could be considered for merging. Single objects can give negative contribution to the cohesion of a cluster or to the separation between two clusters, in particular border objects.

4.2.2 Silhouette

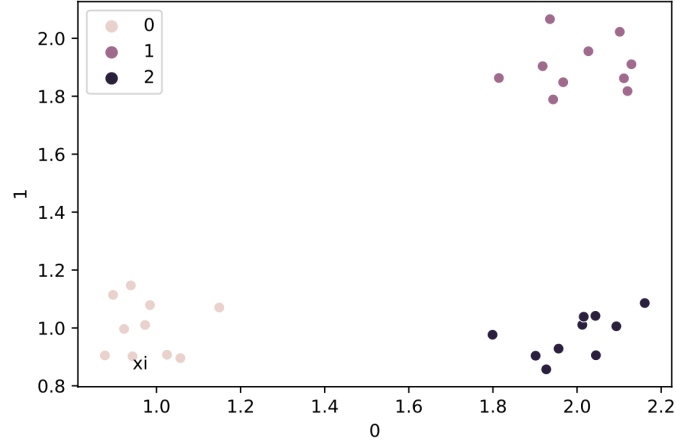
Requirements for a clustering quality score:

- values are in a standard range, e.g. -1, 1

- increases with the separation between clusters
- decreases for clusters with low cohesion, or, in other words, with high sparsity

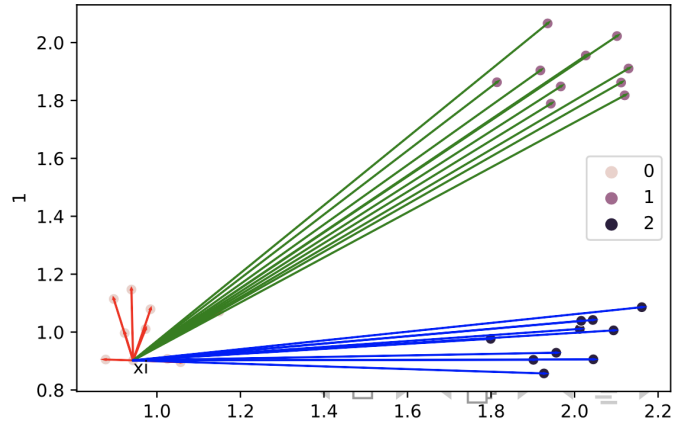
Consider the individual contribution of each object, say x_i .
Contribution to cluster sparsity: the average of red distances is

$$a_i = \text{average}_{j, y(x_j)=y(x_i)} \text{dist}(x_i, x_j) \quad (4.1)$$



Contribution to separation from other clusters: the minimum of the two averages of green and blue distances (not influenced by the cluster's size) is

$$b_i = \min_{k \in Y, k \neq y(x_i)} (\text{average} - \text{dist}_{j, y(x_j)=y(x_i)} \text{dist}(x_i, x_j)) \quad (4.2)$$



The silhouette score of x_i is $s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$. For the global score of a cluster/clustering scheme compute the average score over the cluster/dataset. Note that when the score is less than zero for an object it means that there is a dominance of objects in other clusters at a distance smaller than objects of the same cluster.

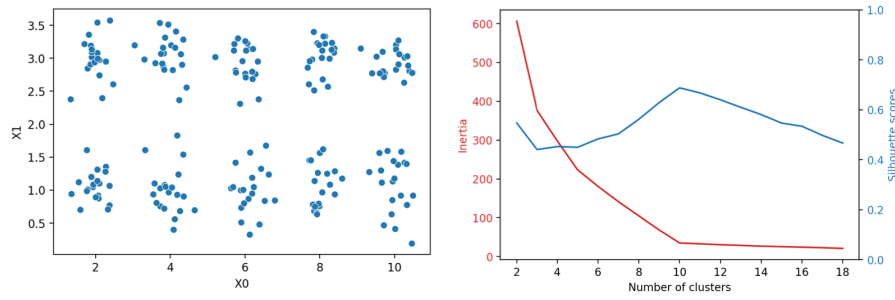


Figure 4.14: Testing K-means with different numbers of clusters. Notice the maximum of the silhouette for $K=10$.

4.2.3 Choice of K

Some algorithms, such as K-means, require the number of clusters as a parameter. Measures, such as SSE and Silhouette, are obviously influenced by the number of clusters and can be used to optimize K. Computation of Silhouette score is expensive, $\sim O(n^2)$. SSE decreases monotonically for increasing K, it is equal to TSS for $K=1$ and goes to zero when $K=N$.

The inertia varying K has frequently one or more points where the slope decreases: one of this points is frequently a plausible value for K (this is called elbow method). The silhouette score varying K has frequently a maximum, in this case it indicates the best value for K

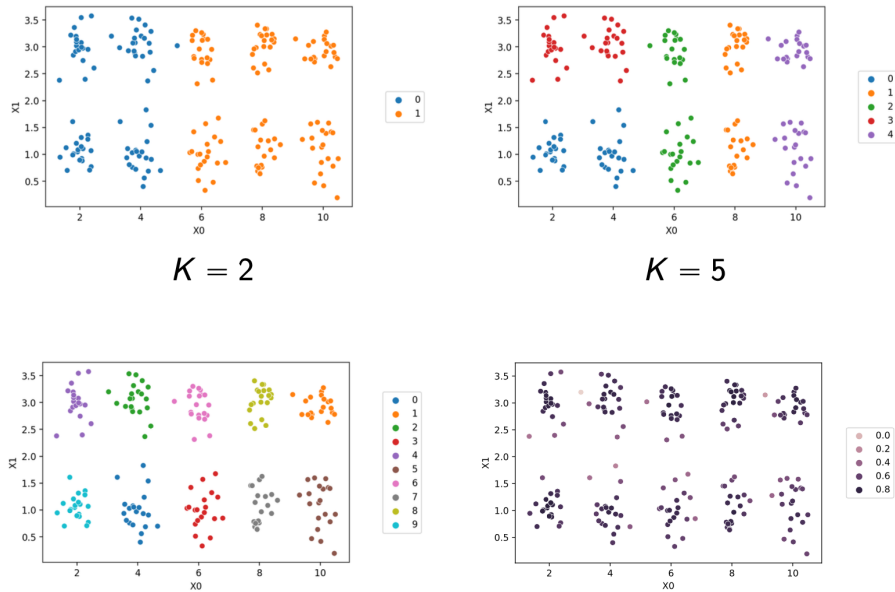


Figure 4.15: Best silhouette for $K=10$. The scale color in the right plot interprets the silhouette for each point.

4.2.4 Supervised measures

Sometimes we have available a partition of a dataset similar to the data to be clustered, which we call **gold standard**, and defined by a labelling scheme $y_g(\cdot)$; it

is the same as the labels attached to supervised data for training a classifier. Consider a clustering scheme $y_k(\cdot)$, the cardinalities of the sets of distinct labels generated by the two schemes V_g and V_k can be different, and also in case of identity of the two grouping schemes, a permutation of labels could be necessary to make them equal.

Why should we compare it with the gold standard?

Validate a clustering technique which can be applied later to new, unlabelled data, the purpose is quite similar to testing a classifier with the only difference that in this case we are more interested in grouping new data than in labelling them following the Gold Standard scheme.

Classification-oriented measures

Measure how the gold standard classes are distributed among the clusters, with confusion matrix, precision, recall, f-measure. We see in the figure 4.16 the confusion matrix for the Iris dataset: the prediction is made randomly and we obtain the best match with permutation of the predicted labels $0 \leftarrow 1, 1 \leftarrow 0, 2 \leftarrow 2$.

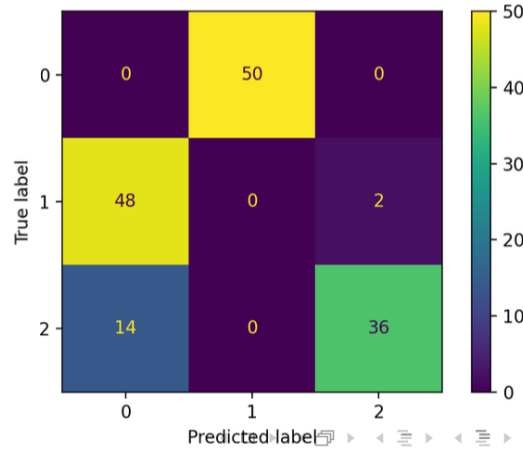


Figure 4.16: In the main diagonal we should have the highest values

Similarity oriented measures

Analogous to compare binary data. Consider a clustering scheme $y_k(\cdot)$ and compare it with the Gold Standard $y_g(\cdot)$. Any pair of objects can be labelled as:

- SGSK if they belong to the same set in $y_g(\cdot)$ and $y_k(\cdot)$
- SGDK if they belong to the same set in $y_g(\cdot)$ and not in $y_k(\cdot)$
- DGSK if they belong to the same set in $y_k(\cdot)$ but not in $y_g(\cdot)$
- DGDK if they belong to different sets both in $y_g(\cdot)$ and $y_k(\cdot)$

Results given by
`pair_confusion_matrix(y_g,y_k)`

	SK	DK
SG	13512	1488
DG	1200	6150

Rand Score $\frac{SGSK+DGDK}{SGSK+DGDK+SGDK+DGSK} = 0.88$

Adjusted Rand Score Excludes the count of matches expected by chance² = 0.73

Jaccard Coefficient for label c $\frac{SG_cSK_c}{SG_cSK_c+SG_cDK_c+DG_cSK_c} = (1, 0.75, 0.69)$

- it requires remapping of $y_g(\cdot)$ to obtain the best match

4.3 Hierarchical clustering

We would find a partition of the original dataset different from what obtained with K-means.

Generates a nested structure of clusters:

- Agglomerative (bottom up)
 - as a starting state, each data point is a cluster
 - in each step the two less separated clusters are merged into one
 - a measure of separation between clusters is needed
- Divisive (top down)
 - as a starting state, the entire dataset is the only cluster
 - in each step, the cluster with the lowest cohesion is split
 - a measure of cluster cohesion and a split procedure are needed

The hierarchical clustering output is a dendrogram or a nested cluster algorithm, shown in figure 4.17. They represent the same structure. The representation is the same for agglomerative and divisive but the agglomerative methods are the most used.

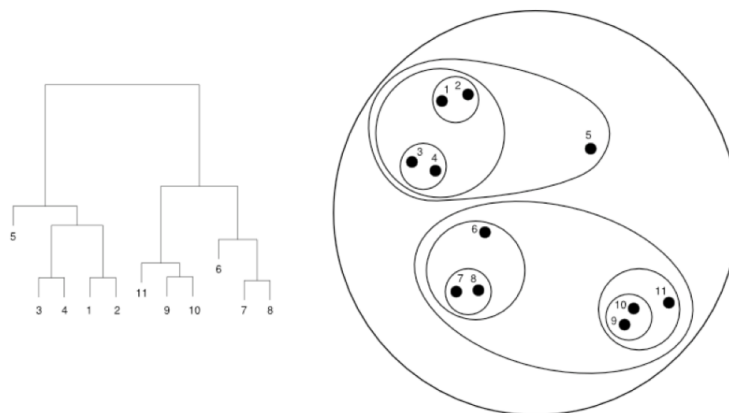


Figure 4.17: Dendrogram (right). On the left the smallest circles represent the shortest arches of the tree

4.3.1 Separation between clusters

The distance between sets is based on the distances between objects belonging to the two sets, respectively

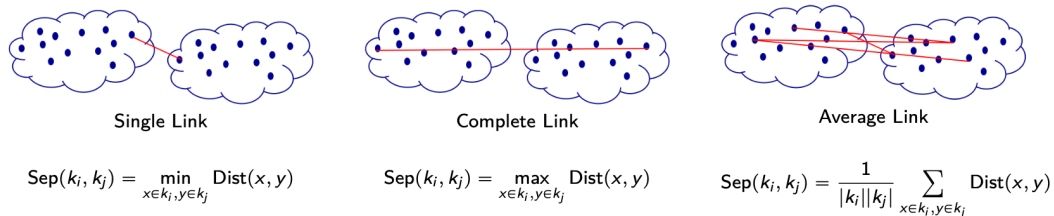


Figure 4.18

Several alternatives, among them: distance between the centroids and Ward's method. The latter is: given two sets with the respective SSE, the separation between the two is measured as the difference between the total SSE resulting in case of merge and the sum of the original SSE; smaller separation implies a lower increase in the SSE after merging.

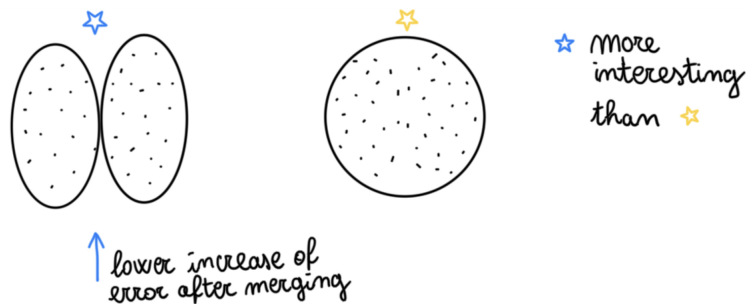


Figure 4.19

4.3.2 Single linkage hierarchical clustering

1. Initialization: every object is a cluster
2. Find the less separated pair
3. Merge in a single cluster
4. Repeat



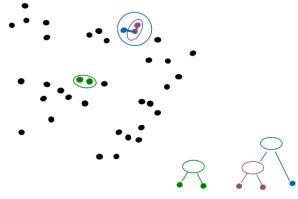


Figure 4.20

The result is a dendrogram (taxonomy, object hierarchy) The algorithm is:

- Initialize the clusters, one for each objects
- Compute the distance matrix between the clusters, squared, symmetric, the size is the number of objects N , the main diagonal is null
- While the number of clusters is greater than 1
 - find the two clusters with lowest separation, say k_r and k_s
 - merge them in a cluster
 - delete from the distance matrix the rows and columns r and s and insert one new row and column with the distances of the new cluster from the others

$$\text{Dist}(k_k, k_{(r+s)}) = \min(\text{Dist}(k_k, k_r), \text{Dist}(k_k, k_s)) \forall k \in [1, K]$$

Time and space complexity

Space and time: $O(n^2)$ for the computation and the storage of the distance matrix. At the worst case $N-1$ iterations to reach the final single cluster. For the i -th step of the main iteration: the search of the pair to merge has a cost $O((N-i)^2)$, while the computation of the distance matrix has a cost $O((N-i))$. The time cost is $O(N^3)$ but can be reduced to $O(N^2 \log(N))$ with indexing structures.

4.3.3 Examples

ITALIAN CITIES EXAMPLE

	BA	FI	MI	NA	RM	TO
BA	0	662	877	255	412	996
FI	662	0	295	468	268	400
MI	877	295	0	754	564	138
NA	255	468	754	0	219	869
RM	412	268	564	219	0	669
TO	996	400	138	869	669	0



	BA	FI	MI	NA	RM	TO
BA	0	662	877	255	412	996
FI	662	0	295	468	268	400
MI	877	295	0	754	564	138
NA	255	468	754	0	219	869
RM	412	268	564	219	0	669
TO	996	400	138	869	669	0



	BA	FI	MI/TO	NA	RM
BA	0	662	877	255	412
FI	662	0	295	468	268
MI/TO	877	295	0	754	564
NA	255	468	754	0	219
RM	412	268	564	219	0



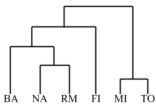
	BA	FI	MI/TO	NA/RM
BA	0	662	877	255
FI	662	0	295	268
MI/TO	877	295	0	564
NA/RM	255	268	564	0



	BA/NA/RM	FI	MI/TO
BA/NA/RM	0	268	564
FI	268	0	295
MI/TO	564	295	0



	BA/FI/NA/RM	MI/TO
BA/FI/NA/RM	0	295
MI/TO	295	0



THE ANIMALS EXAMPLE

50 animals described
with a numeric
attribute (number of
legs, scaled to $[0,1]$)
and 15 boolean
attributes, such as *has
feathers*, *lays eggs*, ...

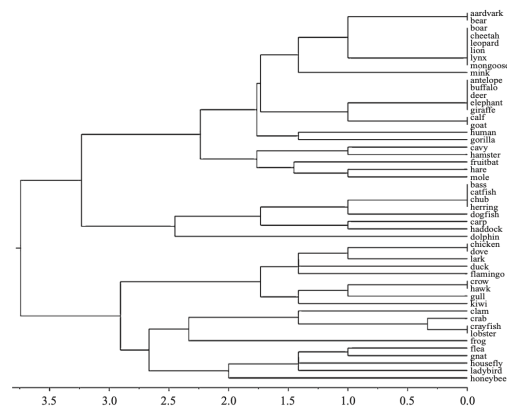


Figure 4.21: Complete Linkage

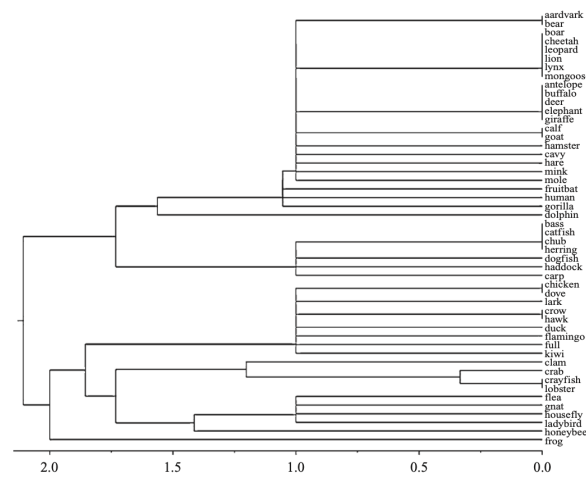
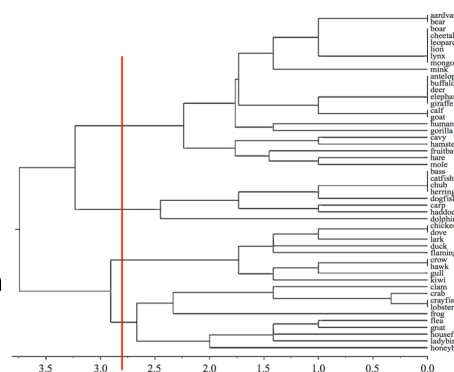


Figure 4.22: Single Linkage

- The desired clustering scheme is obtained by **cutting** the dendrogram at some level
- The choice of the level is application dependent, and can also be guided by indexes, as in the case of K-means

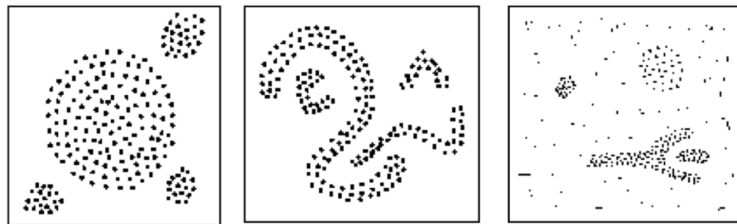


The horizontal axis in the dendrogram is the total dissimilarity inside the clusters, which obviously increases for decreasing number of clusters. The diameter of a cluster is the distance among the most separated objects: single linkage tends to generate clusters with larger diameters also at low levels, complete linkage tends to generate more compact clusters.

- ☹ The scaling is poor, due to the high complexity
- ☹ There isn't a global objective function, the decision is always local and cannot be undone
- ☺ The dendrogram structure is of great help for the interpretation of the result
- ☺ Empirically, the result is frequently good

4.4 Density based clustering

Clusters are high-density regions separate by low-density regions



How compute density? There are two most obvious solutions:

- **Grid-based:** split the (hyper)space into a regularly spaced grid and count the number of objects inside each grid element
- **Object-centered:** define the radius of a (hyper)sphere and attach to each object the number of objects which are inside that sphere

4.4.1 DBSCAN

DBSCAN – Density Based Spatial Clustering of Applications with Noise. Consider the dataset in figure 4.23.

1. Intuitively, p is a *border* point, while q is a *core* point

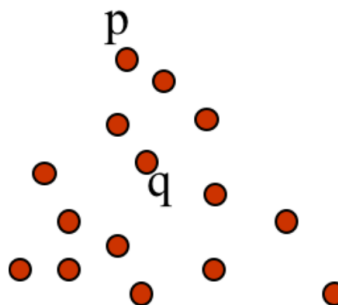
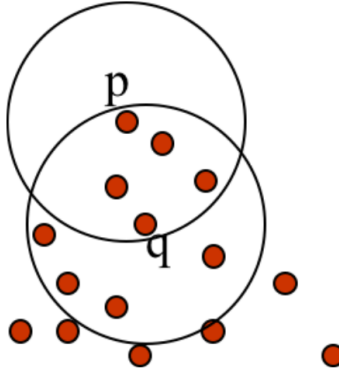
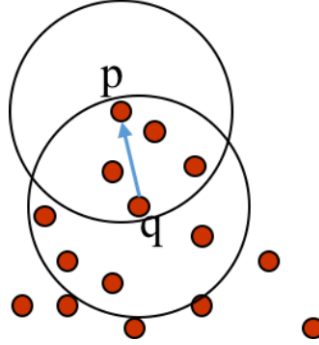


Figure 4.23

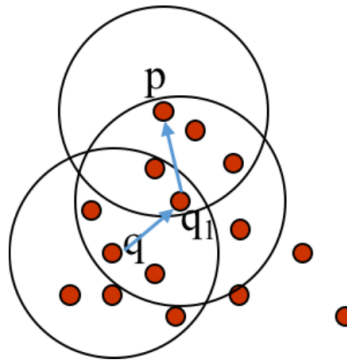
2. Define a radius ϵ and define as *neighborhood* of a point the ϵ -hypersphere centered at that point. When points p and q are one in the neighborhood of the other, as here, we say that the neighborhood is symmetric.



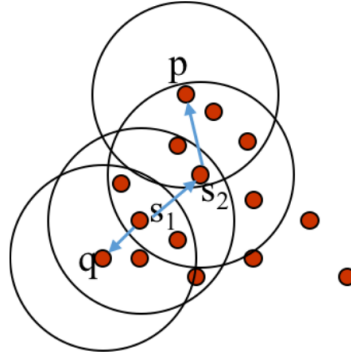
3. Define a threshold minPoints and define as core a point with at least minPoints points in its neighborhood, as border otherwise. In our example with $\text{minPoints}=5$, q is core, p is border.



4. Define that a point p is **directly density reachable** from point q iff q is core and q is in the neighborhood of p . Note that direct density reachability is not symmetric; in the example q is not directly density reachable from p , since p is border.
5. A point p is **density reachable** from point q iff q is core and there is a sequence of points q_i such that q_{i+1} is directly density reachable from q_i , $i \in [1, np]$, q_1 is directly reachable from q and p is directly density reachable from q_{np} . Even reachability is not symmetric; in the example q is not density reachable from p , since p is border.



6. A point p is **density connected** to point q iff there is a point s such that p and q are density reachable from s . Density connection is symmetric.



From the points above we can say that s cluster is a maximal set of points connected by density. Border points which are not connected by density to any core point are labelled as **noise**.

Algorithm

Algorithm DBSCAN

Require: SetOfPoints: UNCLASSIFIED points
Require: Eps, MinPts

```

ClusterId ← nextId(NOISE);
for i = 1 to SetOfPoints.size do
    Point ← SetOfPoints.get(i)
    if Point.CId = UNCLASSIFIED then
        if ExpandCluster(SetOfPoints, Point, ClusterId, Eps, MinPts)
        then
            ClusterId ← nextId(ClusterId)

```

Ensure: SetOfPoints

```

ExpandCluster(SetOfPoints, Point, CId, Eps, MinPts) : Boolean;
seeds := SetOfPoints.regionQuery(Point, Eps);
If seeds.size < MinPts THEN // no core point
    SetOfPoint.changeCId(Point, NOISE);
    RETURN False;
ELSE ...

// all points in seeds are density-reachable from Point
SetOfPoints.changeCIds(seeds, CId);
seeds.delete(Point);
WHILE seeds <> Empty DO
    currentP := seeds.first();
    result := SetOfPoints.regionQuery(currentP, Eps);
    If result.size >= MinPts THEN
        For i FROM 1 TO result.size DO
            resultP := result.get(i);
            If resultP.CId IN {UNCLASSIFIED, NOISE} THEN
                If resultP.CId = UNCLASSIFIED THEN seeds.append(resultP);
            END If;
            SetOfPoints.changeCId(resultP, CId);
        END If; // UNCLASSIFIED or NOISE
    END For;
    END If; // result.size >= MinPts
    seeds.delete(currentP);
END WHILE; // seeds <> Empty
RETURN True;
END If
END; // ExpandCluster

```

How to set ϵ and minPoints?

As in many other machine learning algorithms, a grid search over several combination of hyperparameters can be useful. As a rule of thumb, you can try $\text{minPoints}=2*D$, the number of dimensions, but noise suggest an increase in minPoints.

A guess for ϵ requires more effort, considering the distance of the k -nearest neighbour, with $k=\text{minPoints}$.

- Consider the vector of the k -distances. Choose k and for each point we compute the distance of its k -nearest neighbour and we sort the points for decreasing k -distance.
- Choosing a given k -distance as ϵ , it turns out that all the points with a k -distance bigger than ϵ will be considered as border. In the figure 4.24 they are the points to the left of the vertical of the chosen ϵ .
- Usually, datasets which exhibit some tendency to clustering exhibit also a change of slope.
- The best ϵ can be found with a grid search in the area of the change of slope

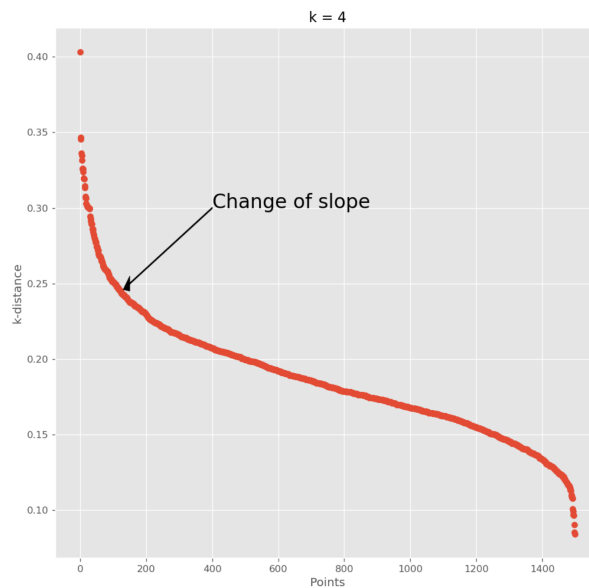
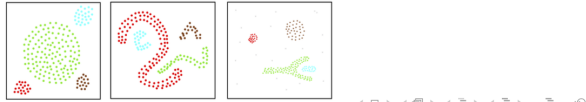


Figure 4.24: The figure refers to a dataset with 1500 points and with $\text{minPoints}=4$. This figure suggests a fine tuning of ϵ in the interval 0.2–0.3

Comments

- ☺ Finds clusters of any shape
- ☺ Is robust w.r.t. noise
- ☹ Problems if clusters have widely varying densities
 - Being based on distances between points, the complexity is $\mathcal{O}(N^2)$
 - reduced to $\mathcal{O}(N \log(N))$ if spatial indexes, such as R*, are available
 - Very sensitive to the values of ϵ and minPoints
 - Decreasing ϵ and increasing minPoints reduces the cluster size and increases the number of noise points



4.4.2 Kernel Density Estimation

It describes the distribution of the data by a function. The overall density function is the sum of the **influence functions** (or kernel functions) associated with each point. Each point has an influence in the space around it, but this influence decreases with the distance. The kernel function must be symmetric and monotonically decreasing and usually has a parameter to set the decreasing rate.

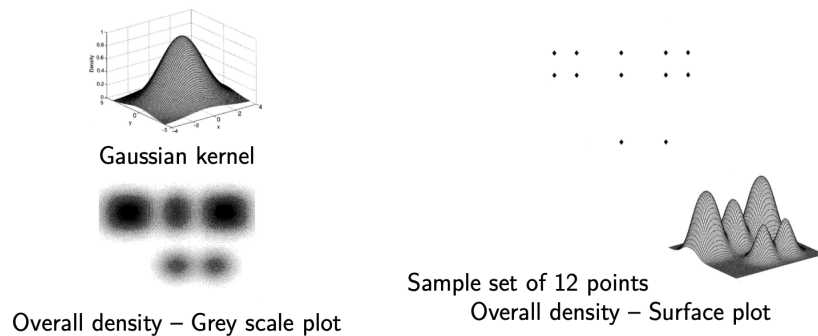


Figure 4.25: KDE example

4.4.3 DENCLUE

1. Derive a density function for the space occupied by the data points
2. Identify the points that are local maxima
3. Associate each point with a density attractor by moving in the directions of maximum increase in density
4. Define clusters consisting of points associated with a particular density attractor
5. Discard clusters whose density attractor has a density less than a user-specified threshold ξ

6. Combine clusters that are connected by a path of points that all have a density of ξ or higher

Comments

- ☺ It has a strong theoretical foundation on statistics
 - precise computation of density
 - DBSCAN is a special case of DENCLUE where the influence is a step function
- ☺ Good at dealing with noise and clusters of different shapes and sizes
- ☹ expensive computation $\mathcal{O}(N^2)$
 - can be optimized with approximated *grid based* computation
- ☹ Troubles with high dimensional data and clusters with different densities

4.5 Model based clustering

The idea is to estimate the parameters of a statistical model to maximize the ability of the model to explain the data. The main technique is to use the mixture models, i.e. view the data as a set of observation from a mixture of different probability distributions (usually the base model is a multivariate normal). Attributes are assumed to be random independent variables.

The estimation is usually done using the **maximum likelihood**.

4.5.1 Gaussian Mixture

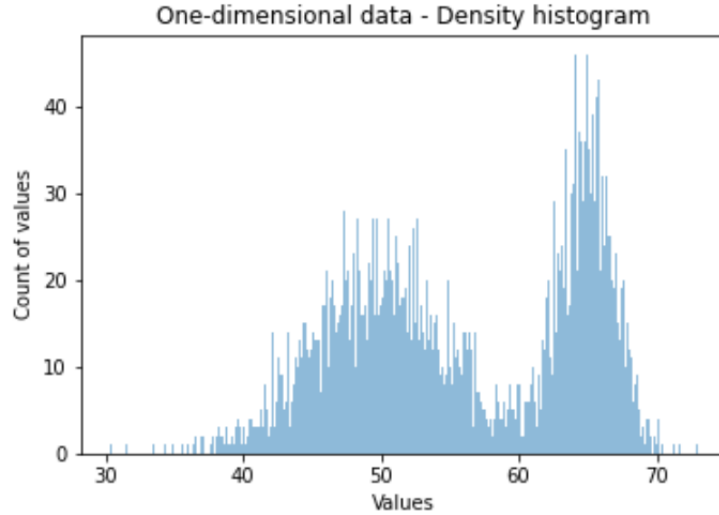
If the data can be approximated by a single distribution, the derivation of the parameters is straightforward. In the general case, with many mixed distributions, the EM algorithm is used.

How works the EM algorithm:

1. Select an initial set of model parameters
2. repeat
 - (a) Expectation Step – For each object, calculate the probability that each object belongs to each distribution
 - (b) Maximization Step – Given the probabilities from the expectation step, find the new estimates of the parameters that maximize the expected likelihood
3. until – the parameters do not change (or the change is below a specified threshold)

One dimension mixture example

Consider the case with one dimension, two components. Synthetic data randomly generated with two gaussian ($\mu_A = 50$, $\sigma_A = 5$, $p_A = 0.6$, $\mu_B = 65$, $\sigma_B = 2$, $p_B = 0.4$)



So we need to estimate 5 independent parameters: the mean and the standard deviation for cluster A, the mean and the standard deviation for cluster B, and the sampling probability p for cluster A

$$\Pr(A|x) = \frac{\Pr(x|A)\Pr(A)}{\Pr(x)} = \frac{f(x; \mu_A, \sigma_A)p_A}{\Pr(x)}$$

$$f(x; \mu_A, \sigma_A) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

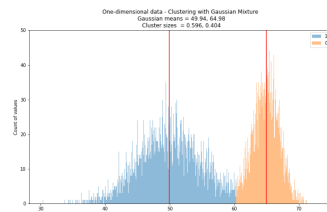
We have to repeat until convergence:

- Expectation: Compute p_A and p_B using the current distribution parameters; compute the numerators for $\Pr(A|x)$ and $\Pr(B|x)$ and normalize dividing by their sum.
- Maximization of the distribution likelihood given the data; compute the new distributions parameters, weighting the probabilities according to the current distribution parameters

After convergence label each object with A or B according to the maximum probability, given the last distribution parameters

Estimated parameters of the distributions

	weight	mean	deviation
0	0.4037	64.978754	2.030968
1	0.5963	49.939990	4.999235



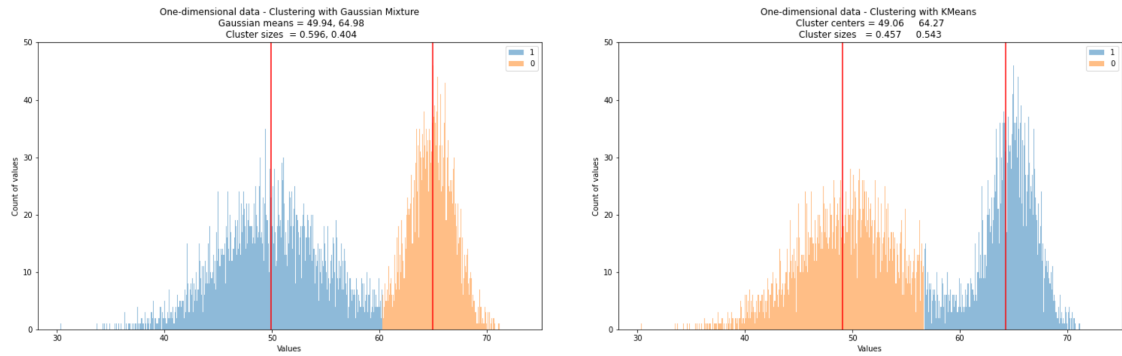
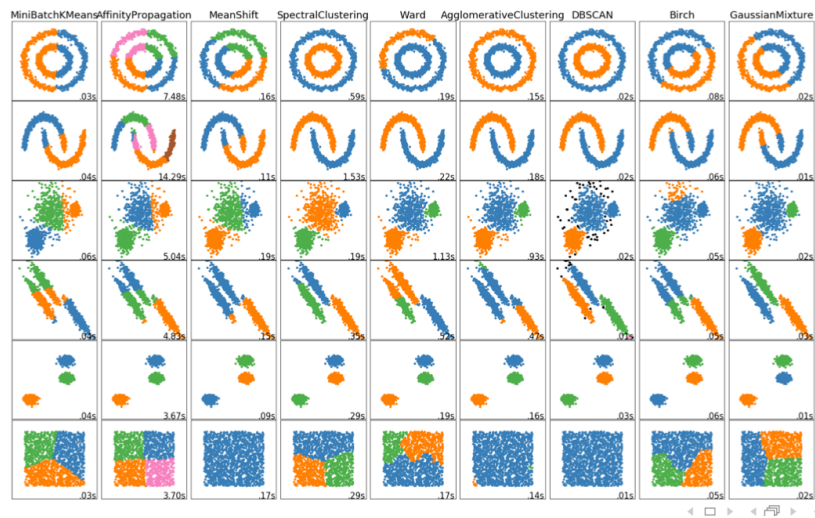


Figure 4.26: Gaussian Mixture and KMeans – Comparison of results

These data have a bimodal, gaussian-like distribution. The EM algorithm is founded on the hypothesis of modelling data with gaussians while KMeans is non-parametric, and in this case the performance is worse.

4.6 Final remarks



Method name	Parameters	Scalability	Usecase	Geometry (metric used)
K-Means	number of clusters	Very large n_{samples} , medium n_{clusters} with MiniBatch code	General-purpose, even cluster size, flat geometry, not too many clusters, inductive	Distances between points
Affinity propagation	damping, sample preference	Not scalable with n_{samples}	Many clusters, uneven cluster size, non-flat geometry, inductive	Graph distance (e.g. nearest-neighbor graph)
Mean-shift	bandwidth	Not scalable with n_{samples}	Many clusters, uneven cluster size, non-flat geometry, inductive	Distances between points
Spectral clustering	number of clusters	Medium n_{samples} , small n_{clusters}	Few clusters, even cluster size, non-flat geometry, transductive	Graph distance (e.g. nearest-neighbor graph)
Ward hierarchical clustering	number of clusters or distance threshold	Large n_{samples} and n_{clusters}	Many clusters, possibly connectivity constraints, transductive	Distances between points

Method name	Parameters	Scalability	Usecase	Geometry (metric used)
Agglomerative clustering	number of clusters or distance threshold, linkage type, distance	Large n_{samples} and n_{clusters}	Many clusters, possibly connectivity constraints, non Euclidean distances, transductive	Any pairwise distance
DBSCAN	neighborhood size	Very large n_{samples} , medium n_{clusters}	Non-flat geometry, uneven cluster sizes, outlier removal, transductive	Distances between nearest points
OPTICS	minimum cluster membership	Very large n_{samples} , large n_{clusters}	Non-flat geometry, uneven cluster sizes, variable cluster density, outlier removal, transductive	Distances between points
Gaussian mixtures	many	Not scalable	Flat geometry, good for density estimation, inductive	Mahalanobis distances to centers
BIRCH	branching factor, threshold, optional global clusterer.	Large n_{clusters} and n_{samples}	Large dataset, outlier removal, data reduction, inductive	Euclidean distance between points

- Effectiveness decreases with
 - dimensionality D
 - noise level
- Computational cost increases with
 - dataset size N , at least linearly
 - dimensionality D

Chapter 5

Association Rules

5.1 Introduction to Market basket Analysis

Given a set of commercial transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction. See the table in figure 5.1.

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Market Basket Transactions

Figure 5.1: Notice that we know what item there is in the basket but not how many

Example of Association Rules:

$$\begin{aligned}\{\text{Diaper}\} &\rightarrow \{\text{Beer}\}, \\ \{\text{Bread, Milk}\} &\rightarrow \{\text{Coke, Eggs}\}, \\ \{\text{Beer, Bread}\} &\rightarrow \{\text{Milk}\}\end{aligned}$$

Implication means co-occurrence, not causality! The implication of Association Rules is different from that of logic (boolean): it can be true with some level of truth.

Some definitions:

- **Itemset**: a collection of one or more items, Example: Bread, Diaper, Milk
- **k-itemset**: an itemset that contains k items
- **Support count** (σ): Frequency of occurrence of an itemset, E.g. $\sigma(\text{Bread, Diaper, Milk})=2$

- **Support:** fraction of transactions that contain an itemset, E.g. $\sigma(\text{Bread, Diaper, Milk})=2/5$
- **Frequent Itemset:** an itemset whose support is greater than or equal to a minsup threshold

An **Association Rule** is An expression of the form $A \Rightarrow C$, where A and C are itemsets.

We define the following rule evaluation metrics:

- **Support:** Fraction of the N transactions that contain both A and C, $sup = \frac{\sigma(\text{Beer,Diaper,Milk})}{N} = \frac{2}{5} = 0.4$
- **Confidence:** Measures how often all the items in C appear in transactions that contain A, $conf = \frac{\sigma(\text{Beer,Diaper,Milk})}{\sigma(\text{Milk,Diaper})}$

The confidence is always greater than the support.

Why support and confidence are useful?

Rules with low support can be generated by random associations. Rules with low confidence are not really reliable. Nevertheless a rule with relatively low support but high confidence can represent an uncommon but interesting phenomenon.

Mining Association Rules

Given a set of transactions N, the goal of association rule mining is to find all rules having (support $\geq$ minsup threshold) and (confidence $\geq$ minconf threshold).

A brute-force approach consist on listing all possible association rules, computing the support and confidence for each rule and finally pruning rules that fail the minsup and minconf thresholds. But this is computationally prohibitive.

Example of Rules:	$\{\text{Diaper, Milk}\} \Rightarrow \{\text{Beer}\}$	$(s = 0.4, c = 0.67)$
	$\{\text{Beer, Milk}\} \Rightarrow \{\text{Diaper}\}$	$(s = 0.4, c = 1.0)$
	$\{\text{Beer, Diaper}\} \Rightarrow \{\text{Milk}\}$	$(s = 0.4, c = 0.67)$
	$\{\text{Beer}\} \Rightarrow \{\text{Diaper, Milk}\}$	$(s = 0.4, c = 0.67)$
	$\{\text{Diaper}\} \Rightarrow \{\text{Beer, Milk}\}$	$(s = 0.4, c = 0.5)$
	$\{\text{Milk}\} \Rightarrow \{\text{Beer, Diaper}\}$	$(s = 0.4, c = 0.5)$

All the rules above are binary partitions of the same itemset Beer,Diaper,Milk. Rules originating from the same itemset have identical support but can have different confidence so we may decouple the support and confidence requirements

Two-step approach:

1. Frequent Itemset Generation: Generate all itemsets whose support is greater than minsup
2. Rule Generation: Generate high confidence rules from each frequent itemset, where each rule is a binary partitioning of a frequent itemset

Frequent itemset generation is still computationally expensive.

5.2 Frequent Itemset Generation

Given D items, there are $M = 2^D$ possible candidate itemsets. See figure 5.2.

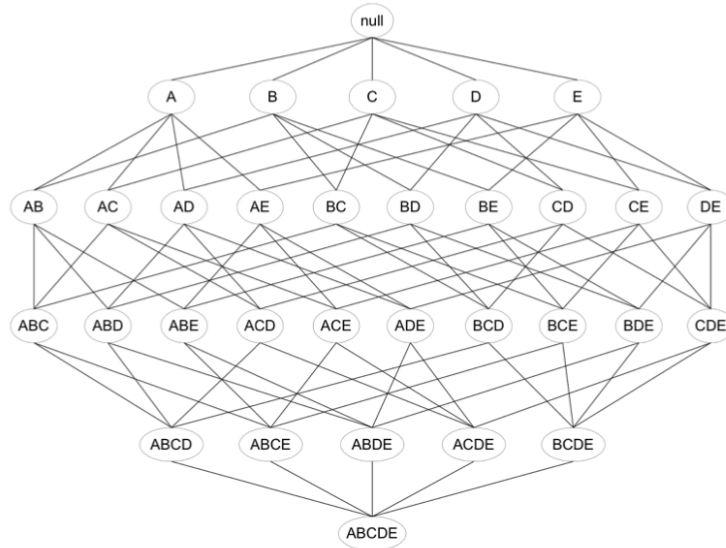


Figure 5.2: The links have a containment meaning

With brute-force approach:

1. each itemset in the lattice is a candidate frequent itemset
2. Count the support of each candidate by scanning the database
3. Match each transaction against every candidate
4. Complexity: $O(NWM) \Rightarrow$ Expensive

<i>TID</i>	<i>Items</i>
1	Bread, Milk
1	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

← W →

Given D unique items the total number of itemsets is 2^D and the total number of possible association rules is

$$R = \sum_{k=1}^{D-1} \left(\binom{D}{k} \times \sum_{j=1}^{D-k} \binom{D-k}{j} \right)$$

$$= 3^D - 2^{D+1} + 1$$

Figure 5.3

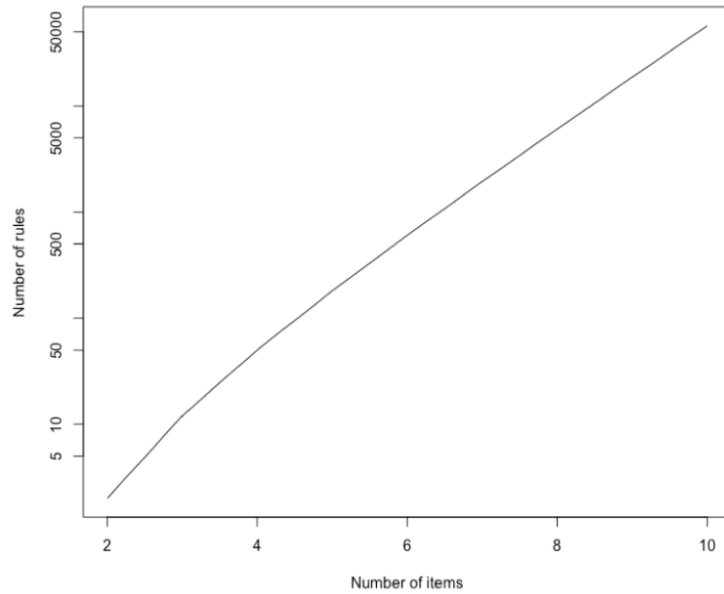


Figure 5.4: Exponential growth, since y axis has log-scale

More on the formula above from figure 5.5

- count the number of ways to create an itemset that forms the left hand side of the rule
- for each size k itemset selected for the left-hand side, count the number of ways to choose the remaining $D - k$ items to form the right-hand side of the rule

Going deeper

- choose k of the D items for the left hand side of the rule, there are $\binom{D}{k}$ ways to do this
- there are $\binom{D-k}{i}$ ways to choose the right hand side of the rule, $1 \leq i \leq D - k$
- the double summation derives from the two points above
- the *binomial theorem* states that $\sum_{i=0}^n \binom{n}{i} x^i = (1 + x)^n$
- using the theorem for $x = 1$ and $x = 2$ leads to the final result (pay attention to the starting value of the summation)

Figure 5.5

Generation Strategies

Reduce the number of candidates M (complete search with $M = 2^D$) with pruning techniques.

Reduce the number of comparisons NM: use efficient data structures to store the candidates or transactions, no need to match every candidate against every transaction

5.2.1 The Apriori principle

If an itemset is frequent, then all of its subsets must also be frequent. It holds due to the following property of the support measure:

$$\forall X, Y : (X \subseteq Y) \Rightarrow \text{sup}(X) \geq \text{sup}(Y) \quad (5.1)$$

It means that the Support of an itemset never exceeds the support of its subsets. This is known as the anti-monotone property of support.

It follows that if an itemset is infrequent we can discard all what is under it. Seeing at the example in figure 5.6, since AB is infrequent than everything below have not to be considered

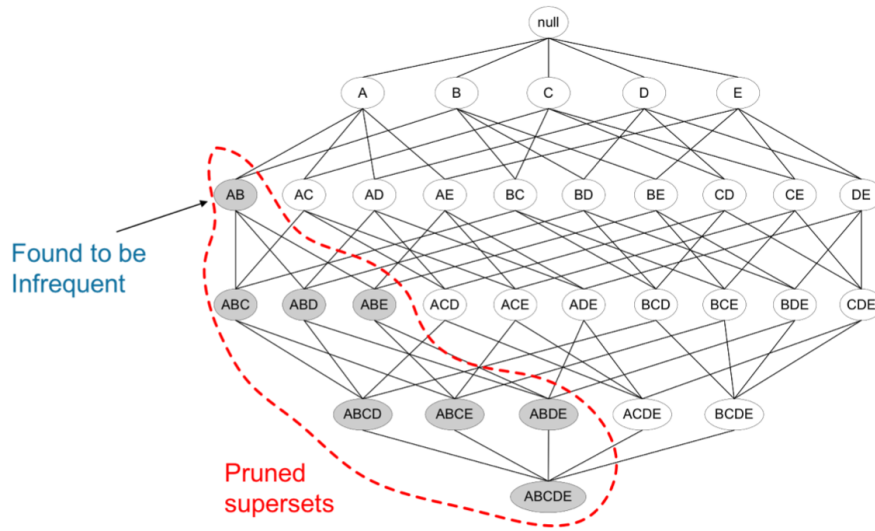


Figure 5.6: Pruning strategy

5.2.2 The Apriori algorithm

Definitions:

- C_k : candidate itemsets of size k
- L_k : frequent itemsets of size k
- $\text{subset}_k(c)$: set of the subsets of c with k elements

Candidate generation

JOIN STEP

- Let L_k be represented as a table with k columns where each row is a frequent itemset
- Let the items in each row of L_k be in lexicographic order

- C_{k+1} is generated by a self join of L_k

```

insert into  $C_{k+1}$ 
select p.item1, p.item2, ... , p.itemk, q.itemk
from  $L_k$  p,  $L_k$  q
where p.item1=q.item1 and ... and p.itemk-1=q.itemk-1
      and p.itemk < q.itemk;

```

PRUNE STEP

Each $(k+1)$ -itemset which includes a k -itemset which is not in L_k is deleted from C_{k+1} .

```

for all  $c \in C_k$  do
  for all  $s \in \text{subset}_{k-1}(c)$  do
    if  $s \notin L_{k-1}$  then
      delete  $c$  from  $C_k$ 
return  $C_k$ 

```

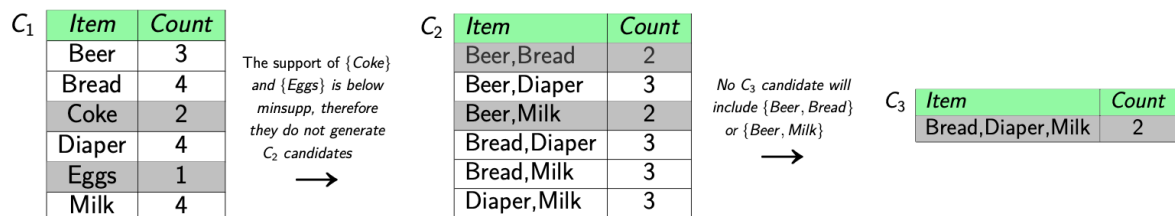
FREQUENT ITEMSET GENERATION

```

 $L_1 \leftarrow$  frequent 1-itemsets
 $k \leftarrow 1$ 
while  $L_k \neq \emptyset$  do
   $C_{k+1} =$  candidates generated from  $L_k$ 
  for all  $t$  transaction in database do
    increment candidate count in  $C_{k+1}$  for candidates found in  $t$ 
   $L_{k+1} \leftarrow \{c\} \in C_{k+1} : \text{sup}(c) \geq \text{minsup}$ 
   $k \leftarrow k + 1$ 
return  $k, L_k$ 

```

Example



Number of itemsets to evaluate:

$$\text{No pruning} = \binom{6}{1} + \binom{6}{2} + \binom{6}{3} = 41$$

$$\text{Support based pruning} = 13$$

5.2.3 Factors Affecting Complexity

Choice of minimum support threshold

Lowering support threshold results in a greater number of frequent itemsets. This may reduce pruning and increase the maximum length of frequent itemsets. the number of complete reads of the dataset is given by the maximum length of frequent itemsets plus one.

Dimensionality (number of items) of the data set

More space is needed to store support count of each item. If number of frequent items also increases, both computation and I/O costs may also increase.

Size of database

Since Apriori makes multiple passes, run time of algorithm may increase with number of transactions

Average transaction width

Transaction width increases with denser data sets. This may increase max length of frequent itemsets and traversals of data structures (number of subsets in a transaction increases with its width).

5.3 Rule Generation

The **confidence** of a rule can be computed from the supports; for confidence based pruning of rules it is sufficient to know the supports of frequent itemsets $conf(A \Rightarrow C) = \frac{sup(A \Rightarrow C)}{sup(A)}$.

Give a frequent itemset L:

- find all the non-empty subsets $f \in L$ such that the confidence of rule $f \Rightarrow (L - f)$ is not less than the minimum confidence (set by the experiment designer).

from $\{Beer, Diaper, Milk\}$ the possible rules are
 $Beer, Diaper \Rightarrow Milk$, $Beer \Rightarrow Diaper, Milk$,
 $Beer, Milk \Rightarrow Diaper$, $Milk \Rightarrow Beer, Diaper$,
 $Diaper, Milk \Rightarrow Beer$, $Diaper \Rightarrow Beer, Milk$

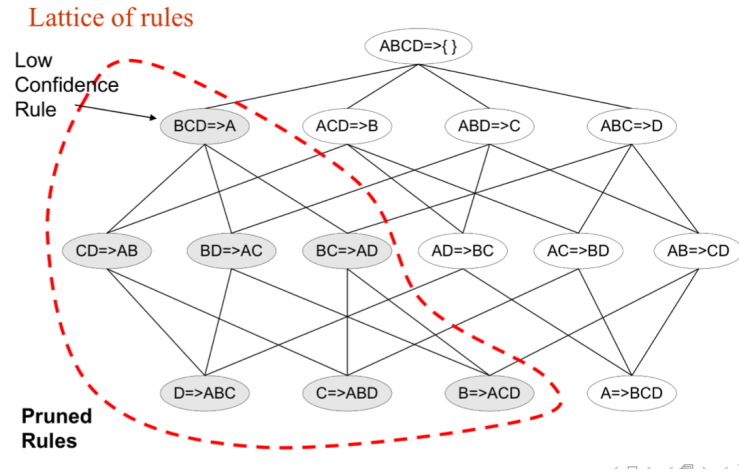
- if $|L| = k$ then there are $2^k - 2$ candidate rules: $L \Rightarrow \emptyset$ and $\emptyset \Rightarrow L$ can be ignored

How to efficiently generate rules from frequent itemsets?

In general, confidence does not have an anti-monotone property,

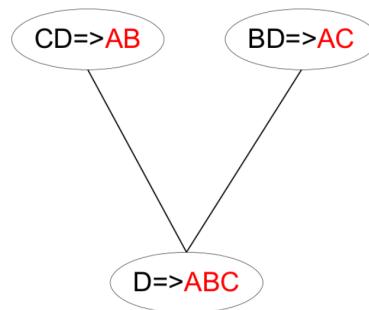
$$conf(ABC \rightarrow D) \geq conf(AB \rightarrow CD) \geq conf(A \rightarrow BCD) \quad (5.2)$$

Confidence of rules generated from the same itemset is anti-monotone w.r.t. the number of items on the RHS of the rule, i.e. it decreases when we move an item from the left hand to the right hand.



Rule Generation in Apriori

Candidate rule is generated by merging two rules that share the same prefix in the rule consequent. $\text{join}(CD \Rightarrow AB, BD \Rightarrow AC)$ would produce the candidate rule $D \Rightarrow ABC$. Then prune rule $D \Rightarrow ABC$ if its subset $AD \Rightarrow BC$ does not have high confidence.



5.3.1 Pattern evaluation

Association rule algorithms tend to produce too many rules, but many of them are uninteresting or redundant. Redundant if $A, B, C \Rightarrow D$ and $A, B \Rightarrow D$ have same support and confidence. Interestingness measures can be used to prune/rank the derived patterns. In the original formulation of association rules, support and confidence are the only measures used.

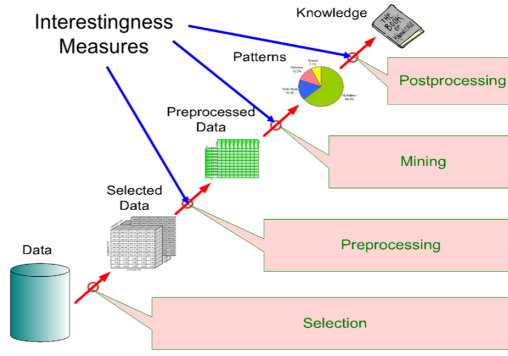


Figure 5.7: Application of Interestingness Measure

Computing Interestingness Measures

Given a rule $A \Rightarrow C$, the information needed to compute rule interestingness can be obtained from a contingency table. The elements of the contingency table are the basis for most of the interestingness measures.

	C	$\overline{C}$	
A	f_{11}	f_{10}	f_{1+}
$\overline{A}$	f_{01}	f_{00}	f_{0+}
	f_{+1}	f_{+0}	

Figure 5.8: Contingency table

Drawback of Confidence

If we compute $conf(Tea \Rightarrow Coffee) = \frac{sup(Tea, Coffee)}{sup(Tea)} = \frac{15}{20} = 0.75$, this is fairly high

	<i>Coffee</i>	$\overline{Coffee}$	
<i>Tea</i>	15	5	20
$\overline{Tea}$	75	5	80
	90	10	100

$Tea \Rightarrow Coffee$

Figure 5.9

From the table in figure 5.9 we have $P(Coffe) = 0.9$ and $Pr(Coffee|\neg Tea) = \frac{75}{80} = 0.9375$; despite the high confidence of $Tea \Rightarrow Coffee$, the absence of Tea increases the probability of $Coffee$. For this rule the confidence is misleading.

5.3.2 Statistical-based Measures

We know that, given

Population of 1000 students

- 600 students know how to swim (S)
- 700 students know how to bike (B)
- 420 students know how to swim and bike (S,B)

- $\Pr(S \wedge B) = \frac{420}{1000} = 0.42$
- $\Pr(S) * P(B) = 0.6 * 0.7 = 0.42$

- $\Pr(S \wedge B) = P(S) * P(B) \Rightarrow$ **Statistical independence**
- $\Pr(S \wedge B) > P(S) * P(B) \Rightarrow$ **Positively correlated**
- $\Pr(S \wedge B) < P(S) * P(B) \Rightarrow$ **Negatively correlated**

Statistical-based measures are measures that take into account the deviation from statistical independence

Lift

$$lift(A \Rightarrow C) = \frac{conf(A \Rightarrow C)}{sup(C)} = \frac{\Pr(A, C)}{\Pr(A) \Pr(C)}$$

It evaluates to 1 for independence. It is insensitive to rule direction. It is the ratio of true cases w.r.t. independence.

Leverage

$$\begin{aligned} leve(A \Rightarrow C) &= \Pr(A, C) - \Pr(A) * \Pr(C) \\ &= sup(A \cup C) - sup(A)sup(C) \end{aligned}$$

It evaluates to 0 for independence. It is insensitive to rule direction. It is the number of additional cases w.r.t. independence.

Conviction (or novelty)

$$conv(A \Rightarrow C) = \frac{1 - sup(C)}{1 - conf(A \Rightarrow C)} = \frac{\Pr(A) (1 - \Pr(C))}{\Pr(A) - \Pr(A, C)}$$

It is infinite if the rule is always true. It is sensitive to rule direction. It is the ratio of the expected frequency that A occurs without C (that is to say, the frequency that the rule makes an incorrect prediction) if A and C were independent divided by the observed frequency of incorrect predictions.

Intuition about measures

- higher support $\Rightarrow$ rule applies to more records
- higher confidence $\Rightarrow$ chance that the rule is true for some record is higher

- higher lift $\Rightarrow$ chance that the rule is just a coincidence is lower
- higher conviction $\Rightarrow$ the rule is violated less often than it would be if the antecedent and the consequent were independent

Properties of a Good Measure – Piatetsky-Shapiro

Three properties a good measure M must satisfy:

1. $M(A,B) = 0$ (or 1) if A and B are statistically independent
2. $M(A,B)$ increases monotonically with $\Pr(A,B)$ when $\Pr(A)$ and $\Pr(B)$ remain unchanged
3. $M(A,B)$ decreases monotonically with $\Pr(A)$ (or $\Pr(B)$) when $\Pr(A,B)$ and $\Pr(B)$ (or $\Pr(A)$) remain unchanged

Examples

Tea $\Rightarrow$ Coffee

$$conf = \frac{0.15}{0.20} = 0.75$$

in a 0 to 1 scale it is apparently high

$$lift = \frac{0.15}{0.90 * 0.20} = 0.83$$

is less than 1, therefore not interesting

$$leve = 0.15 - 0.90 * 0.20 = -0.03$$

is less than 0, therefore not interesting

$$conv = \frac{1-0.9}{1-0.75} = 0.4$$

is low, remembering that absolute truth gives infinite

	C1	$\overline{C1}$	
A1	88	5	93
$\overline{A1}$	5	2	7
	93	7	100

Rule ($A1 \Rightarrow C1$)

$$\begin{aligned} conf &= 0.88/0.93 &= 0.946 \\ lift &= 0.88/(0.93 * 0.93) &= 1.017 \\ leve &= 0.88 - 0.93 * 0.93 &= 0.015 \\ conv &= (1 - 0.93)/(1 - 0.946) &= 1.302 \end{aligned}$$

A high confidence rule can have small lift if both sides are very frequent

	C2	$\overline{C2}$	
A2	2	5	7
$\overline{A2}$	5	88	93
	7	93	100

Rule ($A2 \Rightarrow C2$)

$$\begin{aligned} conf &= 0.02/0.07 &= 0.286 \\ lift &= 0.02/(0.07 * 0.07) &= 4.082 \\ leve &= 0.02 - 0.07 * 0.07 &= 0.015 \\ conv &= (1 - 0.07)/(1 - 0.286) &= 1.302 \end{aligned}$$

A low confidence rule can have high lift if both sides are very infrequent

5.4 Multidimensional association rules

Let's consider a dataset deriving from sensors measuring the concentration of air pollutants.

<i>TID</i>	<i>CO</i>	<i>Tin_Oxide</i>	<i>Titanium</i>
1	high	medium	high
2	medium	low	medium
3	medium	high	low
4	low	medium	medium

Figure 5.10

Look for rules such as CO = high and Tin Oxide = high then Titanium = high (support 0.25 and confidence 1). This can be used for example, if one of the sensor is not available, to guess its qualitative value given the others. Useful for a qualitative analysis, in substitution of regression.

Comparison mono– vs multi–dimensional

- Mono–dimensional (intra–attribute):
 - event: transaction (in this case we can use mono–)
 - event description: items A, B, and C are together in a transaction
- Multi–dimensional (inter–attribute):
 - event: tuple
 - event description: attribute A has value a, attribute B has value b and attribute C has value c in a tuple

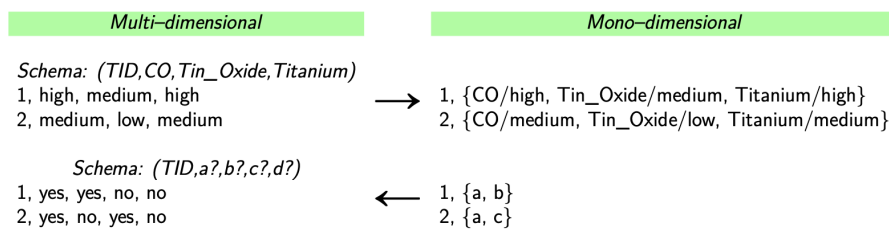


Figure 5.11: Equivalence mono/multi–dimensional

Quantitative attributes

Consider the table below.

<i>TID</i>	<i>CO</i>	<i>Tin Oxide</i>	<i>Titanium</i>
1	2.6	1360	1046
2	2.0	1292	955
3	2.2	1402	939
4	1.6	1376	948

There are too many distinct values for the multi/mono transformation. Most software packages for association rules discovery do not deal with quantitative attributes, thus the solution is discretization. Possibly equifrequency or with mono-dimensional clustering, for optimal covering of the original value domains. Discretisation leads to a dataset like that of table in figure 5.10.